

Codeigniter4 - Webservice Client

Pada tutorial sebelumnya telah berhasil membuat keranjang belanja dengan menerapkan library external. Proses selanjutnya setelah user selesai memilih-milih produk adalah, user melakukan checkout untuk mengkonfirmasi data-data yang diperlukan pada proses pengiriman produk. Pada kegiatan jual beli online, biasanya toko akan menambahkan biaya pengiriman (ongkir) pada nominal total belanja yang harus dibayar oleh pembeli sesuai dengan berat produk, lokasi tujuan pengiriman, dan layanan apa yang dipilih oleh pembeli. Sehingga pada project harus terdapat data detail ongkir itu untuk keperluan perhitungan pada saat pembeli melakukan checkout.

Checkout
Home / Checkout

Checkout

Nama

aprilns

Alamat

Kelurahan

BANJARDOWO, GENUK, SEMARANG, JAWA TENGAH, 50117

Layanan

JNE City Courier (CTCYES) : estimasi 1 day

Ongkir

Kelurahan

BANJARDOWO, GENUK, SEMARANG, JAWA TENGAH, 50117

Layanan

JNE City Courier (CTCYES) : estimasi 1 day

Ongkir

11000

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	1	IDR 10,899,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
		Subtotal	IDR 17,798,000
		Total	IDR 17,809,000

Buat Pesanan

Total	IDR 17,809,000
-------	----------------

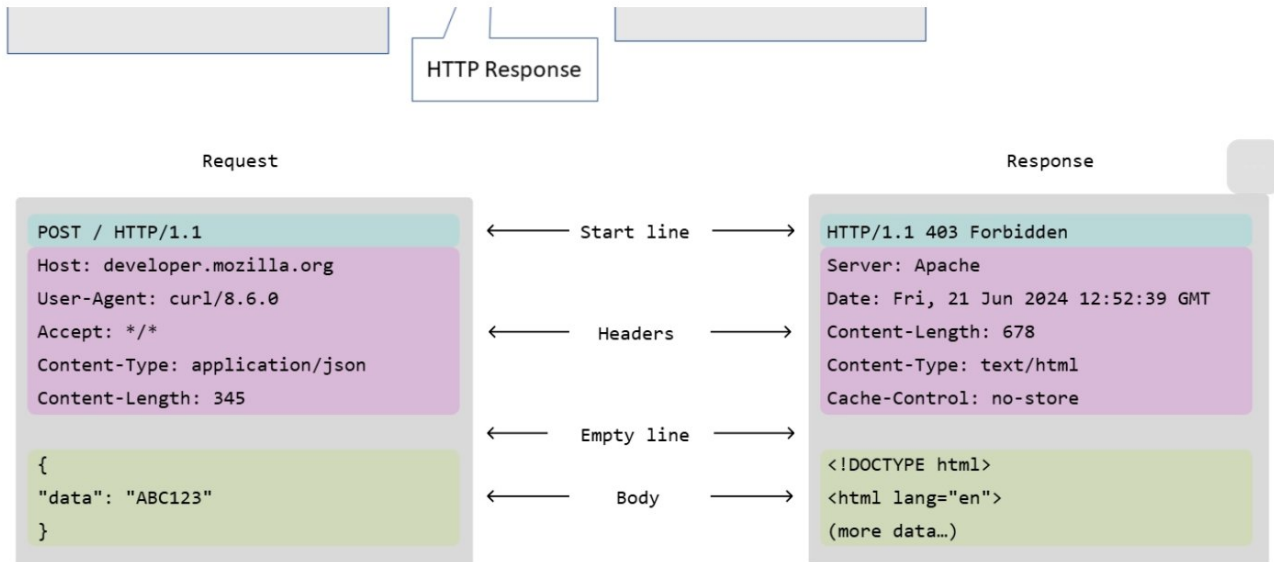
Buat Pesanan

Untuk menggunakan sebuah data tertentu pada proses bisnisnya, project tidak harus menyediakan data itu sendiri. Data bisa juga didapatkan dari luar project menggunakan Webservice. Webservice adalah **sebuah bagian atau jenis dari API (Application Programming Interface)**. Semua webservice adalah API, tetapi tidak semua API adalah webservice. Webservice diakses melalui jaringan internet.

Fungsi utama **Webservice** adalah untuk memungkinkan komunikasi dan pertukaran data antara berbagai aplikasi melalui jaringan (terutama internet), meskipun aplikasi-aplikasi tersebut dibangun

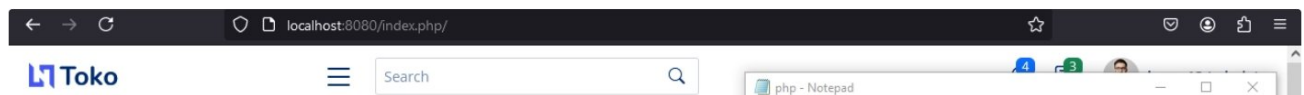
dengan bahasa pemrograman atau platform yang berbeda.

Webservice mampu menukar data tanpa memandang sumber database, bahasa yang digunakan, dan pada platform apa data tersebut dikonsumsi. Kemampuan itulah yang memungkinkan web service menjadi jembatan penghubung untuk berbagai sistem.

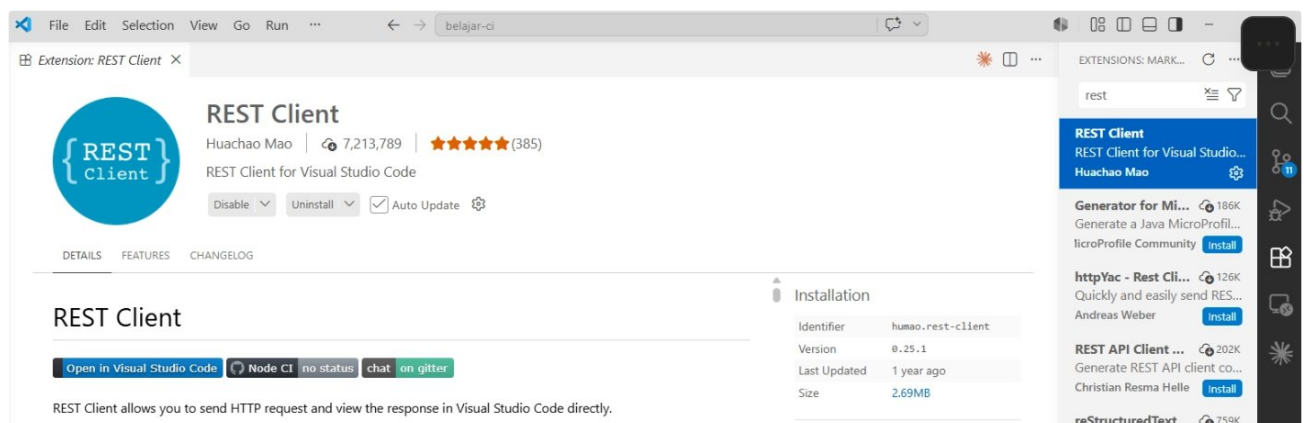


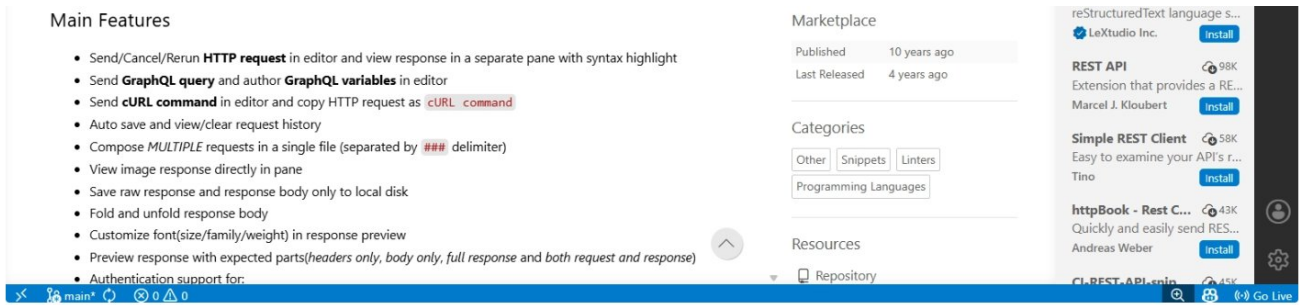
Salah satu library yang bisa digunakan untuk melakukan HTTP Request di PHP adalah **CURL**.

Cek settingan di xampp apakah **extension curl** sudah diaktifkan atau belum, caranya sama seperti saat mengaktifkan extension intl di tutorial awal, lalu restart xampp.



Untuk mempermudah saat nanti uji coba membuat HTTP request ke webservice server dengan jenis REST, tambahkan extension REST Client pada VS Code.



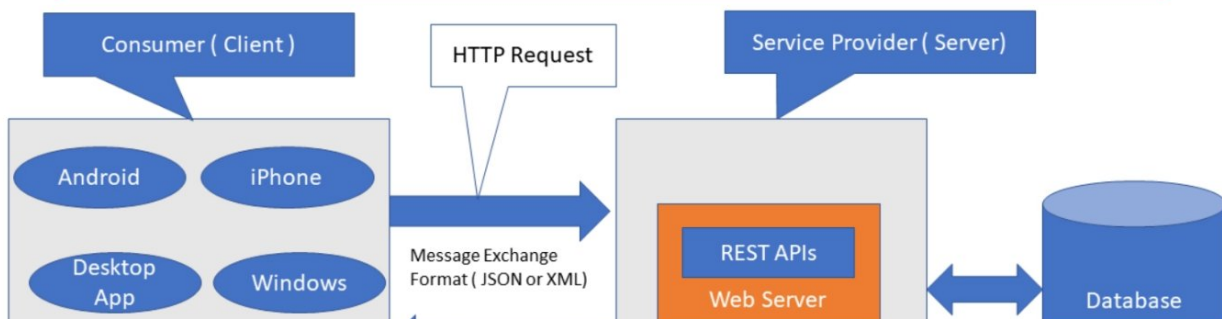


Fungsi utama **Webservice** adalah untuk memungkinkan komunikasi dan pertukaran data antara berbagai aplikasi melalui jaringan (terutama internet), meskipun aplikasi-aplikasi tersebut dibangun dengan bahasa pemrograman atau platform yang berbeda.

Webservice mampu menukar data tanpa memandang sumber database, bahasa yang digunakan, dan pada platform apa data tersebut dikonsumsi. Kemampuan itulah yang memungkinkan web service menjadi jembatan penghubung untuk berbagai sistem.

Salah satu jenis webservice yang sering digunakan adalah REST (Representational State Transfer) karena lebih ringan dan cepat, menggunakan format data seperti JSON atau XML, mengandalkan metode HTTP (GET, POST, PUT, DELETE).

REST – Architecture



external. Proses selanjutnya setelah user selesai memilih-milih produk adalah, user melakukan checkout untuk mengkonfirmasi data-data yang diperlukan pada proses pengiriman produk. Pada kegiatan jual beli online, biasanya toko akan menambahkan biaya pengiriman (ongkir) pada nominal total belanja yang harus dibayar oleh pembeli sesuai dengan berat produk, lokasi tujuan pengiriman, dan layanan apa yang dipilih oleh pembeli. Sehingga pada project harus terdapat data detail ongkir itu untuk keperluan perhitungan pada saat pembeli melakukan checkout.

Checkout

Home / Checkout

Checkout

Nama

aprilns

Alamat

Kelurahan

BANJARDOWO, GENUK, SEMARANG, JAWA TENGAH, 50117

Layanan

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	1	IDR 10,899,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
		Subtotal	IDR 17,798,000
		Total	IDR 17,809,000

Buat Pesanan

JNE City Courier (CTCYES) : estimasi 1 day

Ongkir

11000

Untuk menggunakan sebuah data tertentu pada proses bisnisnya, project tidak harus menyediakan data itu sendiri. Data bisa juga didapatkan dari luar project menggunakan

Codeigniter4 - Webservice Client

Pada tutorial sebelumnya telah berhasil membuat keranjang belanja dengan menerapkan library external. Proses selanjutnya setelah user selesai memilih-milih produk adalah, user melakukan checkout untuk mengkonfirmasi data-data yang diperlukan pada proses pengiriman produk. Pada kegiatan jual beli online, biasanya toko akan menambahkan biaya pengiriman (ongkir) pada nominal total belanja yang harus dibayar oleh pembeli sesuai dengan berat produk, lokasi tujuan pengiriman, dan layanan apa yang dipilih oleh pembeli. Sehingga pada project harus terdapat data detail ongkir itu untuk keperluan perhitungan pada saat pembeli melakukan checkout.

Checkout

Home / Checkout

Checkout

Nama

aprilns

Alamat

Kelurahan

BANJARDOWO, GENUK, SEMARANG, JAWA TENGAH, 50117

Layanan

JNE City Courier (CTCYES) : estimasi 1 day

Ongkir

11000

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	1	IDR 10,899,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
		Subtotal	IDR 17,798,000
		Total	IDR 17,809,000

Buat Pesanan

Untuk menggunakan sebuah data tertentu pada proses bisnisnya, project tidak harus menyediakan data itu sendiri. Data bisa juga didapatkan dari luar project menggunakan Webservice. Webservice adalah **sebuah bagian atau jenis dari API (Application Programming Interface)**. Semua webservice adalah API, tetapi tidak semua API adalah webservice. Webservice

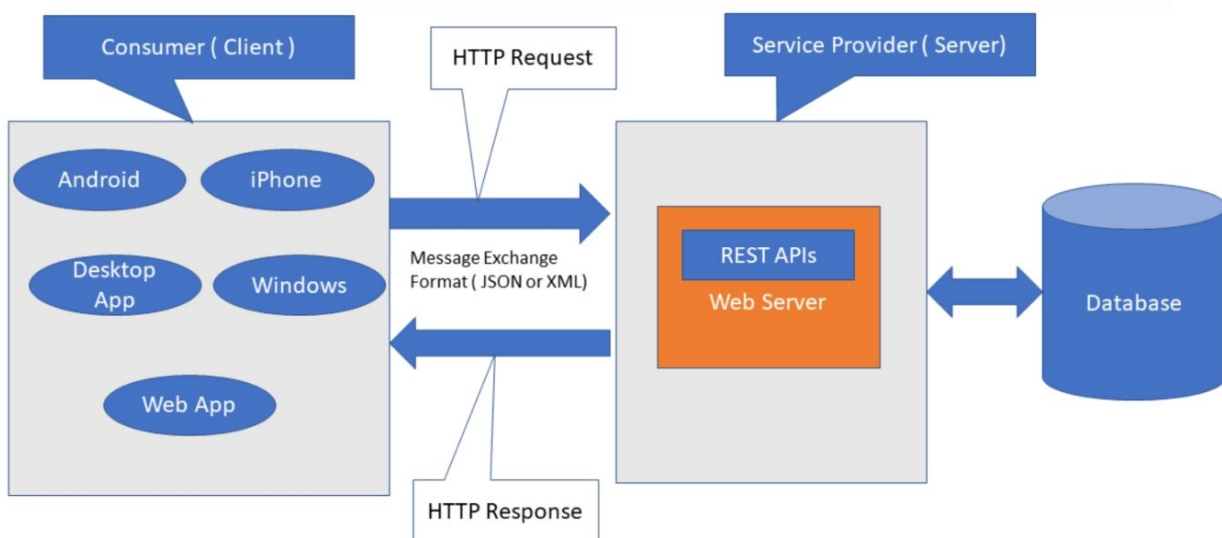
diakses melalui jaringan internet.

Fungsi utama **Webservice** adalah untuk memungkinkan komunikasi dan pertukaran data antara berbagai aplikasi melalui jaringan (terutama internet), meskipun aplikasi-aplikasi tersebut dibangun dengan bahasa pemrograman atau platform yang berbeda.

Webservice mampu menukar data tanpa memandang sumber database, bahasa yang digunakan, dan pada platform apa data tersebut dikonsumsi. Kemampuan itulah yang memungkinkan web service menjadi jembatan penghubung untuk berbagai sistem.

Salah satu jenis webservice yang sering digunakan adalah REST (Representational State Transfer) karena lebih ringan dan cepat, menggunakan format data seperti JSON atau XML, mengandalkan metode HTTP (GET, POST, PUT, DELETE).

REST – Architecture



Request

```
POST / HTTP/1.1
Host: developer.mozilla.org
User-Agent: curl/8.6.0
Accept: */*
Content-Type: application/json
Content-Length: 345

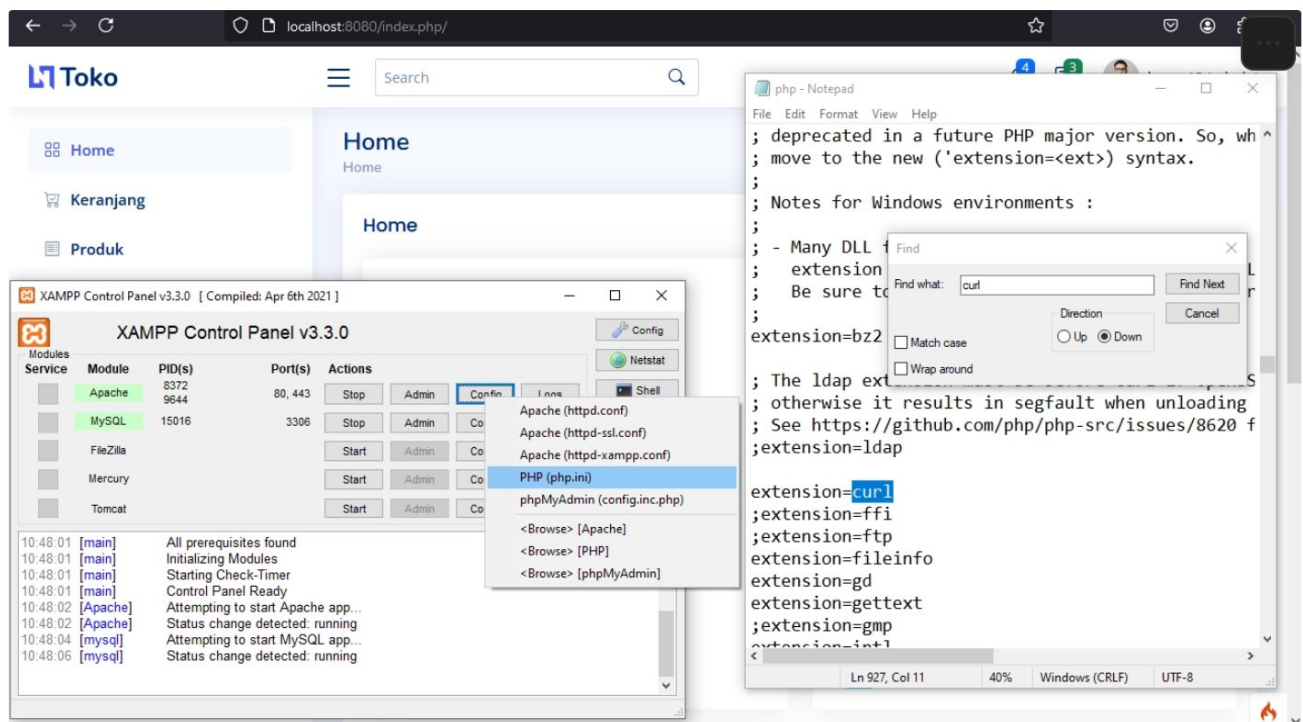
{
  "data": "ABC123"
}
```

Response

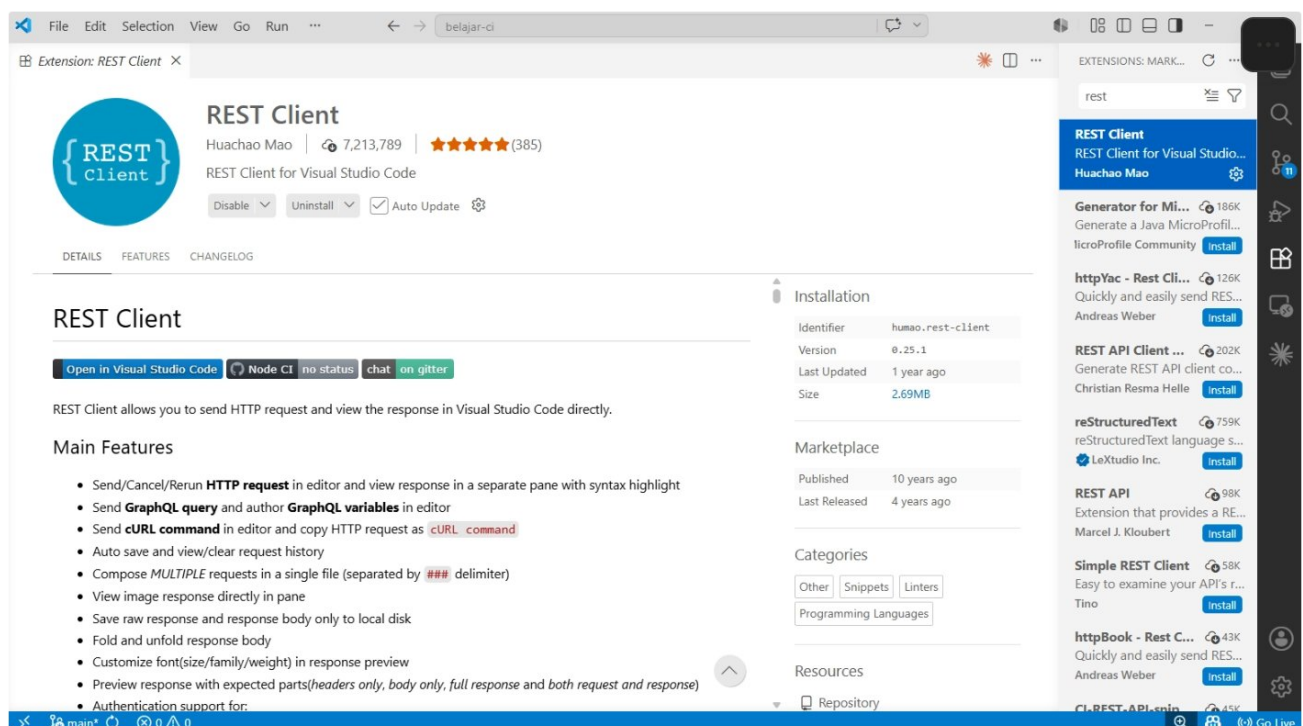
```
HTTP/1.1 403 Forbidden
Server: Apache
Date: Fri, 21 Jun 2024 12:52:39 GMT
Content-Length: 678
Content-Type: text/html
Cache-Control: no-store

<!DOCTYPE html>
<html lang="en">
(more data...)
```

Salah satu library yang bisa digunakan untuk melakukan HTTP Request di PHP adalah **CURL**. Cek settingan di xampp apakah **extension curl** sudah diaktifkan atau belum, caranya sama seperti saat mengaktifkan extension intl di tutorial awal, lalu restart xampp.



Untuk mempermudah saat nanti uji coba membuat HTTP request ke webservice server dengan jenis REST, tambahkan extension REST Client pada VS Code.



Pada kasus kali ini, project Toko akan menggunakan data yang disediakan oleh RajaOngkir. Sehingga bisa disebut project Toko adalah sebagai Webservice Client dan RajaOngkir sebagai Webservice Servernya.

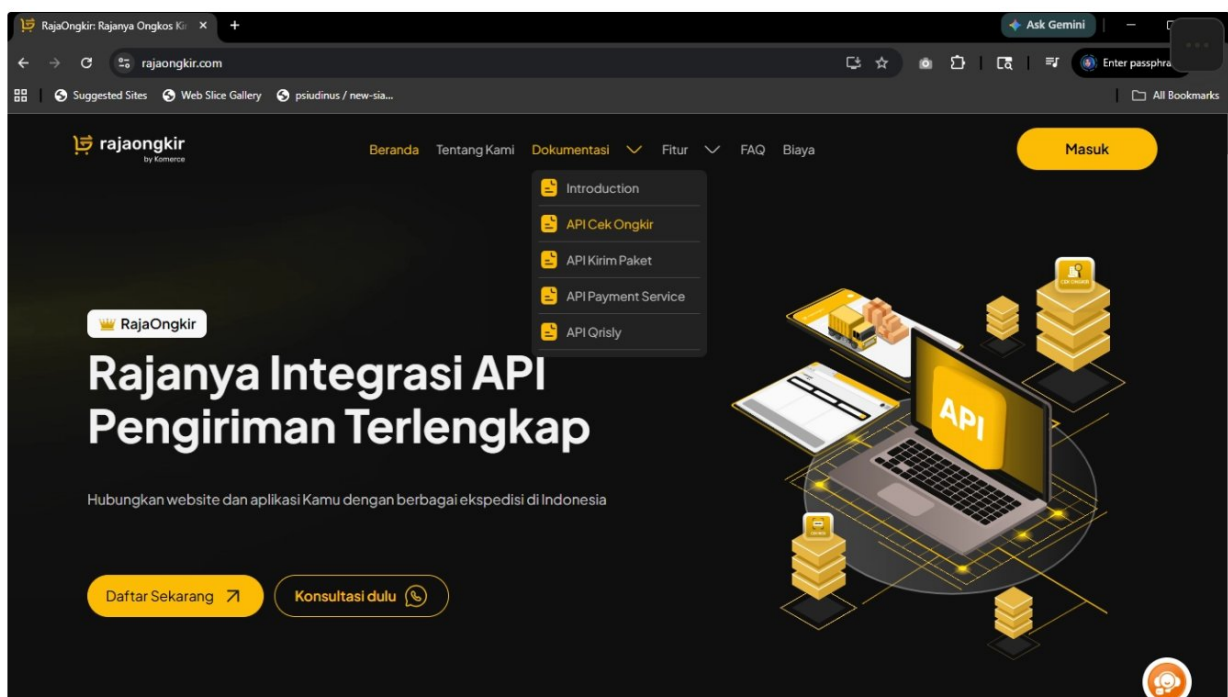
1. Siapkan project, lakukan clone project (git clone) ***jika membutuhkan**. Langkah-langkah untuk clone silakan lihat tutorial sebelumnya.

 [Clone GitHub Repository](#)

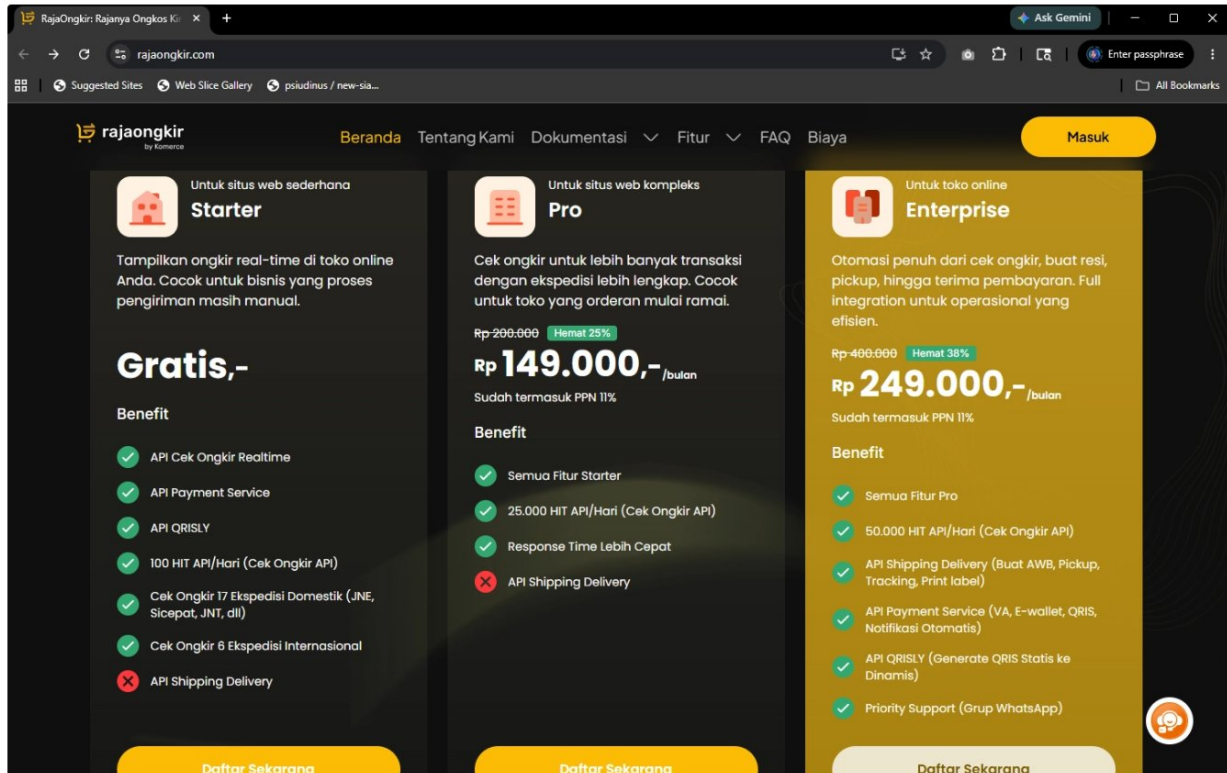
2. Sebelum coding atau melakukan perubahan pada project, dibiasakan melakukan **git pull** dulu pada terminal project di VS Code supaya mendapatkan isi project yang paling up-to-date, terutama bagi yang melakukan coding di lebih dari satu perangkat (misalnya di laptop pribadi dan di komputer lab kampus). Perintahnya

```
git pull origin main
```

3. Seperti yang sudah disebutkan sebelumnya, untuk menyelesaikan proses checkout, perlu menambahkan nominal ongkir sebagai pelengkap perhitungan. Salah satu penyedia data ongkir adalah **RajaOngkir**, mereka menyediakan **webservice** yang dokumentasinya bisa dilihat di websitenya (<https://rajaongkir.com/>) pada menu Dokumentasi - API Cek Ongkir (perhatikan gambar).



Hal yang perlu diperhatikan adalah, RajaOngkir memiliki kategori layanan yang bisa dipilih, untuk project kali ini akan menggunakan layanan yang gratis yaitu Starter sehingga memiliki kekurangan salah satunya adalah '100 HIT API/Hari'. Yang artinya, sistem hanya bisa menerima permintaan (request) data maksimal 100 kali per hari.



Dalam penggunaan Webservice dari RajaOngkir untuk mengambil data ongkir ini, project Toko yang akan menjadi **pengguna data (Webservice Client)** dan pihak RajaOngkir yang menjadi **pemilik dan penyedia data (Webservice Server)**.

Langkah-langkah penggunaan RajaOngkir API.

Step 1: Register for an API Key

- Membuat akun dan mendapatkan API Key (Shipping Cost)

Step 2: Understand API Endpoints

- Search Domestics Destination Endpoint :
Untuk mendapatkan data daerah asal dan tujuan pengiriman di area domestic.
- Cost Domestics Endpoint :
Untuk mendapatkan jenis layanan dan biaya pengiriman antara dua daerah di Indonesia (Berdasarkan ID daerah).

Step 3: Make Your First API Request

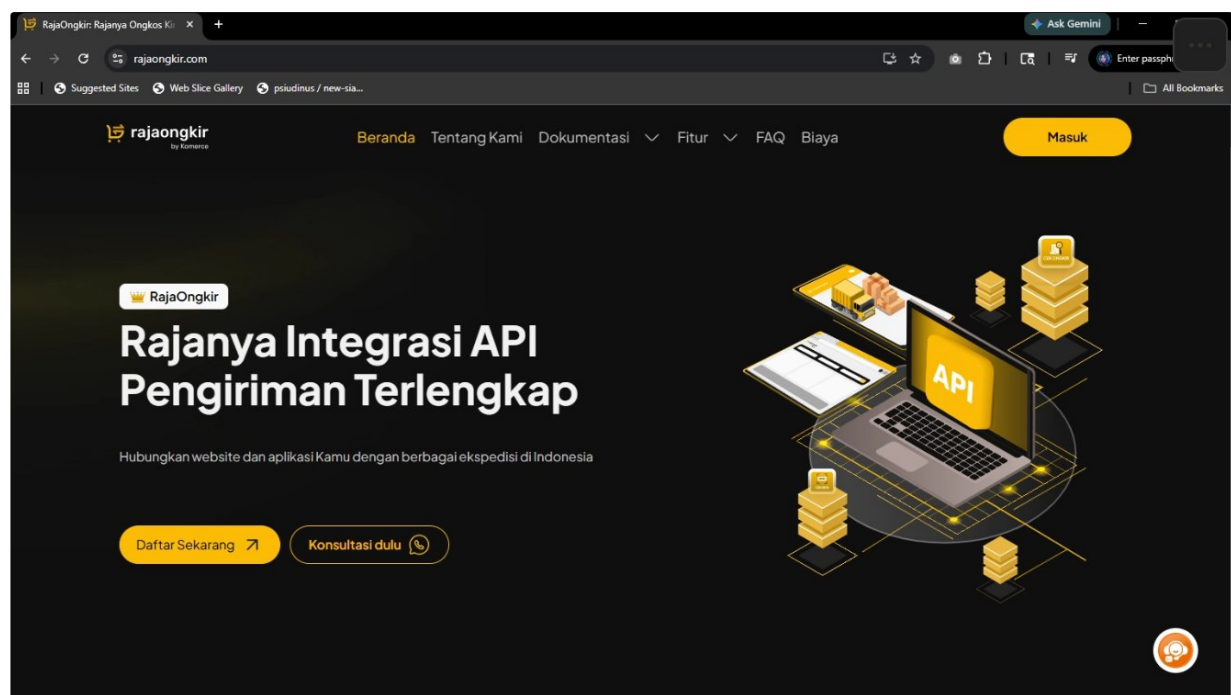
- Melakukan HTTP request ke endpoint RajaOngkir dengan menyertakan API Key.

Step 4: Handle API Responses

- Mengelola JSON hasil response dari endpoint untuk ditampilkan atau digunakan dalam project.

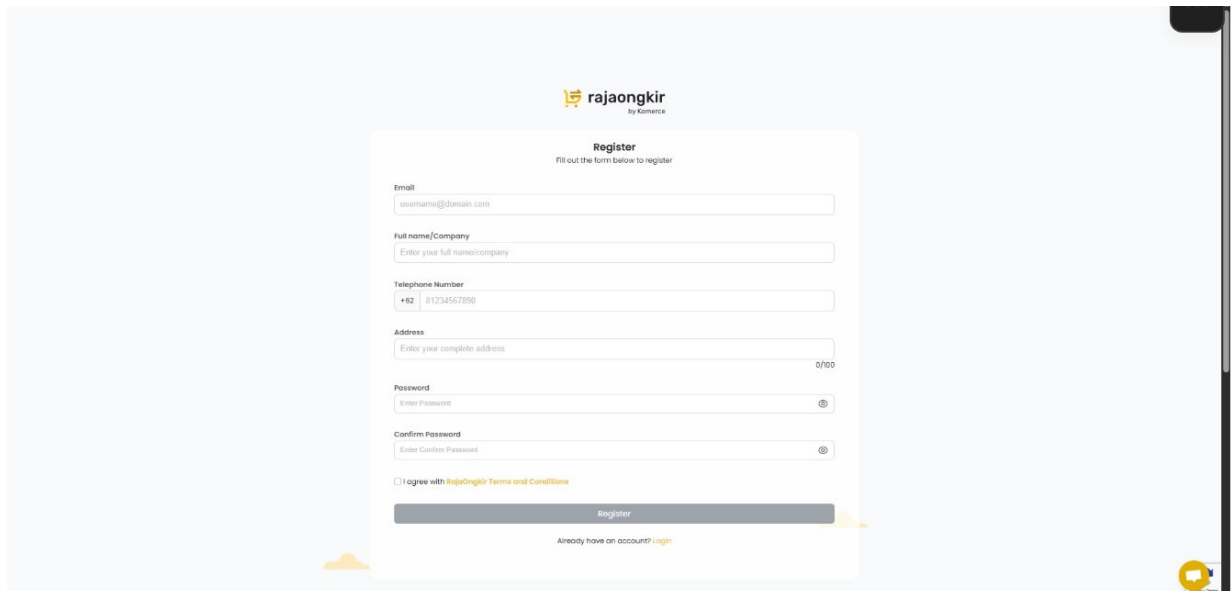
Lakukan Step 1, RajaOngkir mewajibkan client menyertakan API Key untuk setiap request data yang dilakukan. Sehingga, buat dulu akunnya (Register) lalu lakukan Log In.

<https://rajaongkir.com/>



<https://collaborator.kommerce.id/registration>





The image shows a registration form for Rajaongkir, a service by Komerce. The form is titled "Register" and includes fields for Email, Full name/Company, Telephone Number, Address, Password, and Confirm Password. There is a checkbox for "I agree with Rajaongkir Terms and Conditions" and a "Register" button. A link for "Already have an account? login" is also present.

Register
Fill out the form below to register

Email
username@domain.com

Full name/Company
Enter your full name/company

Telephone Number
+62 81234567890

Address
Enter your complete address 0700

Password
Enter Password

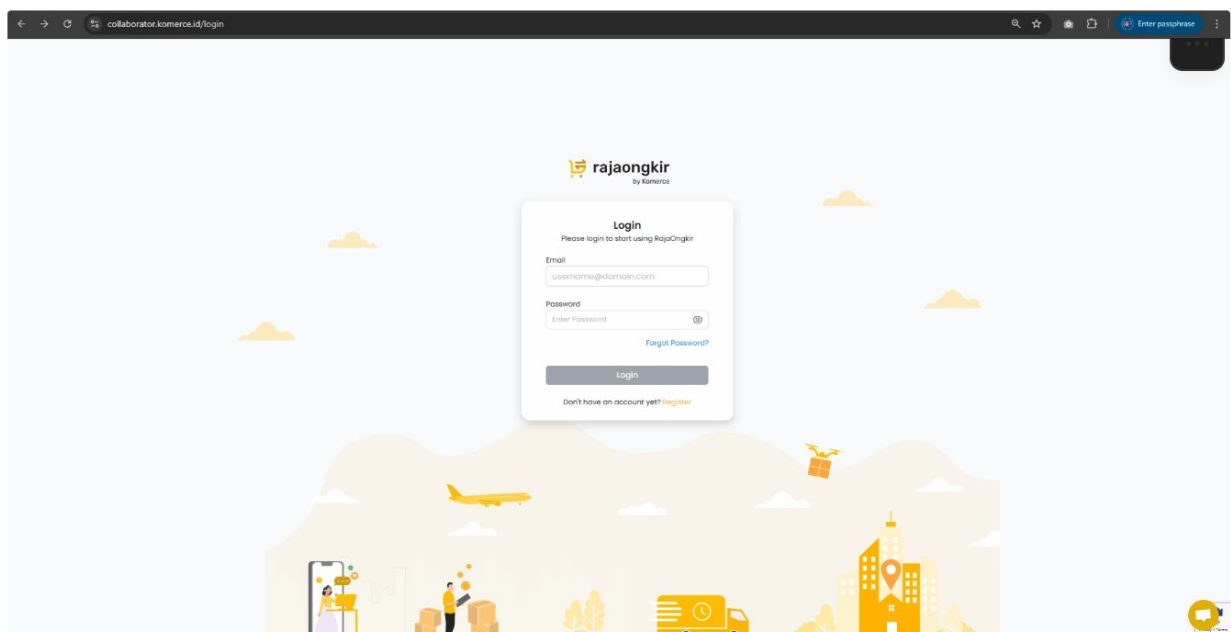
Confirm Password
Enter Confirm Password

☐ I agree with Rajaongkir Terms and Conditions

Register

Already have an account? [login](#)

<https://collaborator.komerce.id/login>



The image shows a login form for Rajaongkir, a service by Komerce. The form is titled "Login" and includes fields for Email and Password. There is a link for "Forgot Password?" and a "Login" button. A link for "Don't have an account yet? Register" is also present. The background features a stylized illustration of a cityscape with a plane and a truck.

Login
Please login to start using Rajaongkir

Email
username@domain.com

Password
Enter Password

[Forgot Password?](#)

Login

Don't have an account yet? [Register](#)

Setelah masuk pada halaman Dashboard, pada sidebar menu silakan pilih Developer - Settings, lalu pilih tab API Key.



The image shows the "Settings" page for the Rajaongkir Collaborator API. The page has a sidebar menu on the left with options like "Developer - Settings" and "API Key". The main content area displays the "API Key" tab, which shows a generated API key and a "Copy" button. The page also includes a header with the Rajaongkir logo and a footer with the Komerce logo and a user profile.

Rajaongkir - Collaborator API

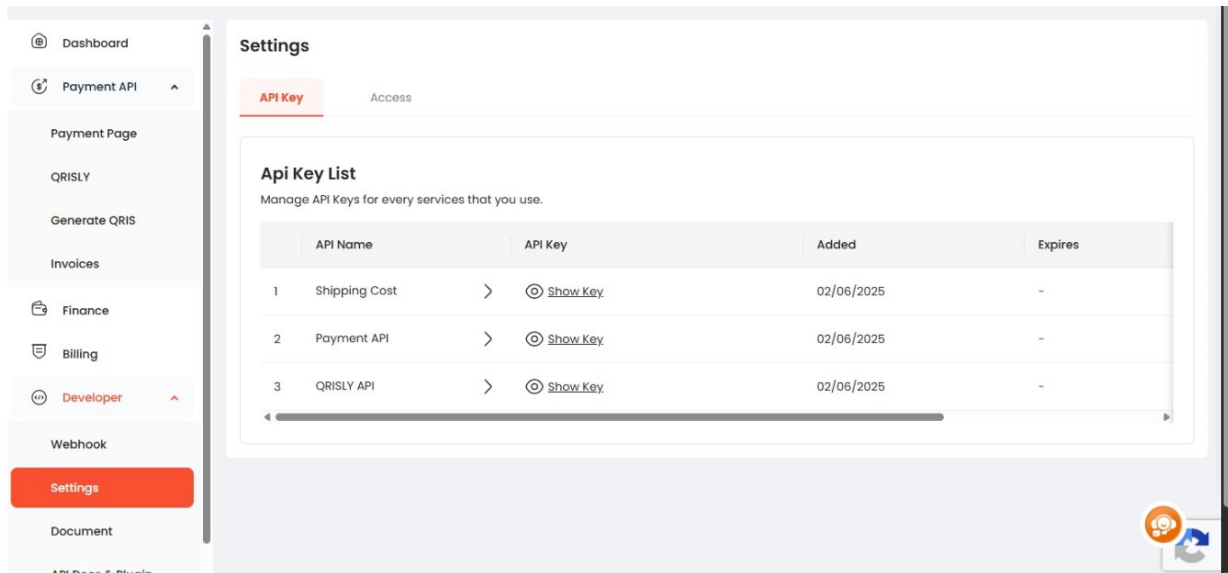
collaborator.komerce.id/settings

Developer - Settings

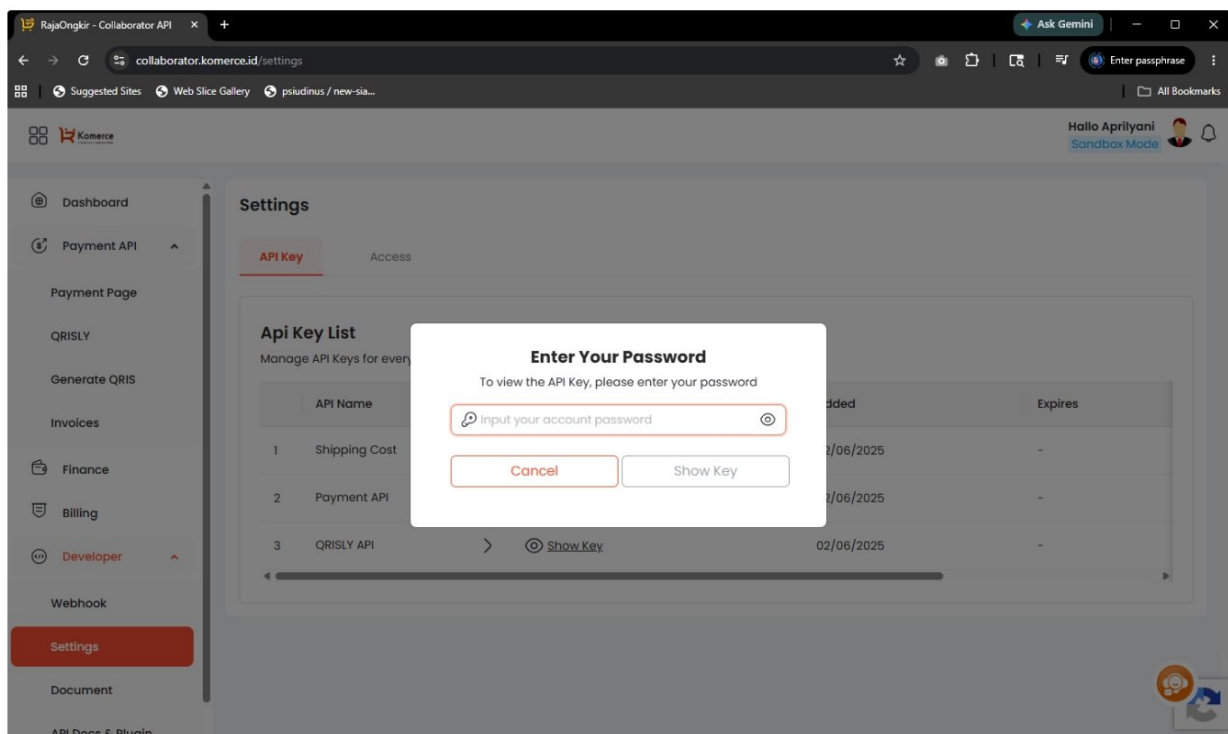
API Key

Copy

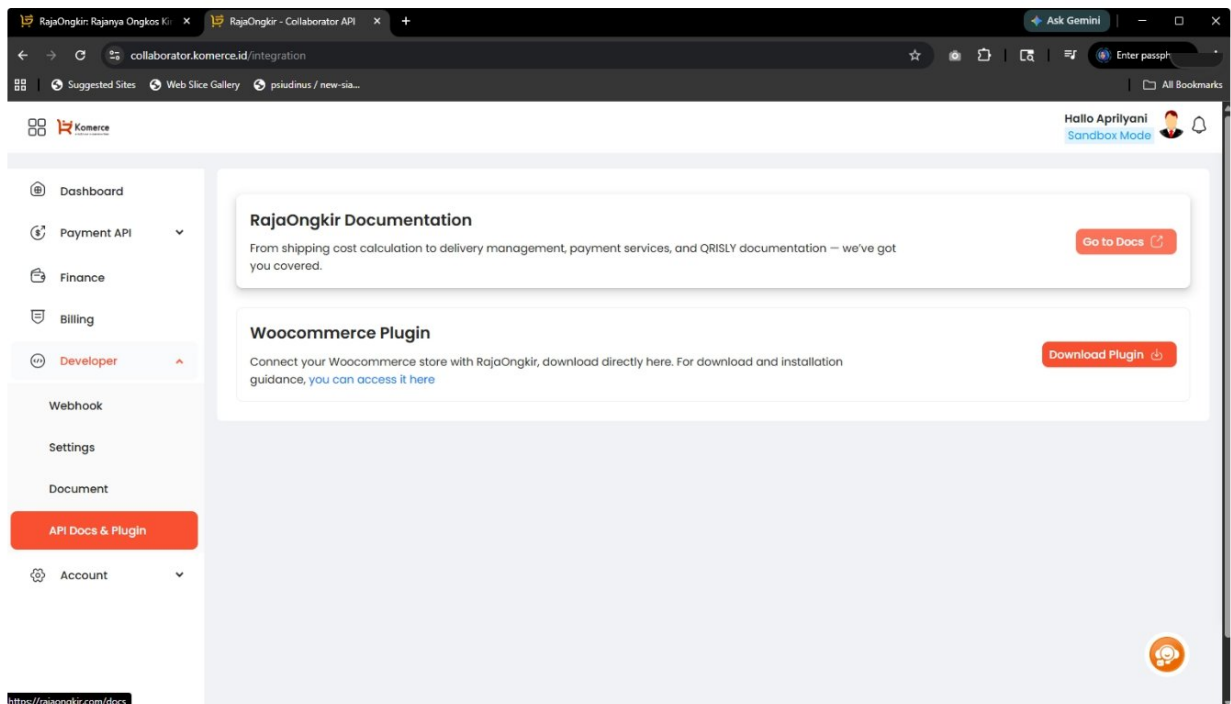
Hallo Apriyani
Sandbox Mode



Untuk melihat API Key silakan klik Show Key pada Shipping Cost, kemudian masukkan password akun. Klik untuk copy API Key yang muncul.



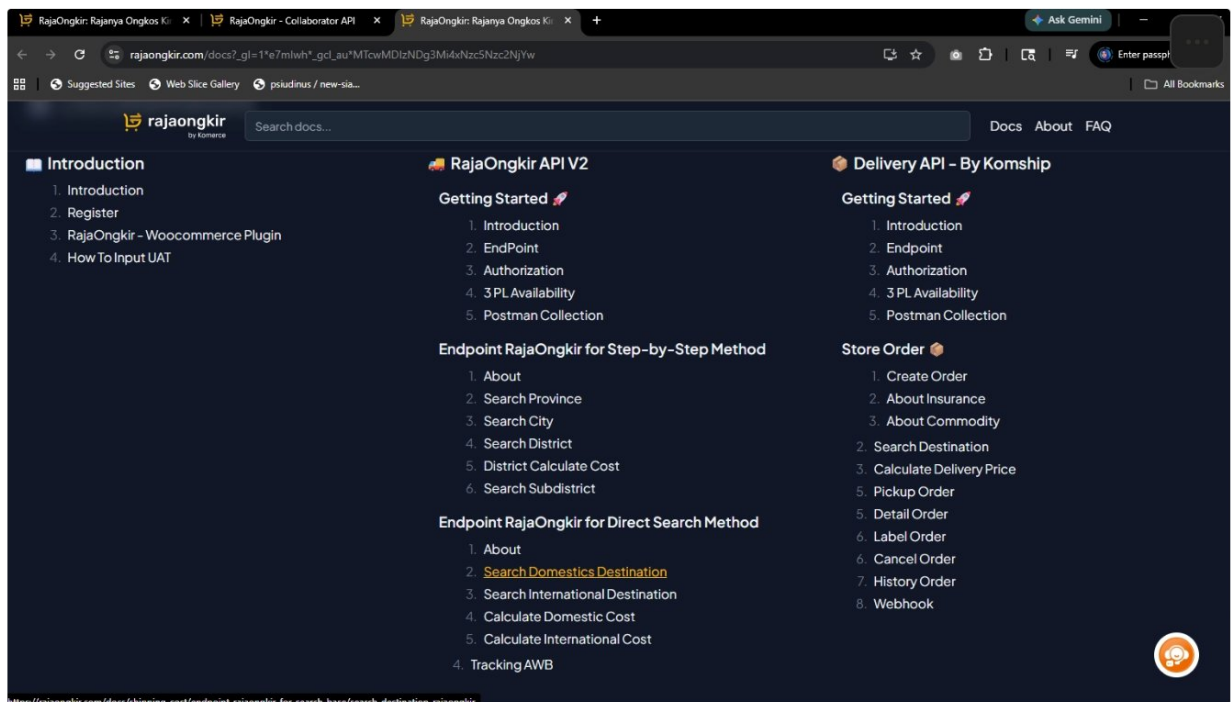
4. Masuk ke Step selanjutnya yaitu memahami cara penggunaan endpointnya. Buka menu Developer - API Docs & Plugin, pilih yang atas 'RajaOngkir Documentation' lalu klik Go to Docs.



Pada bagian RajaOngkir API V2, ada dua endpoint yang akan digunakan pada project ini.

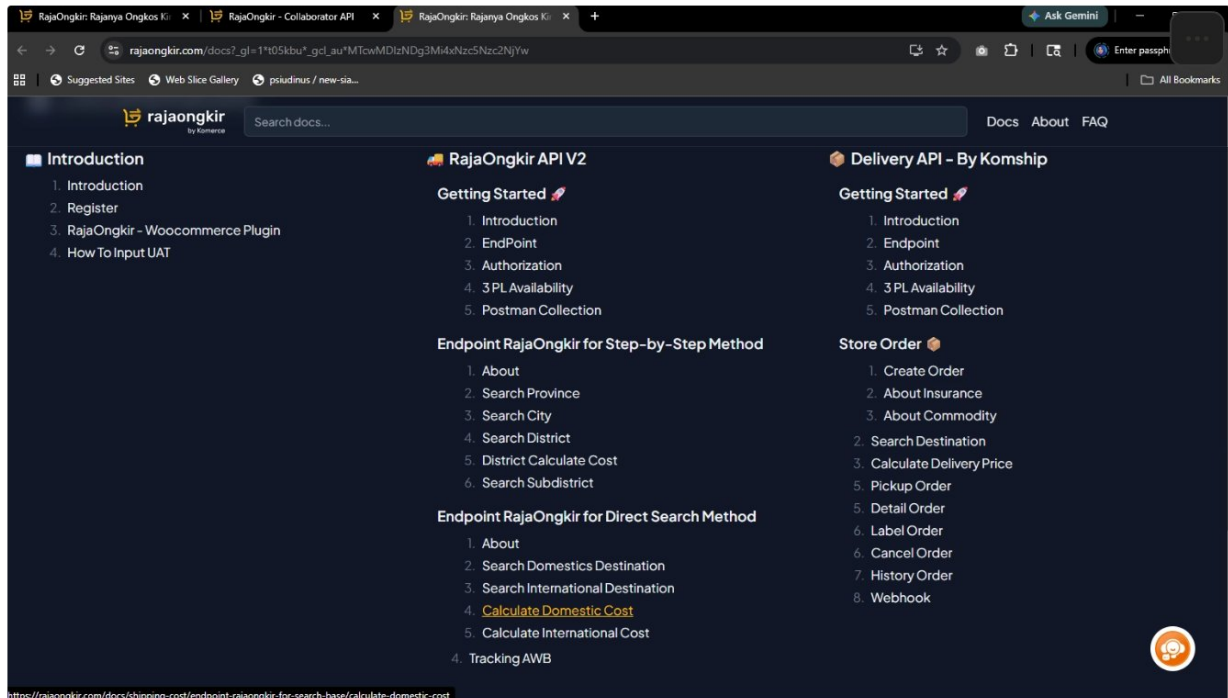
Endpoint pertama adalah pada menu Search Domestic Destination, yang digunakan untuk mencari ID lokasi daerah asal dan daerah tujuan pengiriman

<https://rajaongkir.com/docs/shipping-cost/endpoint-rajaongkir-for-search-base/search-destination-rajaongkir>

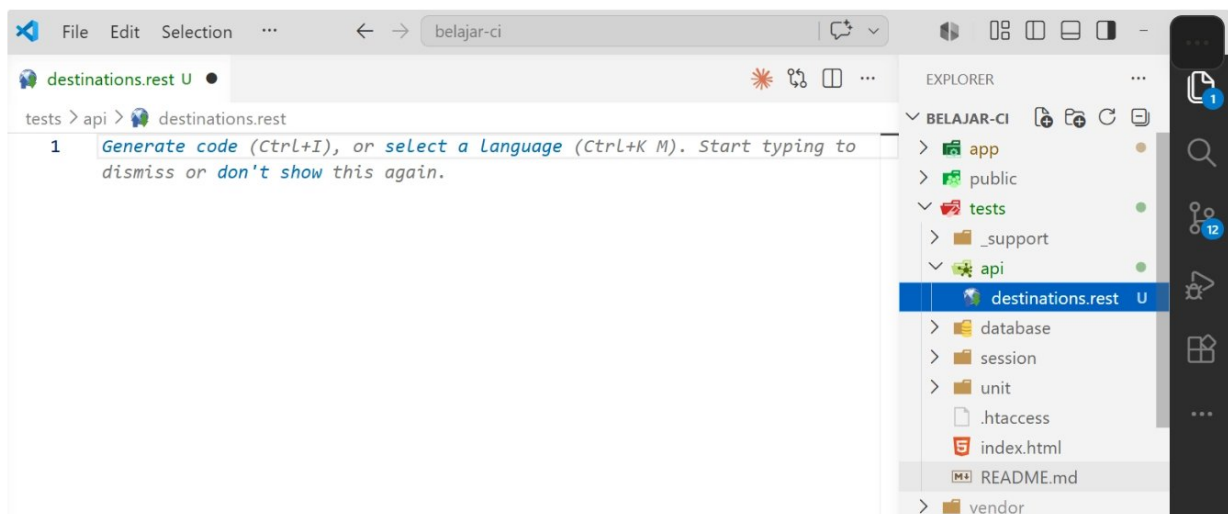


Dan endpoint yang kedua adalah pada menu Calculate Domestic Cost, yang digunakan untuk menghitung biaya pengiriman antara dua daerah di Indonesia. Berdasarkan ID daerah asal (origin), ID daerah tujuan (destination), berat (weight), and kurir (courier).

<https://rajaongkir.com/docs/shipping-cost/endpoint-rajaongkir-for-search-base/calculate-domestic-cost>



5. Untuk API testing, buat folder baru bernama api di dalam folder tests milik project. Lalu di dalam folder api tersebut, tambahkan sebuah file baru bernama destinations.rest untuk uji coba salah satu endpoint milik RajaOngkir yang digunakan untuk mendapatkan data domestic destination.

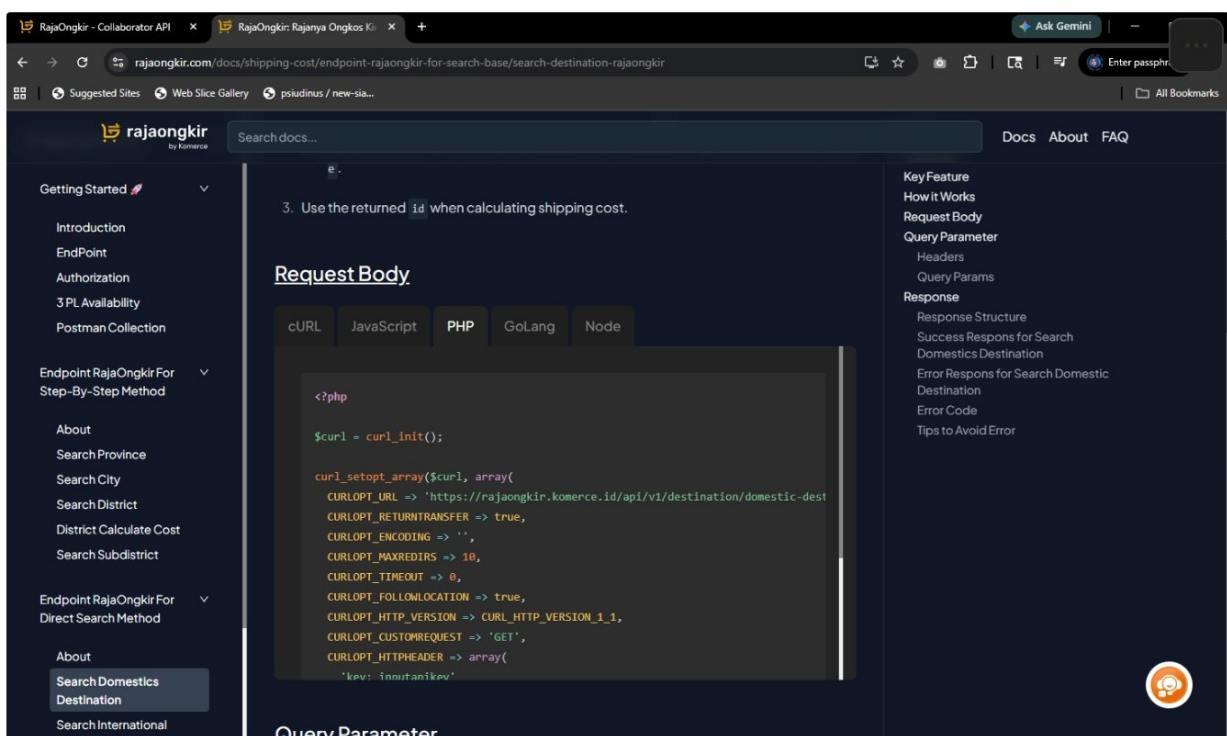




Buka kembali dokumentasi API RajaOngkir kemudian pada menu Search Domestic Destination.

<https://rajaongkir.com/docs/shipping-cost/endpoint-rajaongkir-for-search-base/search-destination-rajaongkir>

Scroll ke bagian bawah halaman ke bagian contoh Request Body.

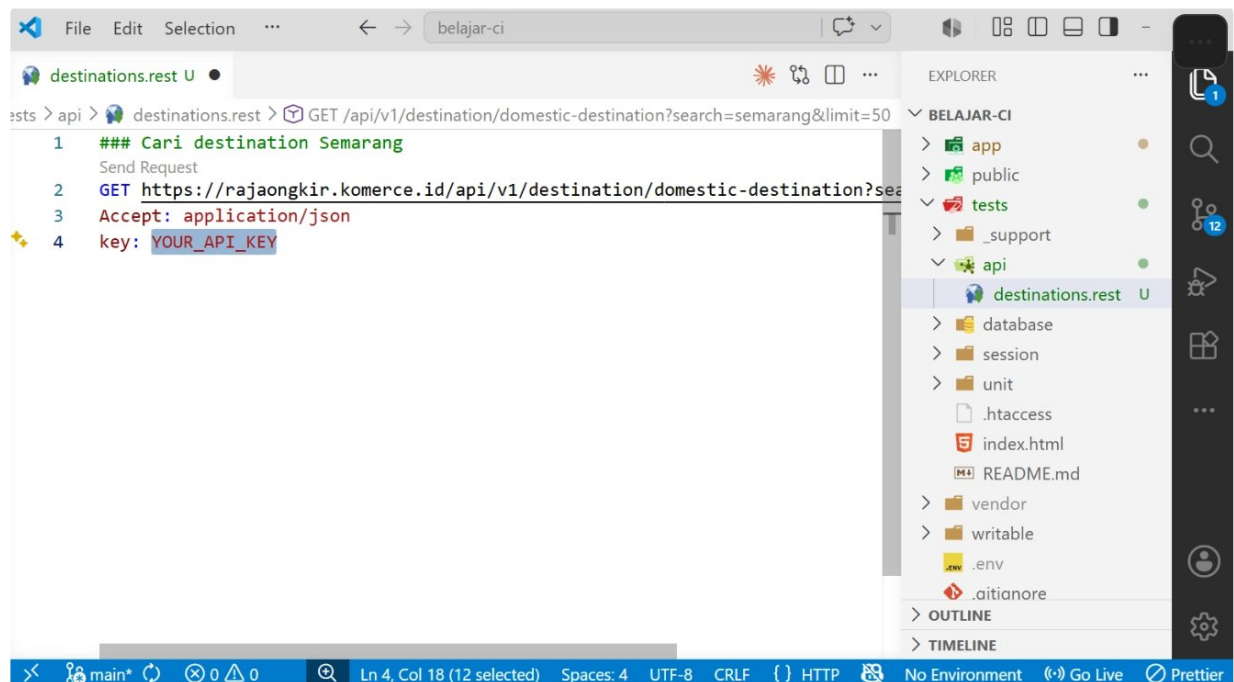


Contoh penggunaan untuk basic request cURL menurut dokumentasi tersebut adalah

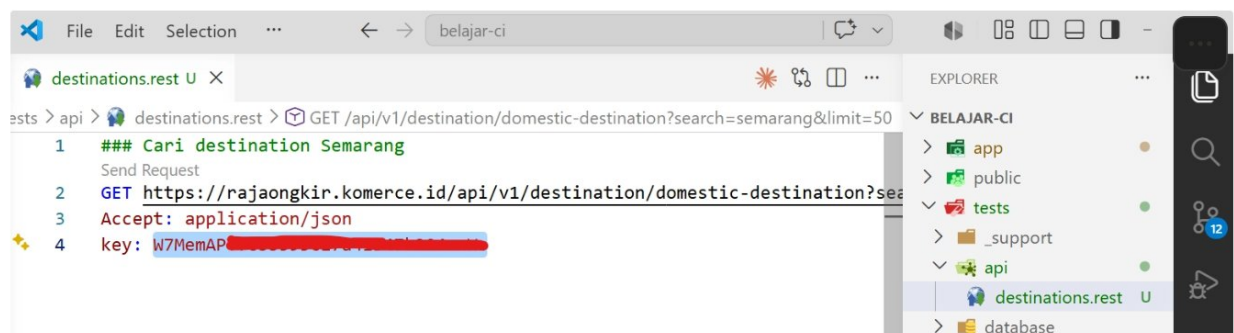
```
curl --location 'https://rajaongkir.kommerce.id/api/v1/destination/domestic-d'
--header 'key: inputapikey'
```

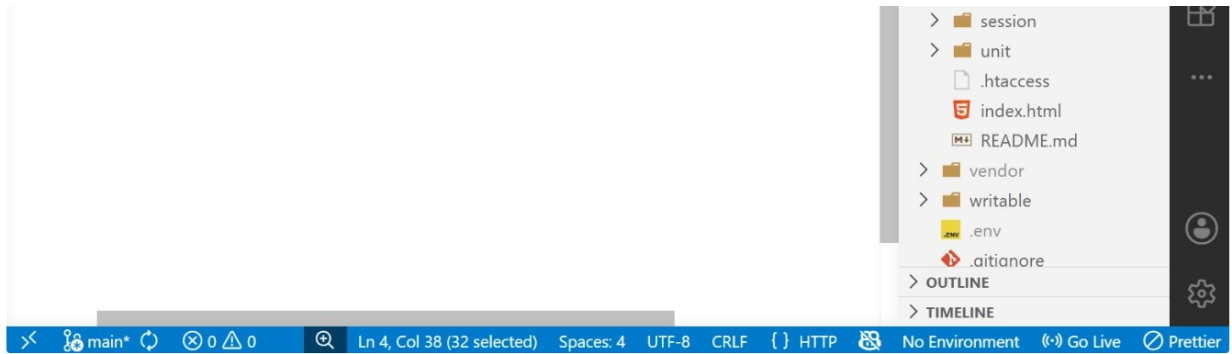
Misalnya untuk mencari data dari sebuah daerah, dengan keyword “semarang” dan batasi (limit) hasilnya 50 saja. Maka berdasarkan dari contoh di dokumentasi tersebut, code untuk uji coba pada file tests\api\destinations.rest adalah.

```
### Cari destination Semarang
GET https://rajaongkir.kommerce.id/api/v1/destination/domestic-destination?search=semarang&limit=50
Accept: application/json
key: YOUR_API_KEY
```

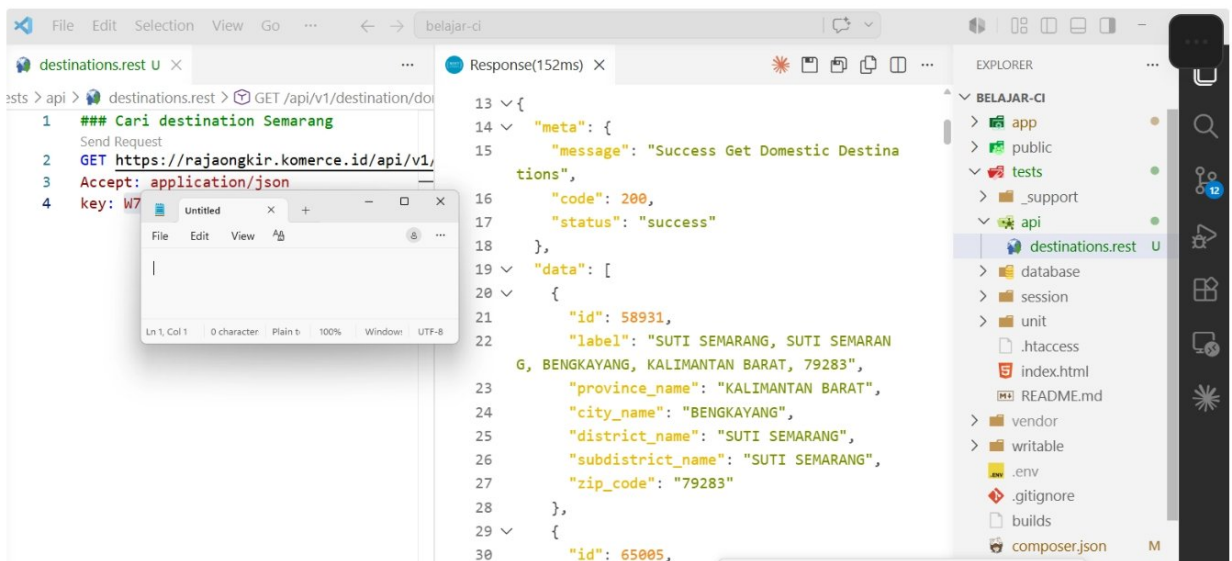
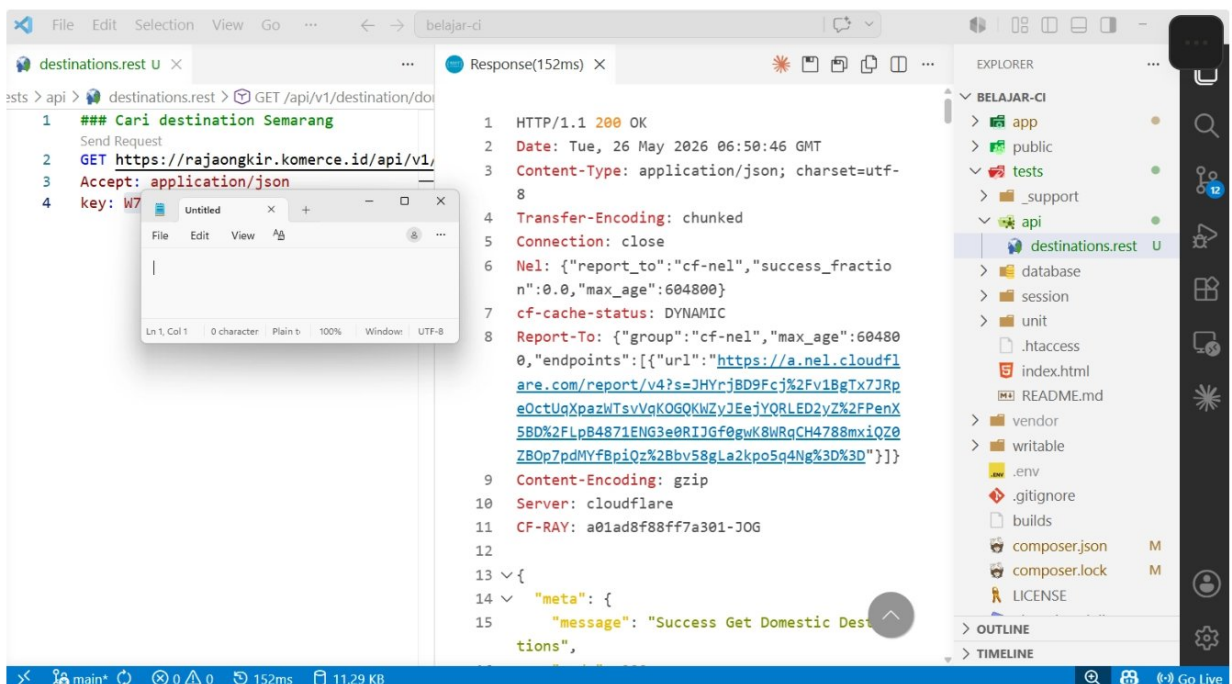


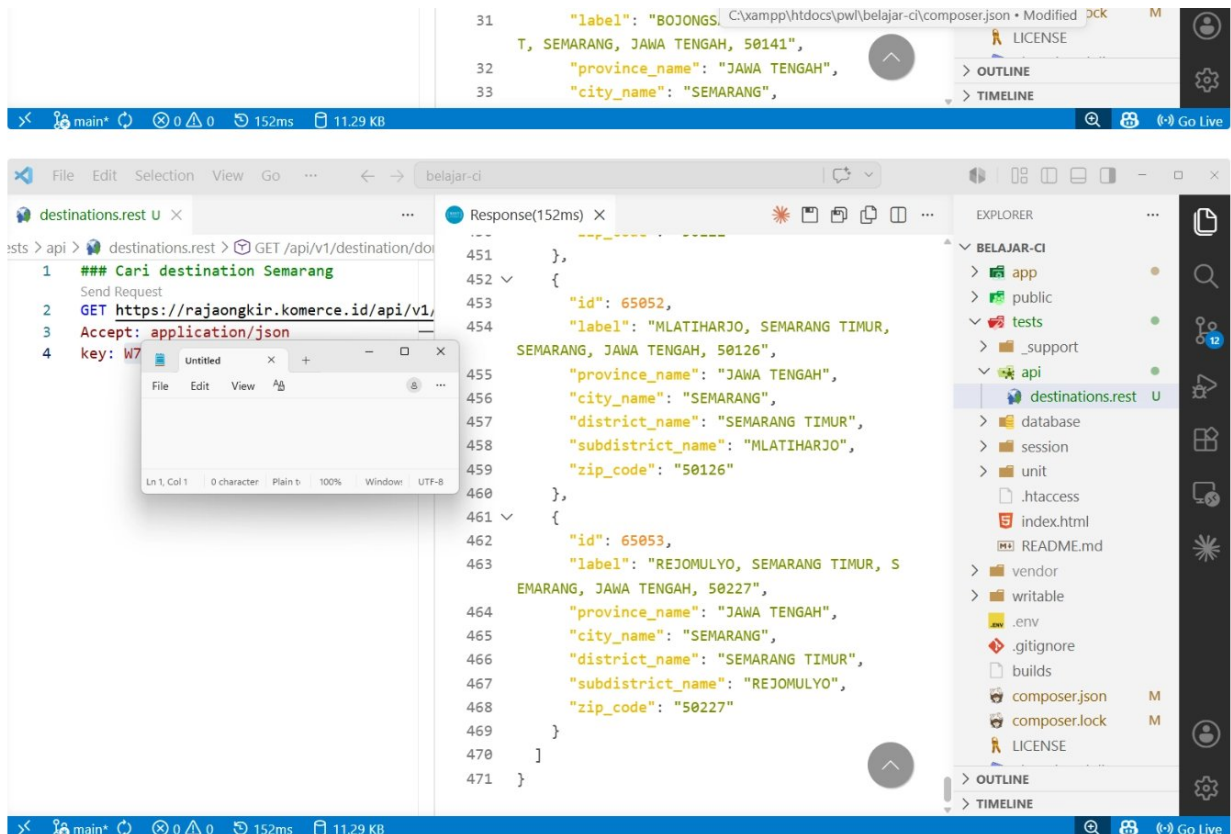
Pada baris ke 4, ganti tulisan YOUR_API_KEY dengan API KEY RajaOngkir dari akun masing-masing (didapatkan dari langkah no 3 tadi).





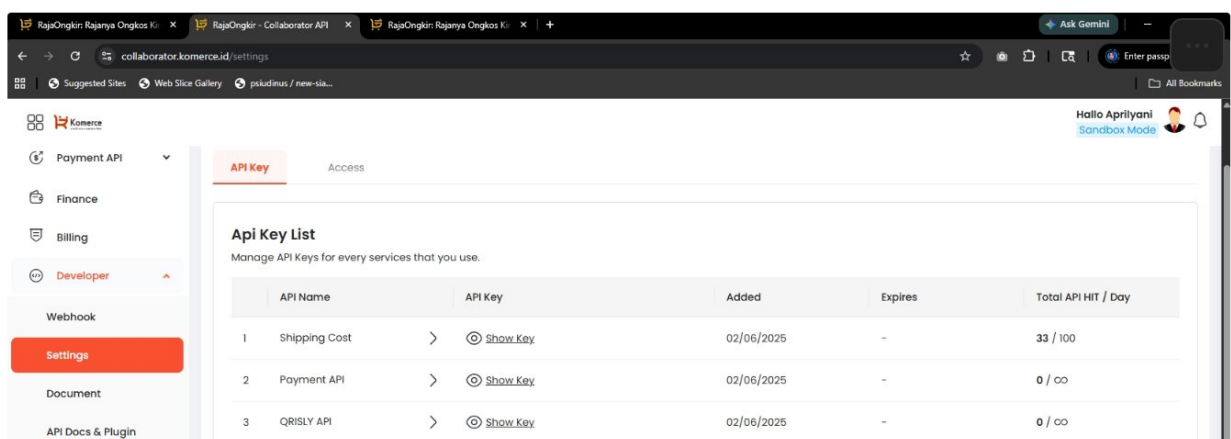
Untuk menjalankannya klik tulisan Send Request berwarna abu-abu pada file tests\api\destinations.rest (pastikan ada koneksi internet). Lalu response nya akan muncul di sebelah kanan.

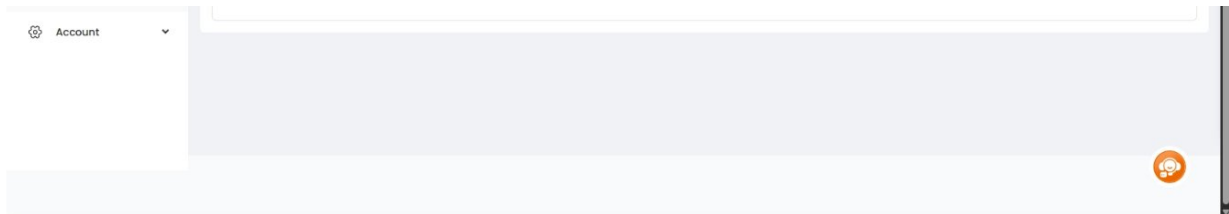




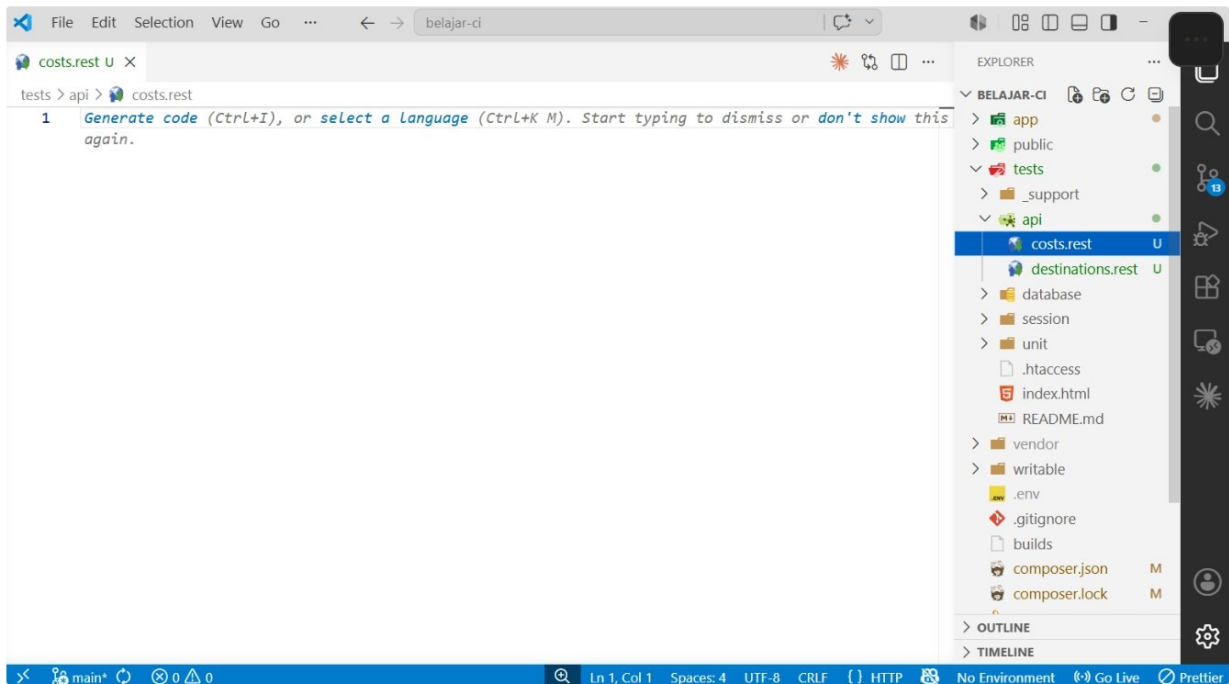
Bagian utama yang akan digunakan pada response JSON tersebut adalah dibagian “data”. Ini lah salah satu cara untuk mendapatkan data daerah/lokasi yang bisa digunakan sebagai data daerah asal pengiriman atau daerah tujuan pengiriman (terutama ID nya).

Setelah berhasil melakukan request data destination ke webservice RajaOngkir, perhatikan jumlah Total API Hit/Day pada menu Developer - Settings. Maksimal per hari hanya bisa 100 kali hit endpoint, kalau sudah 100 maka baru bisa coba hit lagi keesokan harinya.



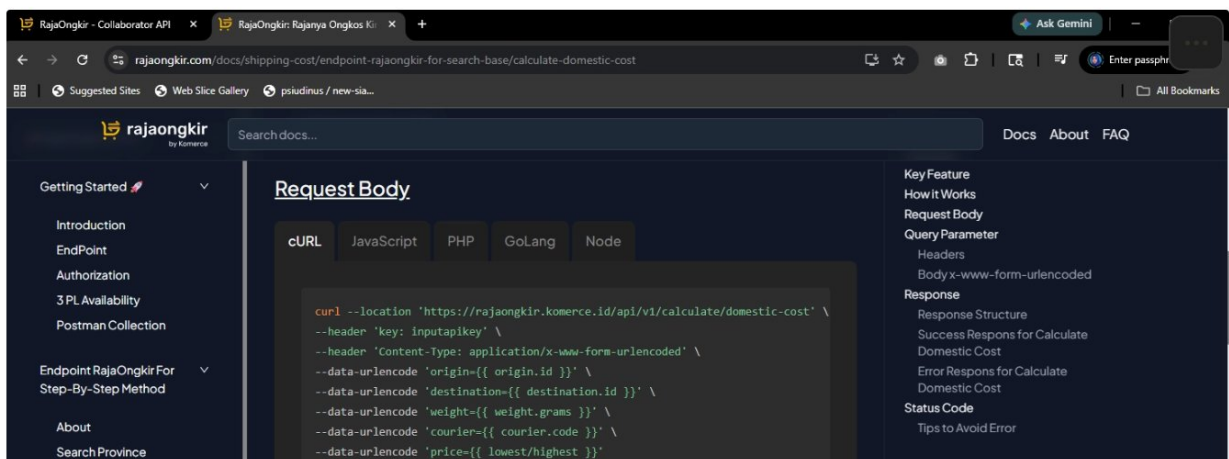


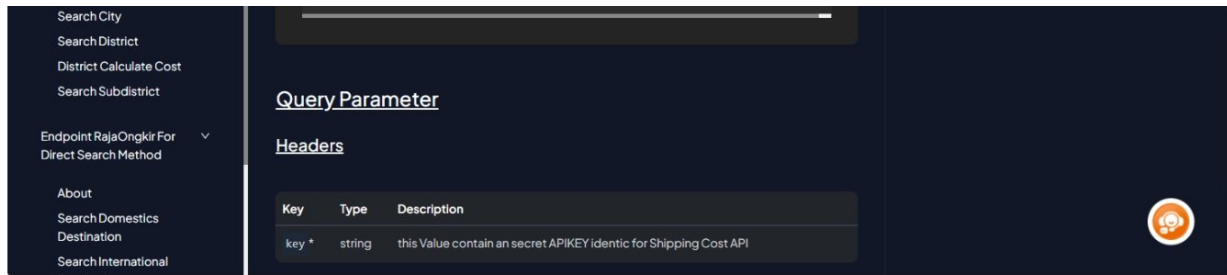
6. Lakukan hal yang sama untuk uji coba endpoint yang kedua yaitu untuk mendapatkan biaya ongkos kirim (cost). Tambahkan sebuah file baru bernama costs.rest di dalam folder 'tests/api'.



Buka kembali dokumentasi API RajaOngkir kemudian pada menu Calculate Domestic Cost. <https://rajaongkir.com/docs/shipping-cost/endpoint-rajaongkir-for-search-base/calculate-domestic-cost>

Scroll ke bagian bawah halaman ke bagian contoh Request Body.





Contoh penggunaan untuk basic request cURL menurut dokumentasi tersebut adalah

```
curl --location 'https://rajaongkir.kommerce.id/api/v1/calculate/domestic-cost' \
--header 'key: inputapikey' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'origin={{ origin.id }}' \
--data-urlencode 'destination={{ destination.id }}' \
--data-urlencode 'weight={{ weight.grams }}' \
--data-urlencode 'courier={{ courier.code }}' \
--data-urlencode 'price={{ lowest/highest }}'
```

Misalnya untuk mencari data layanan dan biaya pengiriman dari satu lokasi ke lokasi yang lain, misalnya isikan pada kolom origin (ID daerah asal) **64999**, destination (ID daerah tujuan) **64942**, weight (berat gram) **1000**, dan courier (kurir) **jne**.

64999 : PEDURUNGAN TENGAH

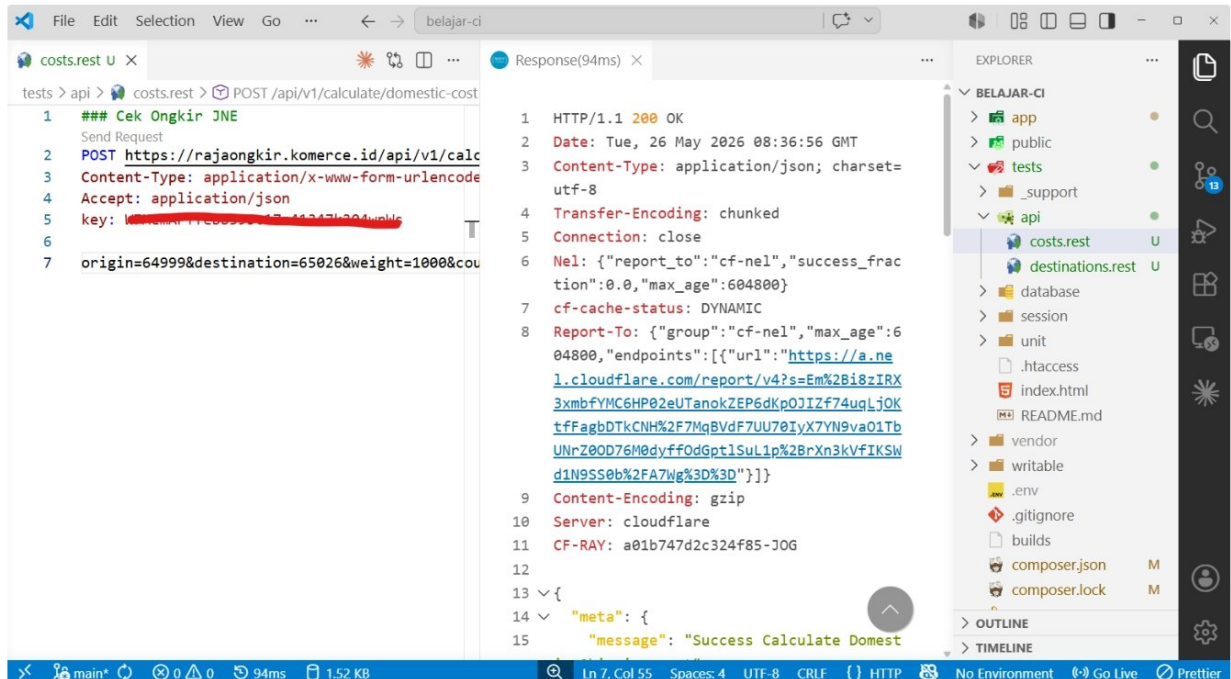
64942 : BANJARDOWO

Maka berdasarkan dari contoh di dokumentasi tersebut, code untuk uji coba pada file tests\api\costs.rest adalah.

```
### Cek Ongkir JNE
POST https://rajaongkir.kommerce.id/api/v1/calculate/domestic-cost
Content-Type: application/x-www-form-urlencoded
Accept: application/json
key: YOUR_API_KEY

origin=64999&destination=64942&weight=1000&courier=jne
```

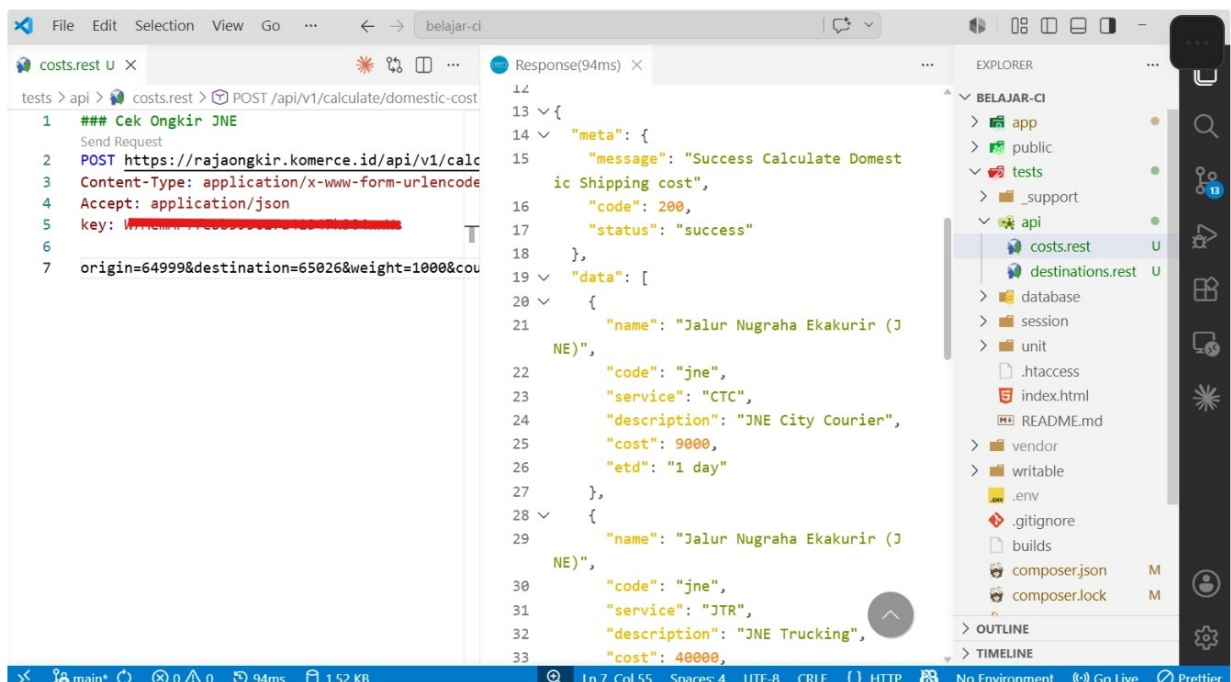
Pada baris ke 5, ganti tulisan YOUR_API_KEY dengan API KEY RajaOngkir dari akun masing-masing (didapatkan dari langkah no 3 tadi).



The screenshot shows a REST client interface in VS Code. The request is a POST to `https://rajaongkir.kommerce.id/api/v1/calculate-domestic-cost` with headers `Content-Type: application/x-www-form-urlencoded` and `Accept: application/json`. The body contains `key: YOUR_API_KEY` (redacted) and `origin=64999&destination=65026&weight=1000&courier=jne`. The response is a 200 OK with a JSON body containing a success message and a URL to a report.

```
1 ### Cek Ongkir JNE
2 POST https://rajaongkir.kommerce.id/api/v1/calculate-domestic-cost
3 Content-Type: application/x-www-form-urlencoded
4 Accept: application/json
5 key: YOUR_API_KEY
6
7 origin=64999&destination=65026&weight=1000&courier=jne
```

```
1 HTTP/1.1 200 OK
2 Date: Tue, 26 May 2026 08:36:56 GMT
3 Content-Type: application/json; charset=utf-8
4 Transfer-Encoding: chunked
5 Connection: close
6 Nel: {"report_to": "cf-nel", "success_fraction": 0.0, "max_age": 604800}
7 cf-cache-status: DYNAMIC
8 Report-To: {"group": "cf-nel", "max_age": 604800, "endpoints": [{"url": "https://a.nel.cloudflare.com/report/v4?s=Em%2Bi8zIRX3xmbfYMC6HP02eUTanokZEP6dKp0JIZf74uqLjOKtffagbDTkCNH%2F7MqBVdF7UU70IyX7YN9va01TbUNrZ00D76M0dyfF0dGpt1SuLip%2BrXn3kVfIKSWd1N9SS0b%2FA7Wg%3D%3D"}]}
9 Content-Encoding: gzip
10 Server: cloudflare
11 CF-RAY: a01b747d2c324f85-JOG
12
13 {
14   "meta": {
15     "message": "Success Calculate Domestic Shipping cost",
16     "code": 200,
17     "status": "success"
18   },
19   "data": [
20     {
21       "name": "Jalur Nugraha Ekakurir (JNE)",
22       "code": "jne",
23       "service": "CTC",
24       "description": "JNE City Courier",
25       "cost": 9000,
26       "etd": "1 day"
27     },
28     {
29       "name": "Jalur Nugraha Ekakurir (JNE)",
30       "code": "jne",
31       "service": "JTR",
32       "description": "JNE Trucking",
33       "cost": 40000
34     }
35   ]
36 }
```



The screenshot shows a REST client interface in VS Code. The request is a POST to `https://rajaongkir.kommerce.id/api/v1/calculate-domestic-cost` with headers `Content-Type: application/x-www-form-urlencoded` and `Accept: application/json`. The body contains `key: YOUR_API_KEY` (redacted) and `origin=64999&destination=65026&weight=1000&courier=jne`. The response is a 200 OK with a JSON body containing a success message and a list of shipping services.

```
1 ### Cek Ongkir JNE
2 POST https://rajaongkir.kommerce.id/api/v1/calculate-domestic-cost
3 Content-Type: application/x-www-form-urlencoded
4 Accept: application/json
5 key: YOUR_API_KEY
6
7 origin=64999&destination=65026&weight=1000&courier=jne
```

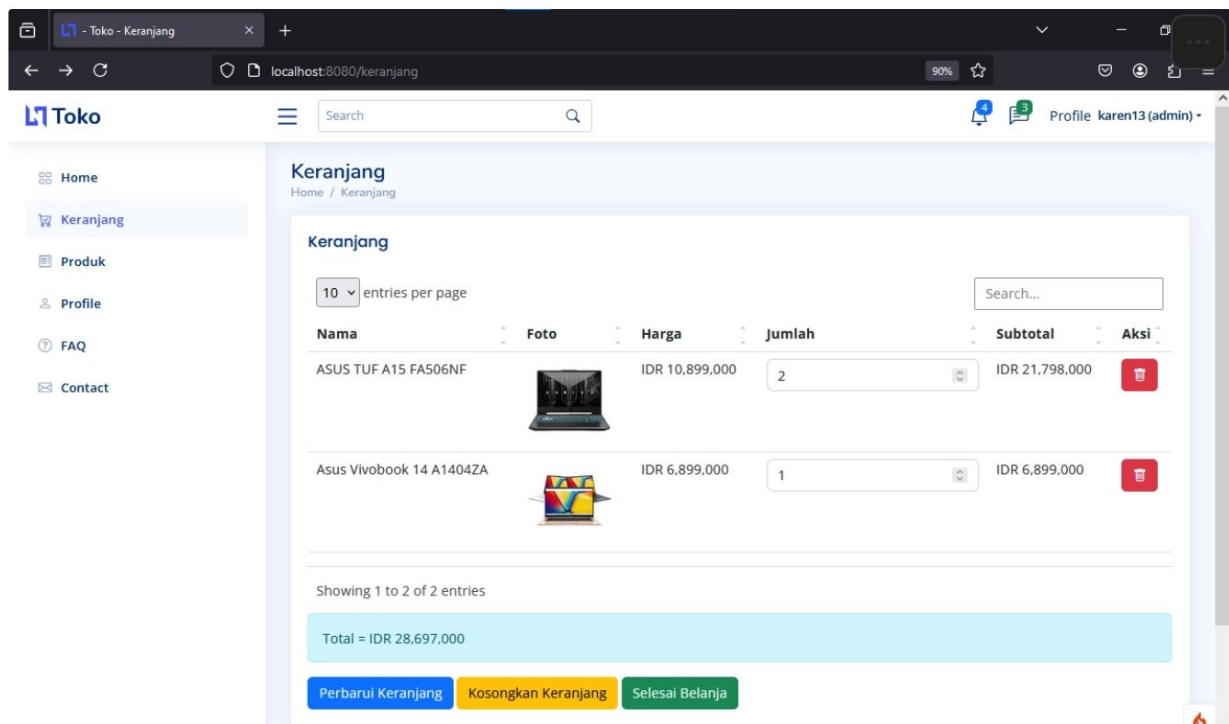
```
14 {
15   "meta": {
16     "message": "Success Calculate Domestic Shipping cost",
17     "code": 200,
18     "status": "success"
19   },
20   "data": [
21     {
22       "name": "Jalur Nugraha Ekakurir (JNE)",
23       "code": "jne",
24       "service": "CTC",
25       "description": "JNE City Courier",
26       "cost": 9000,
27       "etd": "1 day"
28     },
29     {
30       "name": "Jalur Nugraha Ekakurir (JNE)",
31       "code": "jne",
32       "service": "JTR",
33       "description": "JNE Trucking",
34       "cost": 40000
35     }
36   ]
37 }
```

Sama dengan endpoint yang sebelumnya, bagian utama yang akan digunakan pada response JSON tersebut adalah dibagian "data".

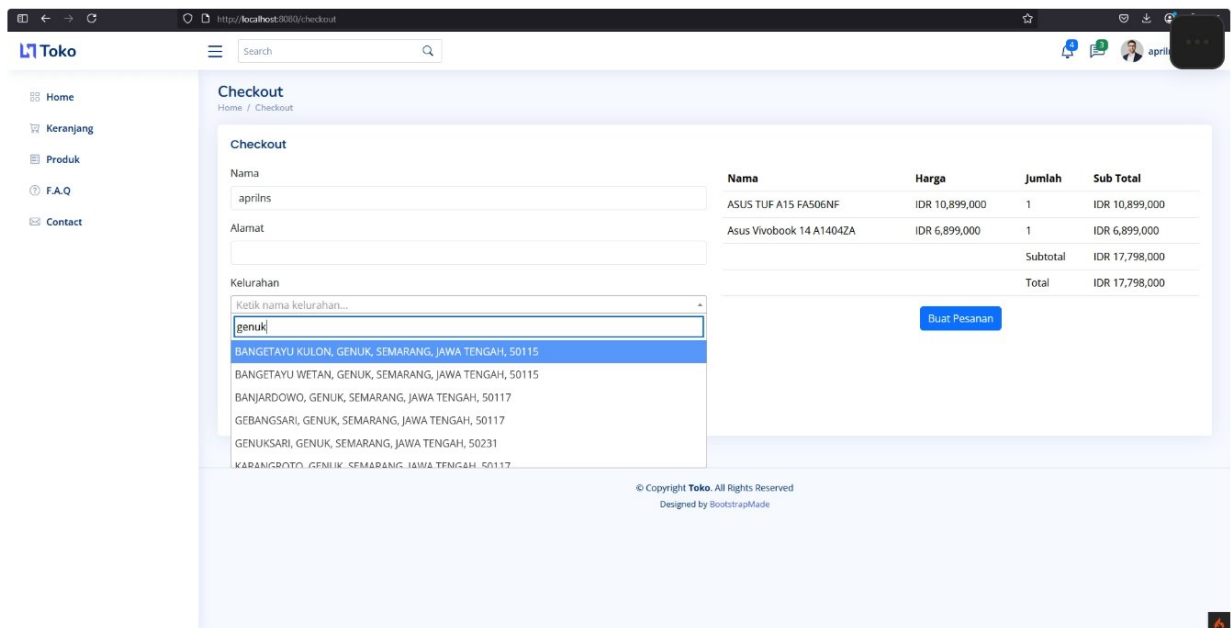
Kedepannya, file costs.rest dan destinations.rest di folder tests/api ini sebaiknya tidak disimpan dalam repository publik atau disertakan pada distribusi aplikasi (hosting/deploy), karena di dalam file tersebut terdapat API Key yang merupakan credential rahasia untuk mengakses layanan RajaOngkir. Sebagai best practice, API Key sebaiknya disimpan pada file .env dan tidak ditulis secara langsung (hardcoded) di dalam source code maupun file pengujian yang dapat diakses oleh pihak lain.

7. Setelah berhasil uji coba kedua endpoint untuk melakukan request sampai mendapatkan response dari API RajaOngkir, lanjutkan dengan integrasi request tersebut ke dalam project Toko.

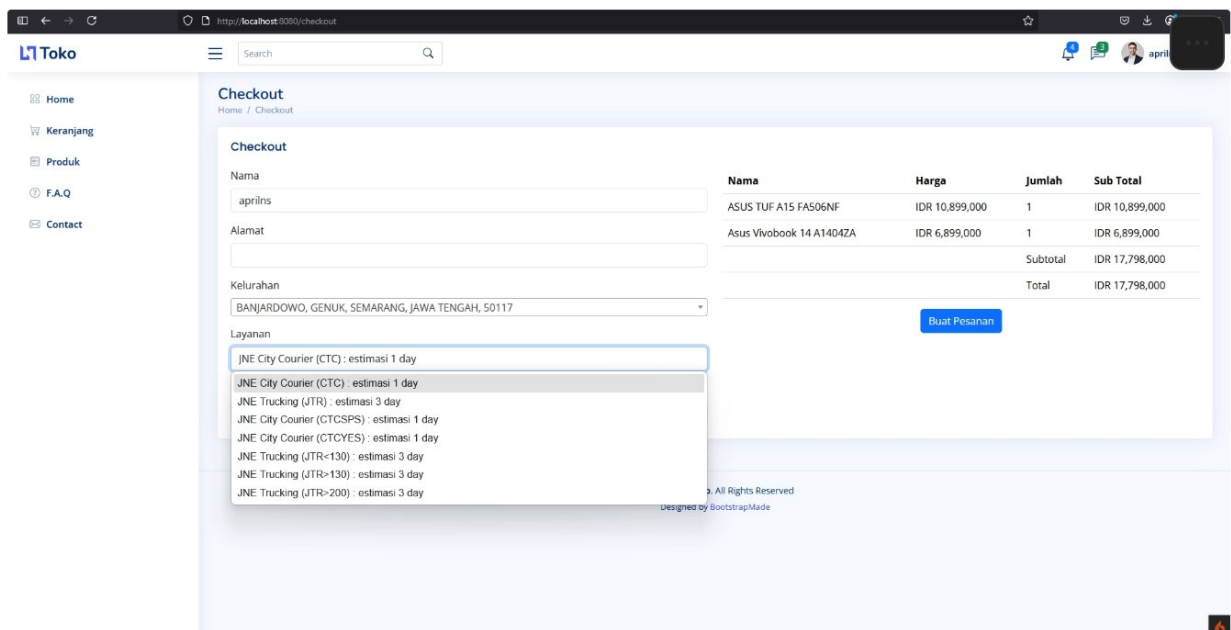
Alur yang akan dibuat nanti adalah seperti ini, setelah user selesai dari halaman keranjang belanja, user klik tombol Selesai Belanja kemudian diarahkan ke halaman Checkout.



Di halaman ini, user melengkapi data yang digunakan untuk detail pengiriman produk. Data lokasi dan layanan ongkir yang didapat dari RajaOngkir akan digunakan pada halaman ini. Pertama user mencari data lokasi pengiriman dengan mengetikkan keyword.



Kemudian setelah user memilih lokasi, muncul data layanan dan ongkir yang tersedia untuk lokasi tersebut.



Setelah uer memilih layanan, muncul nominal ongkir sesuai dengan layanan yang dipilih. Nominal ini dijumlahkan ke nilai total yang harus dibayar user (pada bagian kanan halaman).



Home

Keranjang

Produk

FAQ

Contact

Checkout

Home / Checkout

Nama

aprlins

Alamat

Kelurahan

BANJAROWO, GENUK, SEMARANG, JAWA TENGAH, 50117

Layanan

JNE City Courier (CTCVES) : estimasi 1 day

Ongkir

11000

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	1	IDR 10,899,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
		Subtotal	IDR 17,798,000
		Total	IDR 17,809,000

Buat Pesanan

© Copyright Toko. All Rights Reserved

Designed by BootstrapMade

8. Awali dengan menambahkan route untuk aksi checkout pada file app\Config\Routes.php.

```
$routes->get('checkout', 'TransaksiController::checkout', ['filter' => 'auth
```

File Edit Selection View Go ...

belajar-ci

Update

EXPLORER

BELAJAR-CI

app

Config

Format.php

Generators.php

Honeypot.php

Hostnames.php

Images.php

Kint.php

Logger.php

Migrations.php

Mimes.php

Modules.php

Optimize.php

Pager.php

Paths.php

Publisher.php

Routes.php M

Routing.php

Security.php

Services.php

OUTLINE

TIMELINE

Routes.php M

app > Config > Routes.php

```

11 $routes->post('login', 'AuthController::login');
12 $routes->get('logout', 'AuthController::logout');
13
14 $routes->group('produk', ['filter' => 'auth'], function ($routes) {
15     $routes->get('', 'ProdukController::index');
16     $routes->post('', 'ProdukController::create');
17     $routes->post('edit/(.*)', 'ProdukController::edit/$1');
18     $routes->get('delete/(.*)', 'ProdukController::delete/$1');
19     $routes->get('download', 'ProdukController::download');
20 });
21
22 $routes->group('keranjang', ['filter' => 'auth'], function ($routes) {
23     $routes->get('', 'TransaksiController::index');
24     $routes->post('', 'TransaksiController::cart_add');
25     $routes->post('edit', 'TransaksiController::cart_edit');
26     $routes->get('delete/(.*)', 'TransaksiController::cart_delete/$1');
27     $routes->get('clear', 'TransaksiController::cart_clear');
28     $routes->get('checkout', 'TransaksiController::checkout');
29 });
30
31 $routes->get('checkout', 'TransaksiController::checkout', ['filter' => 'auth']);
32

```

main* Not Committed Yet Ln 31, Col 1 (80 selected) Spaces: 4 UTF-8 CRLF PHP Go Live Prettier

9. Aksi checkout dapat dilakukan setelah user selesai berbelanja. Jadi tambahkan saja elemen

```
<?php if (!empty($items)) : ?>
    <a class="btn btn-success" href="<?php echo base_url() ?>checkout">Seles;
<?php endif; ?>
```

```

15 <table class="table datatable">
16 <tbody>
42 <?php
43     endforeach;
44 endif;
45 ?>
46 </tbody>
47 </table>
48 <!-- End Table with stripped rows -->
49
50 <div class="alert alert-info">
51 <?php echo "Total = " . number_to_currency($total, 'IDR') ?>
52 </div>
53
54 <button type="submit" class="btn btn-primary">Perbarui Keranjang</button>
55 <a class="btn btn-warning" href="<?php echo base_url() ?>keranjang/clear">Kosongkan Keranjang</a>
56 <?php if (empty($items)) : ?>
57 * <a class="btn btn-success" href="<?php echo base_url() ?>checkout">Selesai Belanja</a>
58 <?php endif; ?>
59 <?php echo form_close() ?>
60 <?= $this->endSection() ?>

```

EXPLORER

- BEELAJAR-CI
 - app
 - Config
 - Controllers
 - Database
 - Filters
 - Helpers
 - Language
 - Libraries
 - Models
 - ThirdParty
 - Views
 - .htaccess
 - Common.php
 - index.html
 - public
 - tests
 - vendor
 - writable
 - .env
 - .gitignore

OUTLINE

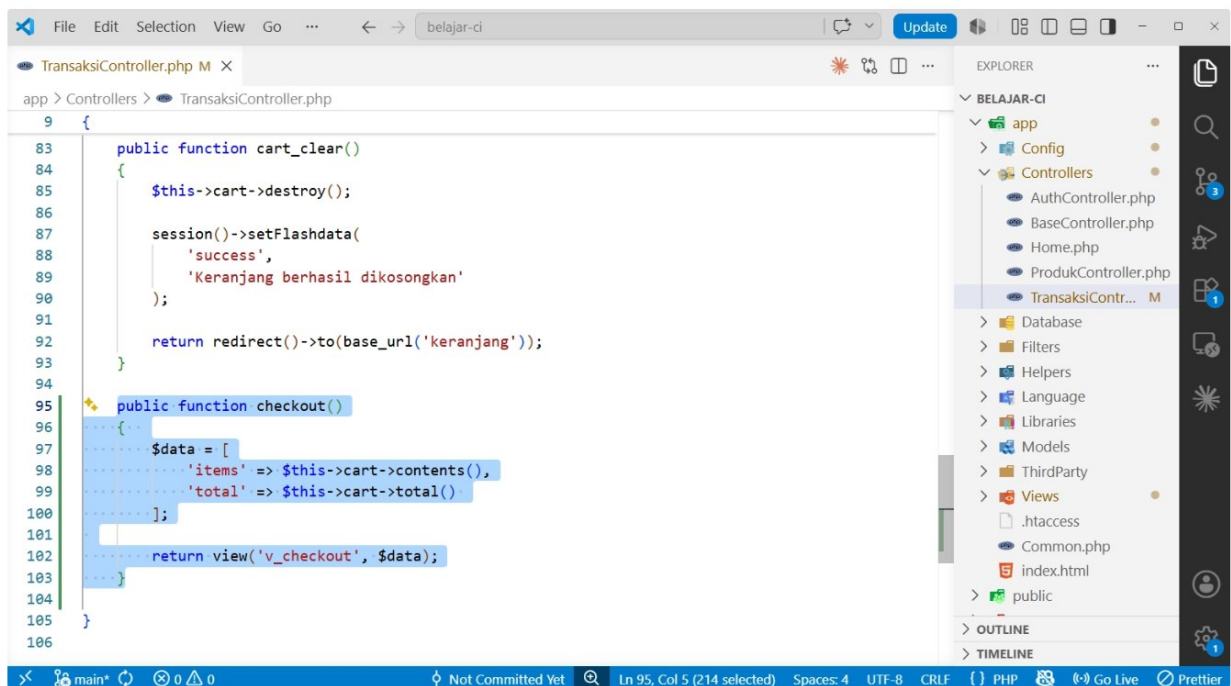
TIMELINE



10. Sesuai dengan route untuk aksi checkout yang sudah dibuat sebelumnya, tambahkan fungsi **checkout()** pada file `app/Controllers/TransaksiController.php`. Tambahkan saja di bawah function **cart_clear()**.

```
public function checkout()
{
    $data = [
        'items' => $this->cart->contents(),
        'total' => $this->cart->total()
    ];

    return view('v_checkout', $data);
}
```



Pada fungsi ini coba sediakan data keranjang dan nominal total belanja untuk ditampilkan pada file view `v_checkout`.

11. Buat file baru di dalam folder `app/Views` dengan nama `v_checkout.php` dan coba tampilkan isi data keranjang dan nominal total belanja.

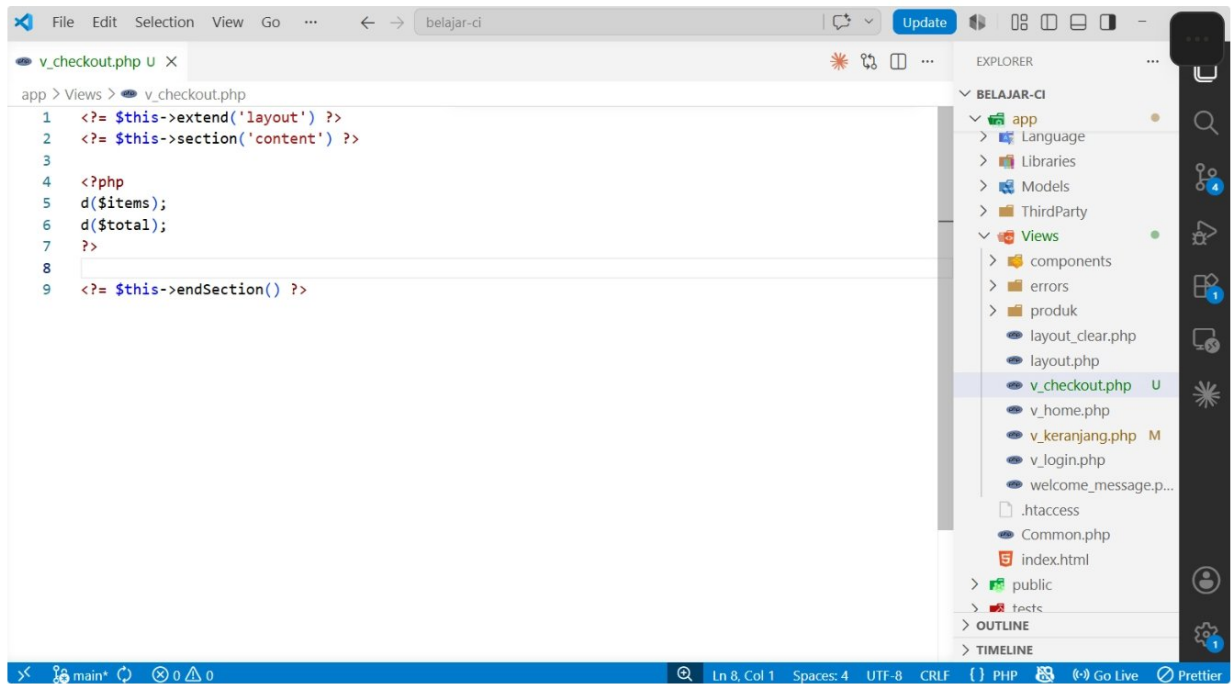
```

<?= $this->extend('layout') ?>
<?= $this->section('content') ?>

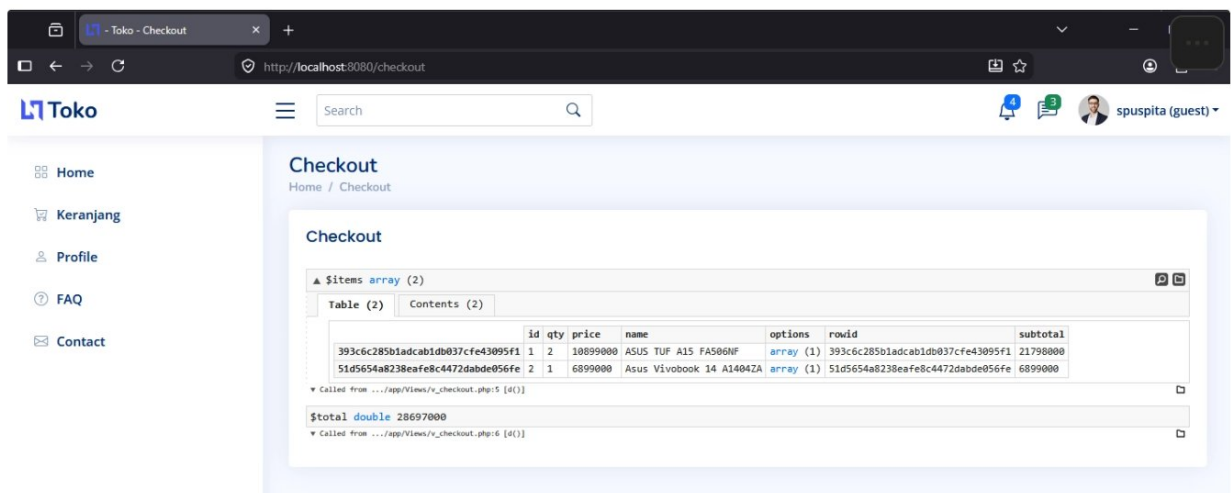
<?php
d($items);
d($total);
?>

<?= $this->endSection() ?>

```

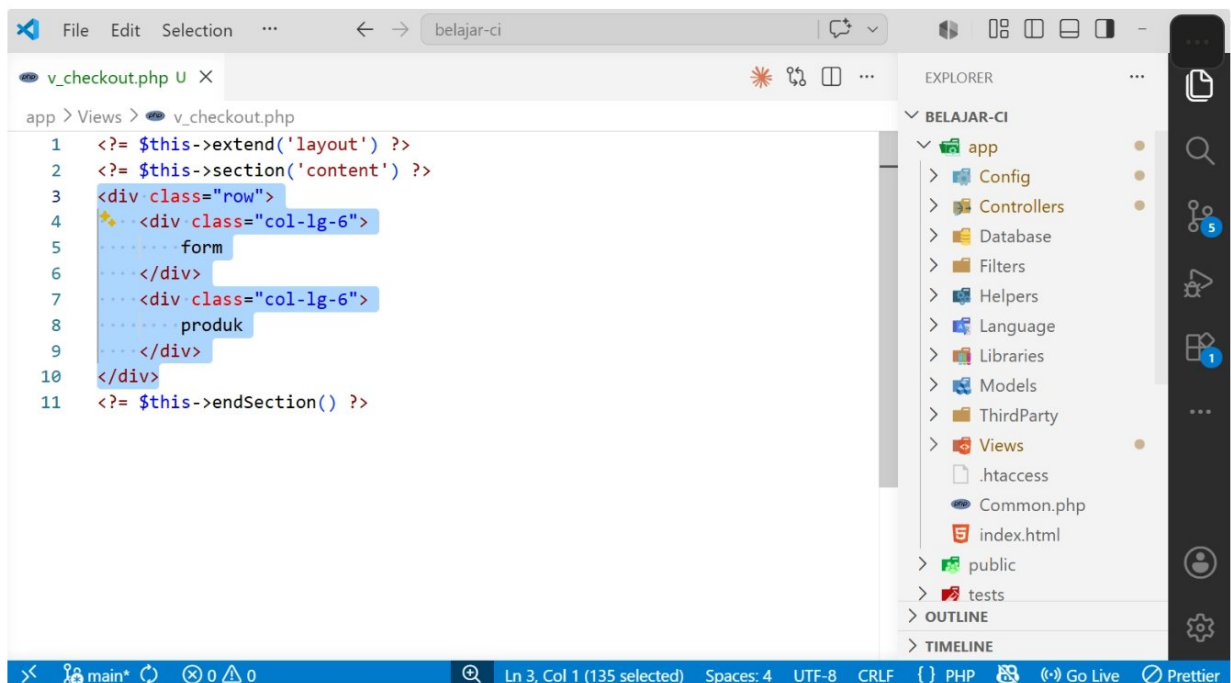


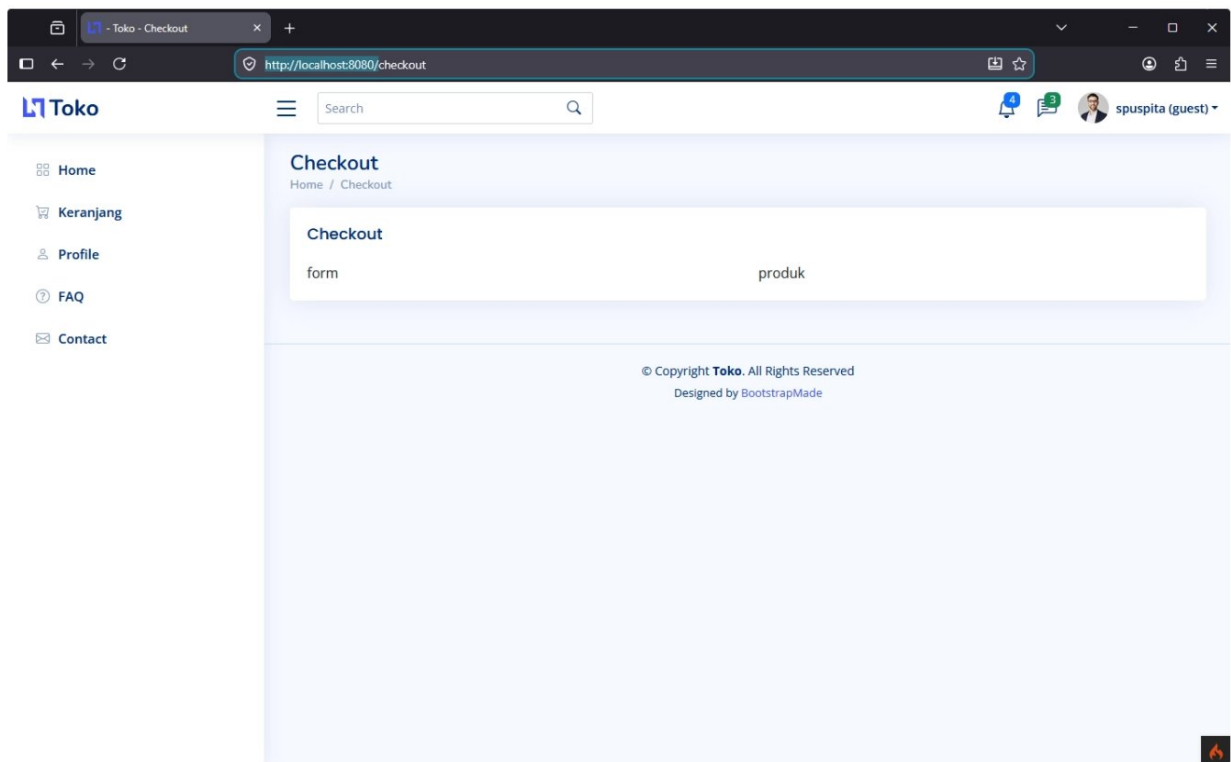
Sehingga jika tombol **Selesai Belanja** pada menu **Keranjang** diklik akan mengarah ke tampilan **Checkout**.



12. Data keranjang dan nominal total belanja sudah siap digunakan pada file `app/Views/v_checkout.php`. Selanjutnya, atur layout untuk menampilkan data tersebut, bagi tampilan menjadi 2 kolom. Karena grid bernilai 12, jadi jika tampilan dibagi 2 kolom maka masing-masing lebarnya adalah 6. Tampilan kiri untuk pengisian data diri pada form checkout. Tampilan kanan untuk daftar data produk yang dibeli dan nominalnya.

```
<div class="row">
  <div class="col-lg-6">
    form
  </div>
  <div class="col-lg-6">
    produk
  </div>
</div>
```





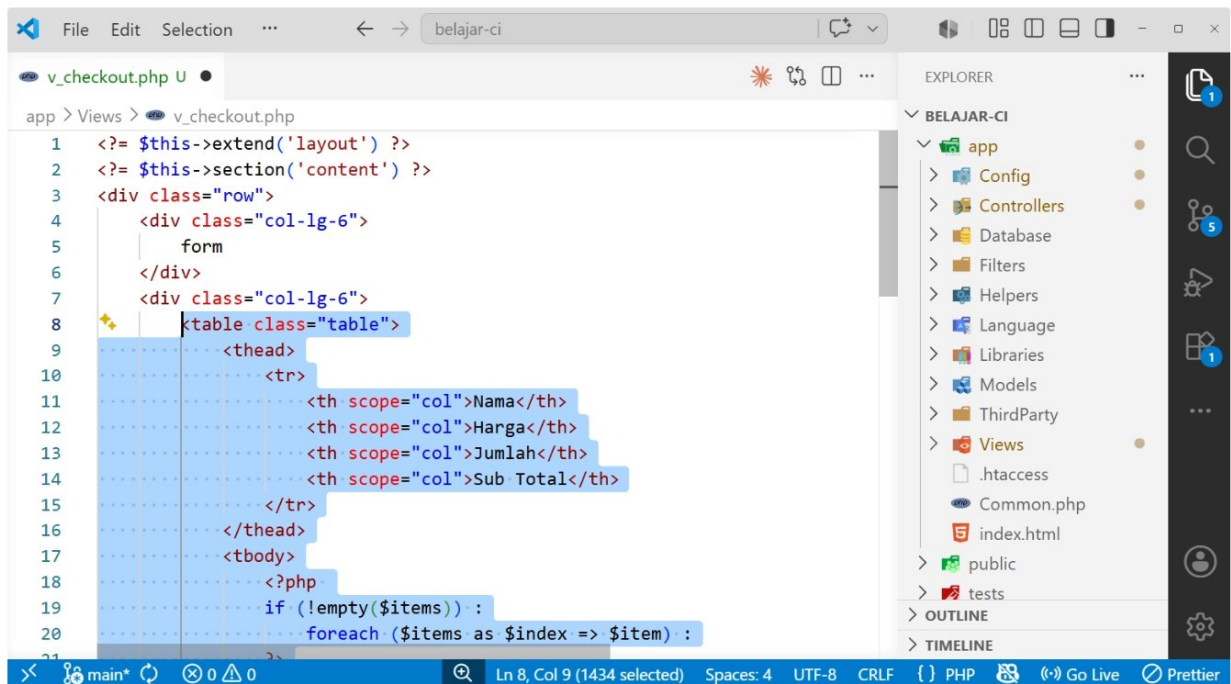
13. Buat dulu kolom sebelah kanan, ganti tulisan “produk” dengan code yang menampilkan data keranjang dalam sebuah table seperti ini.

```
<table class="table">
  <thead>
    <tr>
      <th scope="col">Nama</th>
      <th scope="col">Harga</th>
      <th scope="col">Jumlah</th>
      <th scope="col">Sub Total</th>
    </tr>
  </thead>
  <tbody>
    <?php
    if (!empty($items)) :
      foreach ($items as $index => $item) :
        ?>
          <tr>
            <td><?= $item['name'] ?></td>
            <td><?= number_to_currency($item['price'], 'IDR') ?></td>
            <td><?= $item['qty'] ?></td>
            <td><?= number_to_currency($item['price'] * $item['qty'],
```

```

        </tr>
    <?php
        endforeach;
    endif;
    ?>
    <tr>
        <td colspan="2"></td>
        <td>Subtotal</td>
        <td><?= number_to_currency($total, 'IDR') ?></td>
    </tr>
    <tr>
        <td colspan="2"></td>
        <td>Total</td>
        <td><span id="total"><?= number_to_currency($total, 'IDR') ?></span>
    </tr>
</tbody>
</table>

```



Pada bagian Total, dibungkus dalam sebuah elemen `<span>` dengan `id="total"` supaya bagian ini nilainya bisa diubah-ubah dengan mudah sesuai dengan nominal ongkir yang dipilih pembeli nantinya (id digunakan sebagai selector).



Checkout

Home / Checkout

Checkout

form

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	2	IDR 21,798,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
Subtotal			IDR 28,697,000
Total			IDR 28,697,000



14. Kemudian kolom sebelah kanan, ganti tulisan “form” dengan code untuk menampilkan form yang harus diisi user untuk menyimpan detail data pengiriman.

```
<?= form_open('buy', 'class="row g-3"') ?>

<?= form_hidden('username', session()->get('username')) ?>
<?= form_hidden('total_harga', '', ['id' => 'total_harga']) ?>

<div class="col-12">
    <?= form_label('Nama', 'nama', ['class' => 'form-label']) ?>
    <?= form_input([
        'name'      => 'nama',
        'id'        => 'nama',
        'class'     => 'form-control',
        'value'     => session()->get('username'),
        'readonly' => true]) ?>
</div>
<div class="col-12">
    <?= form_label('Alamat', 'alamat', ['class' => 'form-label']) ?>
    <?= form_input([
        'name' => 'alamat',
        'id'   => 'alamat',
        'class' => 'form-control']) ?>
</div>
<div class="col-12">
```

```

        <?= form_label('Kelurahan', 'kelurahan', ['class' => 'form-label']) ?>
        <strong>select kelurahan</strong>
    </div>
    <div class="col-12">
        <?= form_label('Layanan', 'layanan', ['class' => 'form-label']) ?>
        <strong>select layanan</strong>
    </div>
    <div class="col-12">
        <?= form_label('Ongkir', 'ongkir', ['class' => 'form-label']) ?>
        <?= form_input([
            'name'      => 'ongkir',
            'id'        => 'ongkir',
            'class'     => 'form-control',
            'readonly'  => true]) ?>
    </div>
    <div class="col-12">
        <?= form_submit(
            'submit',
            'Buat Pesanan',
            ['class' => 'btn btn-primary']) ?>
    </div>

    <?= form_close() ?>

```

```

1 <?= $this->extend('layout') ?>
2 <?= $this->section('content') ?>
3 <div class="row">
4     <div class="col-lg-6">
5         <?= form_open('buy', 'class="row g-3"> ?>
6
7         <?= form_hidden('username', session()->get('username')) ?>
8
9         <?= form_input([
10             'type' => 'hidden',
11             'name' => 'total_harga',
12             'id' => 'total_harga',
13             'value' => '' ?>
14
15     <div class="col-12">
16         <?= form_label('Nama', 'nama', ['class' => 'form-label']) ?>
17         <?= form_input([
18             'name' => 'nama',
19             'id' => 'nama',
20             'class' => 'form-control',
21             'value' => session()->get('username')

```

Action dari form ini menggunakan route “buy” yang digunakan untuk aksi menyimpan data

transaksi pembelian user (rute akan dibuat pada langkah berikutnya).

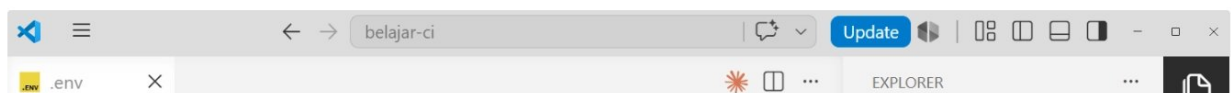
Pada bagian awal form terdapat dua input dengan tipe hidden untuk menyimpan nilai dari username (diambil dari session) dan nilai nominal total dari transaksi pembelian. Input dengan tipe hidden ini nanti untuk kebutuhan insert data ke database table transaction. Dibuat hidden supaya tidak dapat dilihat dan diubah-ubah oleh user melalui tampilan form.

Untuk bagian pilihan kelurahan dan pilihan layanan, sementara biarkan dulu saja karena nanti datanya akan diambil dari webservice Raja Ongkir.

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	2	IDR 21,798,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
Subtotal			IDR 28,697,000
Total			IDR 28,697,000

- Supaya dapat menggunakan webservice RajaOngkir pada project Toko, tambahkan API Key dan Base URL RajaOngkir pada file .env. Sesuaikan dengan key akun RajaOngkir milik masing-masing.

```
#-----  
# API KEY  
#-----  
RAJAONGKIR_API_KEY=xxx  
RAJAONGKIR_BASE_URL=https://rajaongkir.kommerce.id/api/v1/
```



```
.env
132
133 #-----
134 # LOGGER
135 #-----
136
137 # logger.threshold = 4
138
139 #-----
140 # CURLRequest
141 #-----
142
143 # curlrequest.shareOptions = false
144
145 #-----
146 # API KEY
147 #-----
148 RAJAONGKIR_API_KEY=
149 RAJAONGKIR_BASE_URL=https://rajaongkir.kommerce.id/api/v1/
```

BELEJAR-CI

- public
 - NiceAdmin
 - .htaccess
 - favicon.ico
 - index.php
 - robots.txt
 - tests
 - vendor
 - writable
 - .env
 - .gitignore
 - builds
 - composer.json
 - composer.lock
 - LICENSE
 - phpunit.xml.dist

OUTLINE

TIMELINE

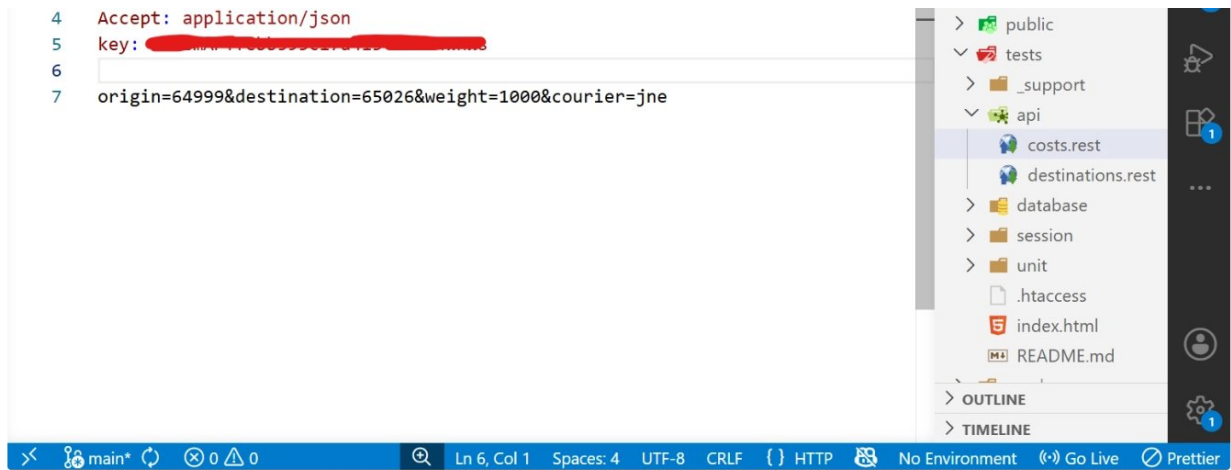
16. Untuk penggunaan webservice pada sebuah project, biasanya dibuat terpisah pada folder dan file service. Pada folder app buat folder baru bernama Services, lalu isi dengan file baru bernama RajaOngkirService.php.
- Tambahkan code untuk melakukan request GET ke webservice RajaOngkir berdasarkan file uji coba yang sudah pernah dibuat sebelumnya (pada file tests\api\destinations.rest dan tests\api\costs.rest).

```
destinations.rest X
tests > api > destinations.rest > GET /api/v1/destination/domestic-destination?search=semarang&limit=50
1 ### Cari destination Semarang
  Send Request
2 GET https://rajaongkir.kommerce.id/api/v1/destination/domestic-destination?search=semarang&limit=50
3 Accept: application/json
4 key: 
```

```
destinations.rest costs.rest X
tests > api > costs.rest > POST /api/v1/calculate/domestic-cost
1 ### Cek Ongkir JNE
  Send Request
2 POST https://rajaongkir.kommerce.id/api/v1/calculate/domestic-cost
3 Content-Type: application/x-www-form-urlencoded
```

EXPLORER

- BELEJAR-CI
 - app
 - .htaccess
 - Common.php
 - index.html



Buat dua fungsi baru pada file `app\Services\RajaOngkirService.php`. Yang pertama adalah fungsi untuk mendapatkan data daerah tujuan yaitu `getDestination()`. Yang kedua adalah fungsi untuk mendapatkan data layanan serta ongkos kirimnya yaitu `getCost()`.

Gunakan API KEY dan BASE URL RajaOngkir yang suda disimpan pada file `env`.

```
<?php
namespace App\Services;
use Config\Services;

class RajaOngkirService
{
    protected $client;
    protected $apiKey;
    protected $baseUrl;

    public function __construct()
    {
        $this->client = Services::curlrequest([
            'timeout' => 10,
            'http_errors' => false,
        ]);

        $this->apiKey = env('RAJAONGKIR_API_KEY');
        $this->baseUrl = env('RAJAONGKIR_BASE_URL');
    }

    public function getDestination(string $keyword): array
    {
        $response = $this->client->get(
```

```

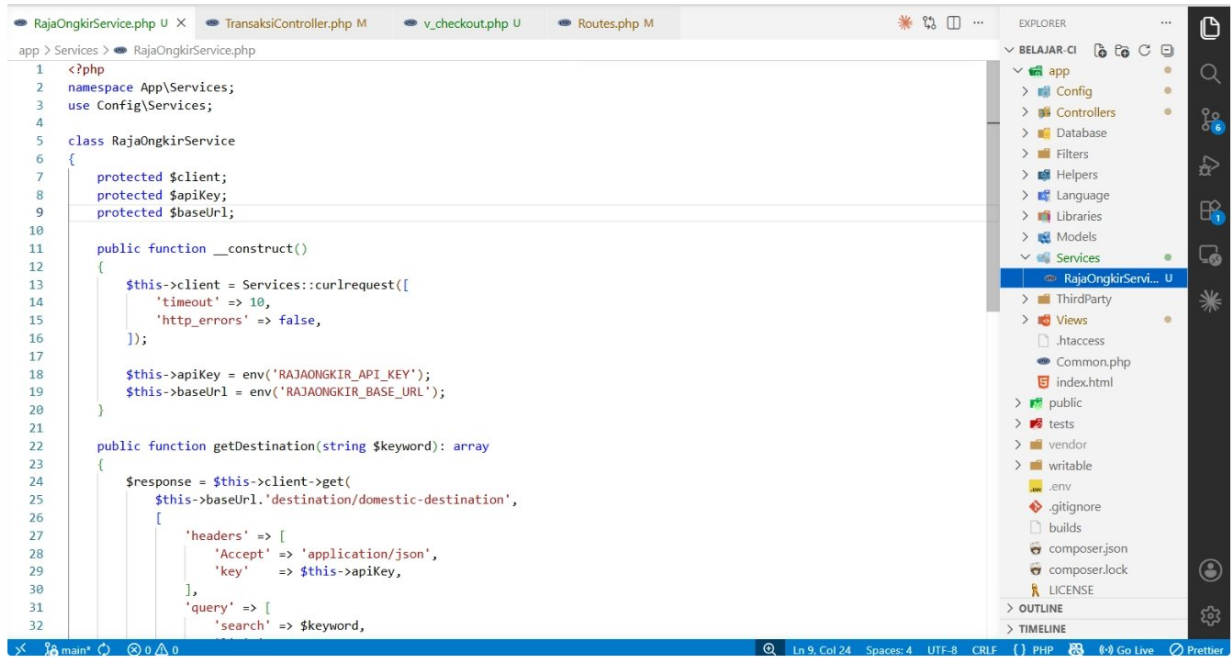
        $this->baseUrl.'destination/domestic-destination',
        [
            'headers' => [
                'Accept' => 'application/json',
                'key'     => $this->apiKey,
            ],
            'query' => [
                'search' => $keyword,
                'limit'  => 50,
            ]
        ]
    );

    return json_decode(
        $response->getBody(),
        true
    );
}

public function getCost(string $origin, string $destination, int $weight)
{
    $response = $this->client->post(
        $this->baseUrl . 'calculate/domestic-cost',
        [
            'headers' => [
                'Accept' => 'application/json',
                'key'     => $this->apiKey,
            ],
            'form_params' => [
                'origin'     => $origin,
                'destination' => $destination,
                'weight'     => $weight,
                'courier'    => $courier,
            ]
        ]
    );

    return json_decode(
        $response->getBody(),
        true
    );
}
}

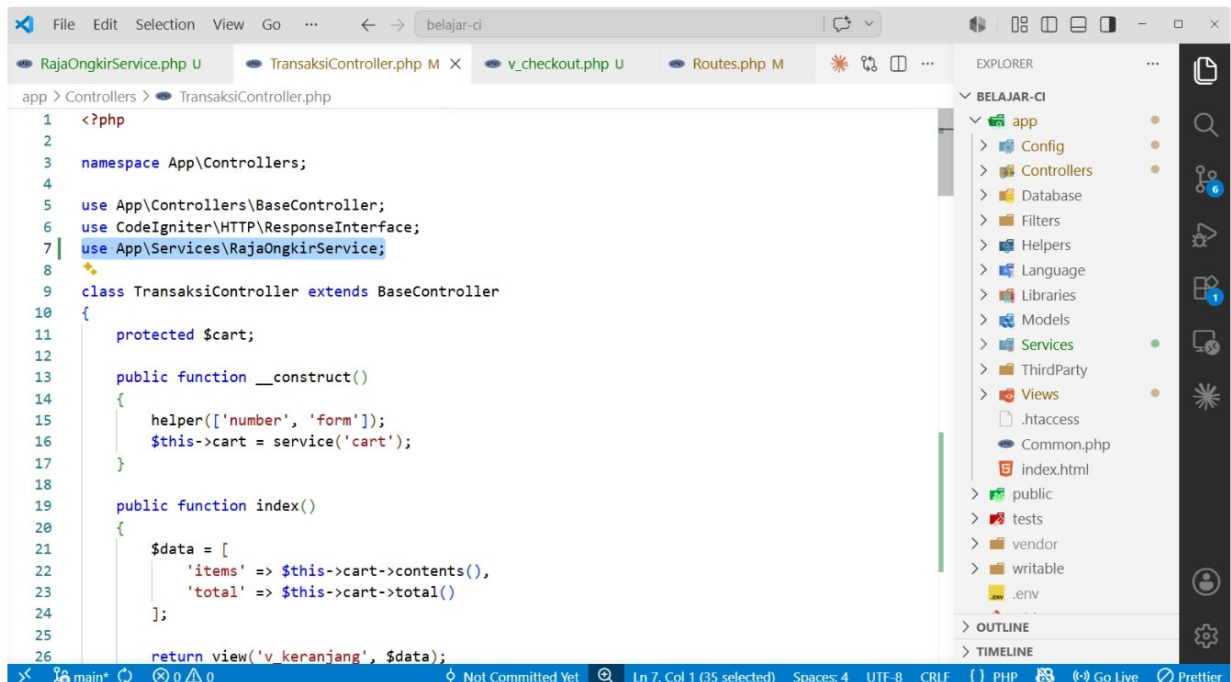
```



```
1 <?php
2 namespace App\Services;
3 use Config\Services;
4
5 class RajaOngkirService
6 {
7     protected $client;
8     protected $apiKey;
9     protected $baseUrl;
10
11     public function __construct()
12     {
13         $this->client = Services::curlrequest([
14             'timeout' => 10,
15             'http_errors' => false,
16         ]);
17
18         $this->apiKey = env('RAJAONGKIR_API_KEY');
19         $this->baseUrl = env('RAJAONGKIR_BASE_URL');
20     }
21
22     public function getDestination(string $keyword): array
23     {
24         $response = $this->client->get(
25             $this->baseUrl.'destination/domestic-destination',
26             [
27                 'headers' => [
28                     'Accept' => 'application/json',
29                     'key' => $this->apiKey,
30                 ],
31                 'query' => [
32                     'search' => $keyword,
```

17. Gunakan service RajaOngkir yang baru saja dibuat tadi pada file app\Controllers\TransaksiController.php.

```
use App\Services\RajaOngkirService;
```

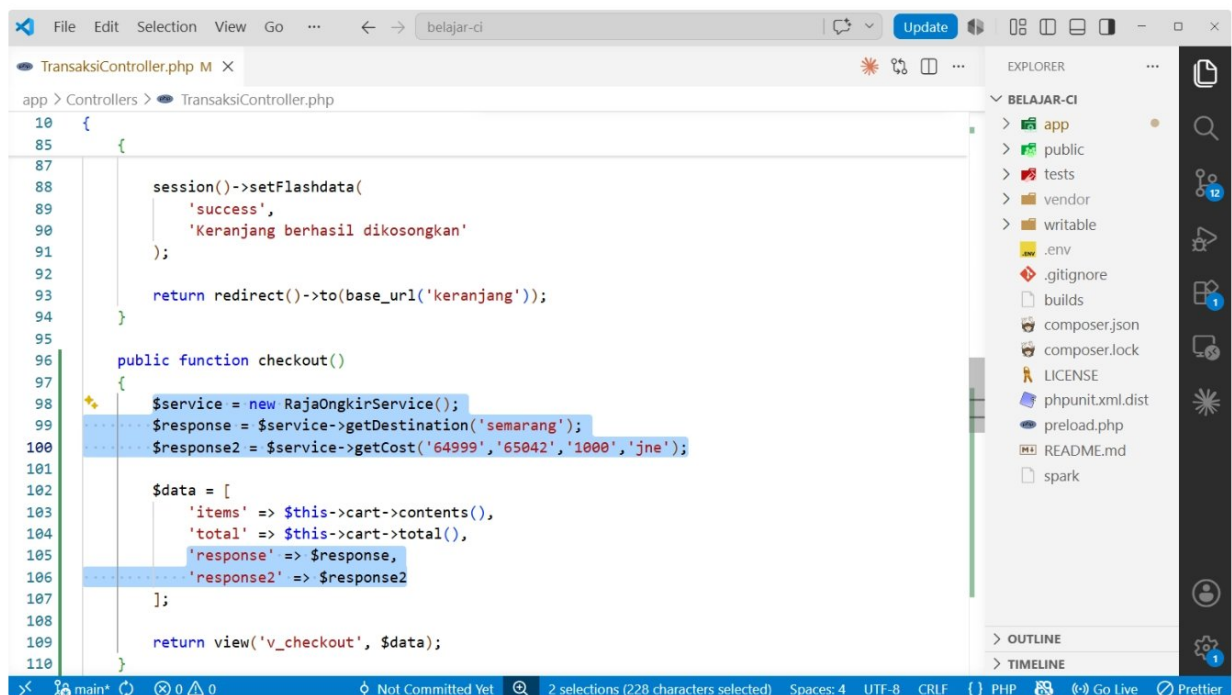


```
1 <?php
2 namespace App\Controllers;
3
4 use App\Controllers\BaseController;
5 use CodeIgniter\HTTP\ResponseInterface;
6 use App\Services\RajaOngkirService;
7
8 class TransaksiController extends BaseController
9 {
10     protected $cart;
11
12     public function __construct()
13     {
14         helper(['number', 'form']);
15         $this->cart = service('cart');
16     }
17
18     public function index()
19     {
20         $data = [
21             'items' => $this->cart->contents(),
22             'total' => $this->cart->total()
23         ];
24
25         return view('v_keranjang', $data);
26     }
```

Coba dulu tampilkan hasilnya pada halaman checkout dengan cara pemanggilan manual untuk `getDestination()` keywordnya "semarang". Dan untuk `getCost()` dari daerah dengan ID 64999 ke daerah dengan ID 65042, berat 1000 gram, dan layanan JNE.

```
$service = new RajaOngkirService();  
$response = $service->getDestination('semarang');  
$response2 = $service->getCost('64999', '65042', '1000', 'jne');
```

```
'response' => $response,  
'response2' => $response2
```

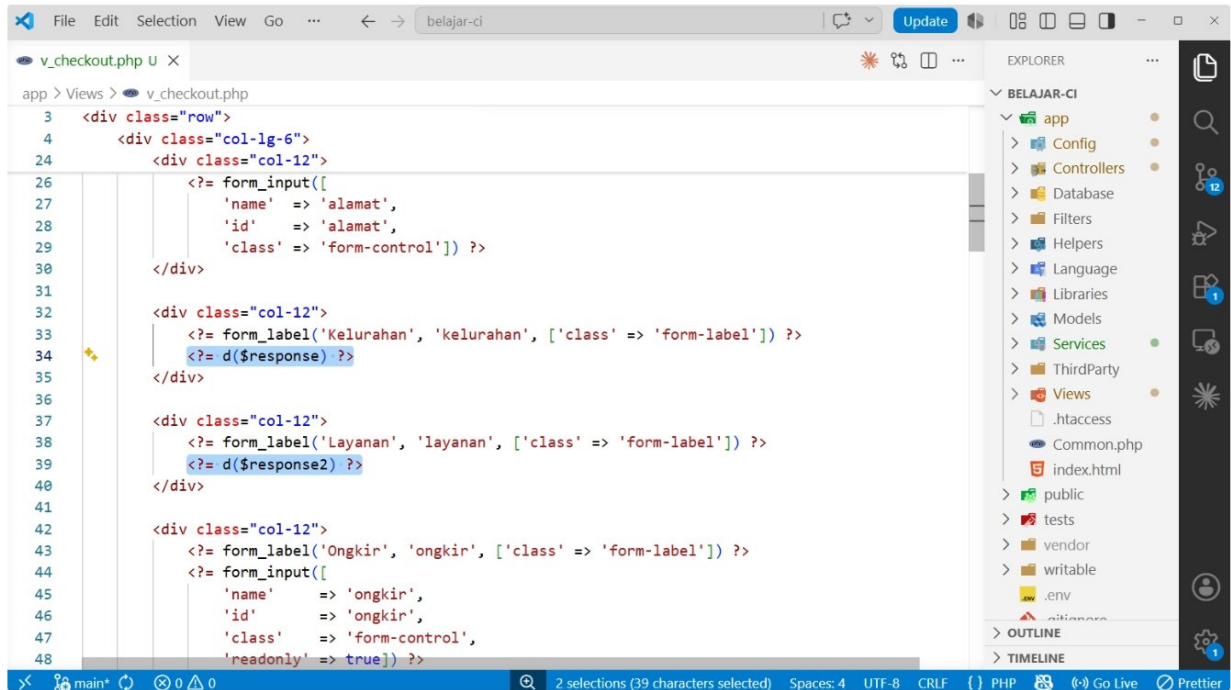


```
10 {  
85 {  
87  
88 session()->setFlashdata(  
89     'success',  
90     'Keranjang berhasil dikosongkan'  
91 );  
92  
93 return redirect()->to(base_url('keranjang'));  
94 }  
95  
96 public function checkout()  
97 {  
98     $service = new RajaOngkirService();  
99     $response = $service->getDestination('semarang');  
100    $response2 = $service->getCost('64999', '65042', '1000', 'jne');  
101  
102    $data = [  
103        'items' => $this->cart->contents(),  
104        'total' => $this->cart->total(),  
105        'response' => $response,  
106        'response2' => $response2  
107    ];  
108  
109    return view('v_checkout', $data);  
110 }
```

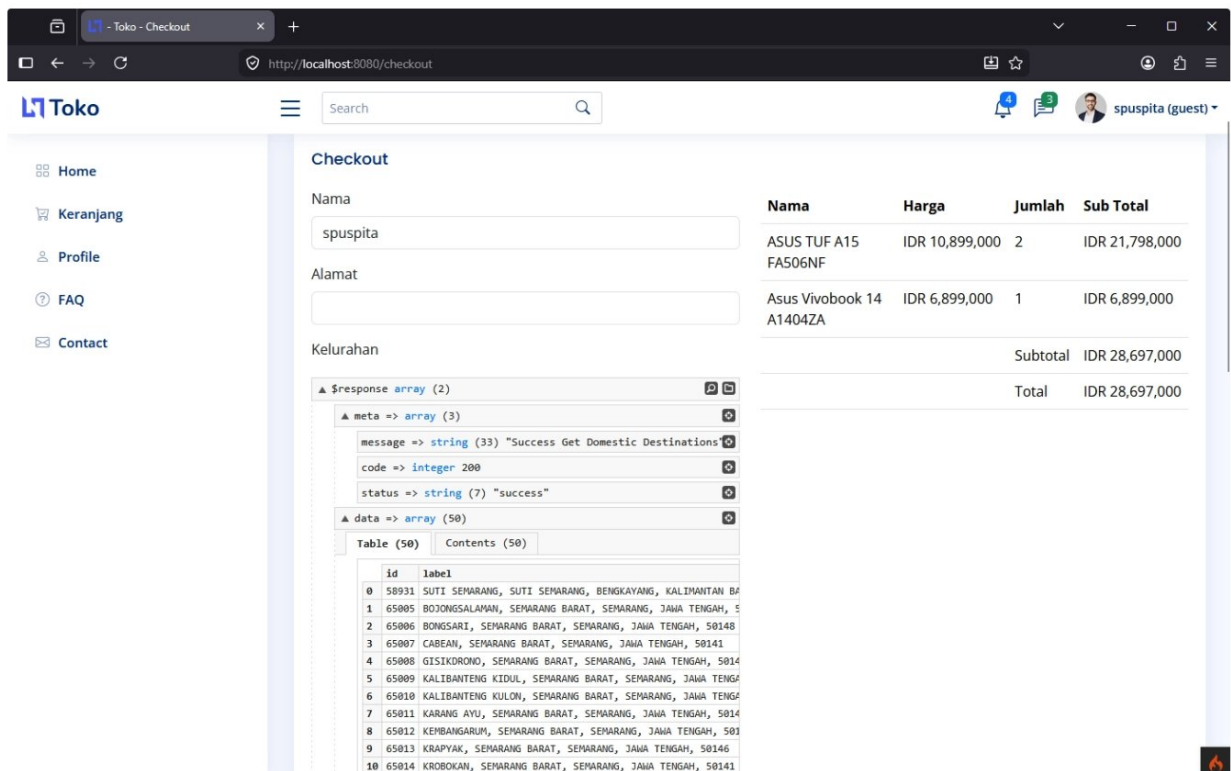
18. Coba lihat hasilnya pada halaman checkout, `app\Views\v_checkout.php`. Letakkan pada bagian form di Kelurahan dan Layanan.

```
<?= d($response) ?>
```

```
<?= d($response2) ?>
```



The screenshot shows a code editor with a file named `v_checkout.php`. The code is a Laravel Blade template for a checkout page. It contains several form elements: an address field, a complaint field, a service field, and an order amount field. The `<?= d($response2) ?>` syntax is used to display data from a response object. The Explorer panel on the right shows the project structure, including the `app` directory with `Views`, `Controllers`, `Database`, `Filters`, `Helpers`, `Language`, `Libraries`, `Models`, `Services`, `ThirdParty`, and `Views` subdirectories. The `index.html` file is also visible in the `public` directory.



The screenshot shows the checkout page of a web application. The page has a sidebar with navigation links: Home, Keranjang, Profile, FAQ, and Contact. The main content area is titled "Checkout" and contains a form with fields for Nama, Alamat, and Kelurahan. Below the form, there is a table showing the items in the cart. The table has columns for Nama, Harga, Jumlah, and Sub Total. The items are ASUS TUF A15 FA506NF and Asus Vivobook 14 A1404ZA. The subtotal is IDR 28,697,000 and the total is IDR 28,697,000. Below the table, there is a section for the response data, which is an array (2) containing meta and data. The meta array (3) contains message, code, and status. The data array (50) contains a table with 50 rows and 2 columns: id and label. The table contains a list of destinations with their IDs and labels.

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	2	IDR 21,798,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
Subtotal			IDR 28,697,000
Total			IDR 28,697,000

▲ \$response array (2)

▲ meta => array (3)

message => string (33) "Success Get Domestic Destinations"

code => integer 200

status => string (7) "success"

▲ data => array (50)

id	label
58931	SUTI SEMARANG, SUTI SEMARANG, BENGKAYANG, KALIMANTAN BA
65805	BOJONGSALAMAN, SEMARANG BARAT, SEMARANG, JAWA TENGAH, S
65806	BONGSARI, SEMARANG BARAT, SEMARANG, JAWA TENGAH, 50148
65807	CABEAN, SEMARANG BARAT, SEMARANG, JAWA TENGAH, 50141
65808	GISIKORONO, SEMARANG BARAT, SEMARANG, JAWA TENGAH, 5014
65809	KALIBANTENG KIDUL, SEMARANG BARAT, SEMARANG, JAWA TENGA
65810	KALIBANTENG KULON, SEMARANG BARAT, SEMARANG, JAWA TENGA
65811	KARANG AYU, SEMARANG BARAT, SEMARANG, JAWA TENGAH, 5014
65812	KEMBANGARUM, SEMARANG BARAT, SEMARANG, JAWA TENGAH, 501
65813	KRAPYAK, SEMARANG BARAT, SEMARANG, JAWA TENGAH, 50146
65814	KROBOKAN, SEMARANG BARAT, SEMARANG, JAWA TENGAH, 50141



Toko Search

Home Keranjang Profile FAQ Contact

```

36 65040 PANDANSARI, SEMARANG TENGAH, SEMARANG, JAWA TENGAH, 50139
37 65041 PEKUNDEN, SEMARANG TENGAH, SEMARANG, JAWA TENGAH, 50241
38 65042 PENDRIKAN KIDUL, SEMARANG TENGAH, SEMARANG, JAWA TENGAH, 50131
39 65043 PENDRIKAN LOR, SEMARANG TENGAH, SEMARANG, JAWA TENGAH, 50131
40 65044 PURWODINATAN, SEMARANG TENGAH, SEMARANG, JAWA TENGAH, 50137
41 65045 SEKAYU, SEMARANG TENGAH, SEMARANG, JAWA TENGAH, 50132
42 65046 BUGANGAN, SEMARANG TIMUR, SEMARANG, JAWA TENGAH, 50126
43 65047 KARANGTEPEL, SEMARANG TIMUR, SEMARANG, JAWA TENGAH, 50232
44 65048 KARANGTURI, SEMARANG TIMUR, SEMARANG, JAWA TENGAH, 50124
45 65049 KEBONAGUNG, SEMARANG TIMUR, SEMARANG, JAWA TENGAH, 50123
46 65050 KEMIJEN, SEMARANG TIMUR, SEMARANG, JAWA TENGAH, 50228
47 65051 MLATIBARU, SEMARANG TIMUR, SEMARANG, JAWA TENGAH, 50122
48 65052 MLATIHARJO, SEMARANG TIMUR, SEMARANG, JAWA TENGAH, 50126
49 65053 REJOMULYO, SEMARANG TIMUR, SEMARANG, JAWA TENGAH, 50227

```

Called from .../app/Views/v_checkout.php:34 [41]

0

Layanan

```

$response2 array (2)
  meta => array (3)
  data => array (7)

```

name	code	service description	cost	etd
0 Jalur Nugraha Ekakurir (JNE)	JNE CTC	JNE City Courier	9000	1 da
1 Jalur Nugraha Ekakurir (JNE)	JNE JTR	JNE Trucking	40000	3 da
2 Jalur Nugraha Ekakurir (JNE)	JNE CTCSPS	JNE City Courier	25000	0 da
3 Jalur Nugraha Ekakurir (JNE)	JNE CTCYES	JNE City Courier	11000	1 da
4 Jalur Nugraha Ekakurir (JNE)	JNE JTR130	JNE Trucking	200000	3 da
5 Jalur Nugraha Ekakurir (JNE)	JNE JTR130	JNE Trucking	350000	3 da
6 Jalur Nugraha Ekakurir (JNE)	JNE JTR200	JNE Trucking	500000	3 da

Called from .../app/Views/v_checkout.php:39 [41]

0

Karena berhasil mengakses data dari webservice RajaOngkir, maka perhatikan angka Total API Hit/Day di akunRajaOngkir akan bertambah jumlahnya. Ingat ya, batasnya hanya 100 kali hit per hari.

RajaOngkir - Collaborator API

collaborator.kommerce.id/settings

Hallo Apriliani Sandbox Mode

Settings

API Key Access

Api Key List

Manage API Keys for every services that you use.

API Name	API Key	Added	Expires	Total API HIT / Day
1 Shipping Cost	> Show Key	02/06/2025	-	12 / 100
2 Payment API	> Show Key	02/06/2025	-	0 / ∞
3 QRISLY API	> Show Key	02/06/2025	-	0 / ∞

Sementara kembalikan dulu code di file app\Views\v_checkout.php dan app\Controllers\TransaksiController.php. Karena pemanggilan webservicenya tidak dilakukan secara manual seperti tadi, melainkan menggunakan elemen select option (dropdown) yang

menggunakan library select2.

The first screenshot shows the `v_checkout.php` Blade template. It contains four columns of form elements. The first column has a label 'Alamat' and an input field. The second column has a label 'Kelurahan' and a `<strong>select kelurahan</strong>` element. The third column has a label 'Layanan' and a `<strong>select layanan</strong>` element. The fourth column has a label 'Ongkir' and an input field. The second and third columns are highlighted with blue selection boxes.

```
3 <div class="row">
4   <div class="col-lg-6">
23
24     <div class="col-12">
25       <?= form_label('Alamat', 'alamat', ['class' => 'form-label']) ?>
26       <?= form_input([
27         'name' => 'alamat',
28         'id' => 'alamat',
29         'class' => 'form-control']) ?>
30     </div>
31
32     <div class="col-12">
33       <?= form_label('Kelurahan', 'kelurahan', ['class' => 'form-label']) ?>
34       <strong>select kelurahan</strong>
35     </div>
36
37     <div class="col-12">
38       <?= form_label('Layanan', 'layanan', ['class' => 'form-label']) ?>
39       <strong>select layanan</strong>
40     </div>
41
42     <div class="col-12">
43       <?= form_label('Ongkir', 'ongkir', ['class' => 'form-label']) ?>
44       <?= form_input([
45         'name' => 'ongkir',
46         'id' => 'ongkir',
```

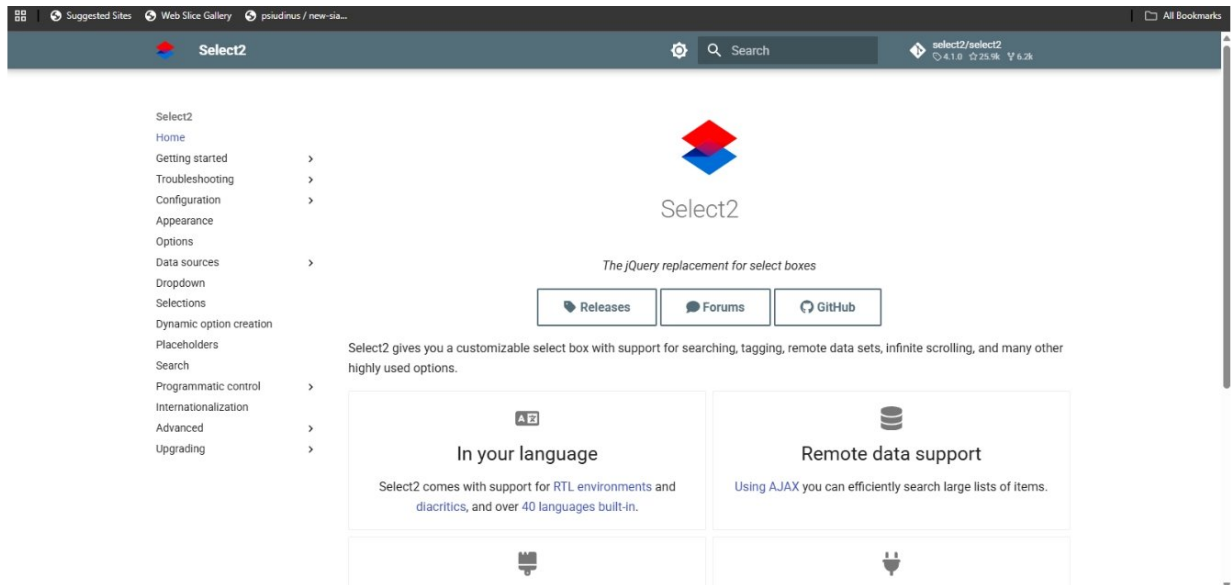
The second screenshot shows the `TransaksiController.php` PHP file. It contains a `checkout()` method that sets a success message and redirects to the 'keranjang' page. The `checkout()` method is highlighted with a blue selection box.

```
10 {
85 {
87
88     session()->setFlashdata(
89         'success',
90         'Keranjang berhasil dikosongkan'
91     );
92
93     return redirect()->to(base_url('keranjang'));
94 }
95
96 public function checkout()
97 {
98     $data = [
99         'items' => $this->cart->contents(),
100         'total' => $this->cart->total()
101     ];
102
103     return view('v_checkout', $data);
104 }
```

19. Untuk pencarian lokasi pengiriman pada form, akan lebih mudah jika menggunakan elemen yang disediakan oleh select2 dibandingkan dengan elemen select option yang biasa.

<https://select2.org/>





Contohnya nanti akan menghasilkan tampilan input seperti ini

Kelurahan

banyumanik

BANYUMANIK, BANYUMANIK, SEMARANG, JAWA TENGAH, 50264

GEDAWANG, BANYUMANIK, SEMARANG, JAWA TENGAH, 50266

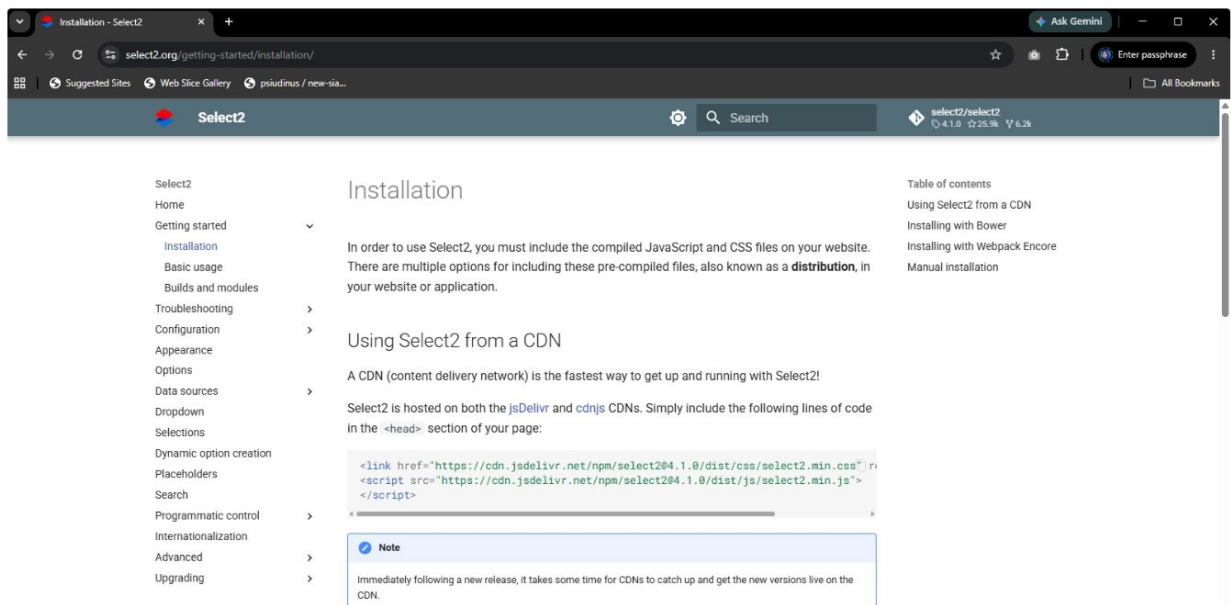
JABUNGAN, BANYUMANIK, SEMARANG, JAWA TENGAH, 50266

NGESREP, BANYUMANIK, SEMARANG, JAWA TENGAH, 50261

PADANGSARI, BANYUMANIK, SEMARANG, JAWA TENGAH, 50267

PEDALANGAN, BANYUMANIK, SEMARANG, JAWA TENGAH, 50268

<https://select2.org/getting-started/installation>



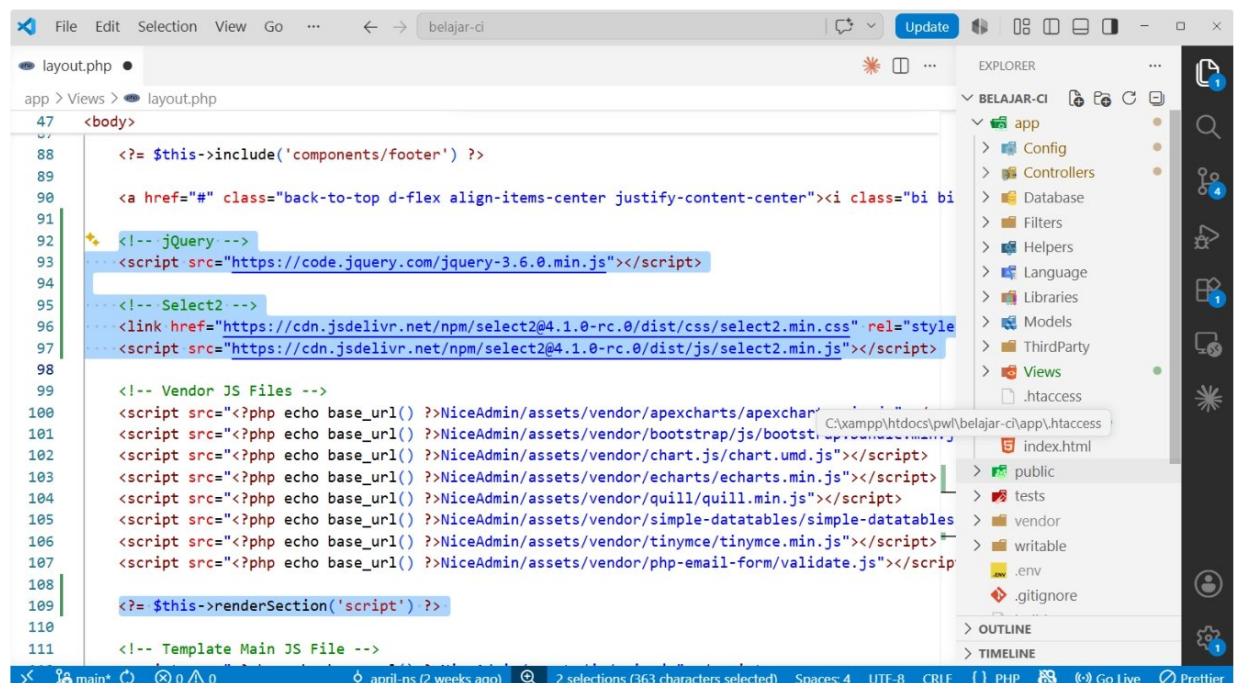
Sesuai dengan petunjuk instalasi dari dokumentasi select2, tambahkan CDN pada file app\Views\layout.php.

```
<!-- jQuery -->
<script src="https://code.jquery.com/jquery-4.0.0.min.js"></script>

<!-- Select2 -->
<link href="https://cdn.jsdelivr.net/npm/select2@4.1.0/dist/css/select2.min.css" rel="style" />
<script src="https://cdn.jsdelivr.net/npm/select2@4.1.0/dist/js/select2.min.js"></script>
```

Serta tambahkan section untuk me-render script yang dibuat pada file-file view yang menggunakan (extend) layout.php.

```
<?= $this->renderSection('script') ?>
```



20. Mulai untuk membuat request GET data yang pertama yaitu data tujuan pengiriman. Pada langkah sebelumnya, di file `app\Views\lv_checkout.php` sudah disiapkan tampilan untuk pilihan kelurahan.

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	2	IDR 21,798,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
Subtotal			IDR 28,697,000
Total			IDR 28,697,000

Ganti baris tersebut dengan inputan `form_dropdown` yang akan menghasilkan elemen `<select><option>` untuk pilihan daerah tujuan pengiriman.

```
<?= form_dropdown('kelurahan', [], '', ['id' => 'kelurahan', 'class' => 'fori
```

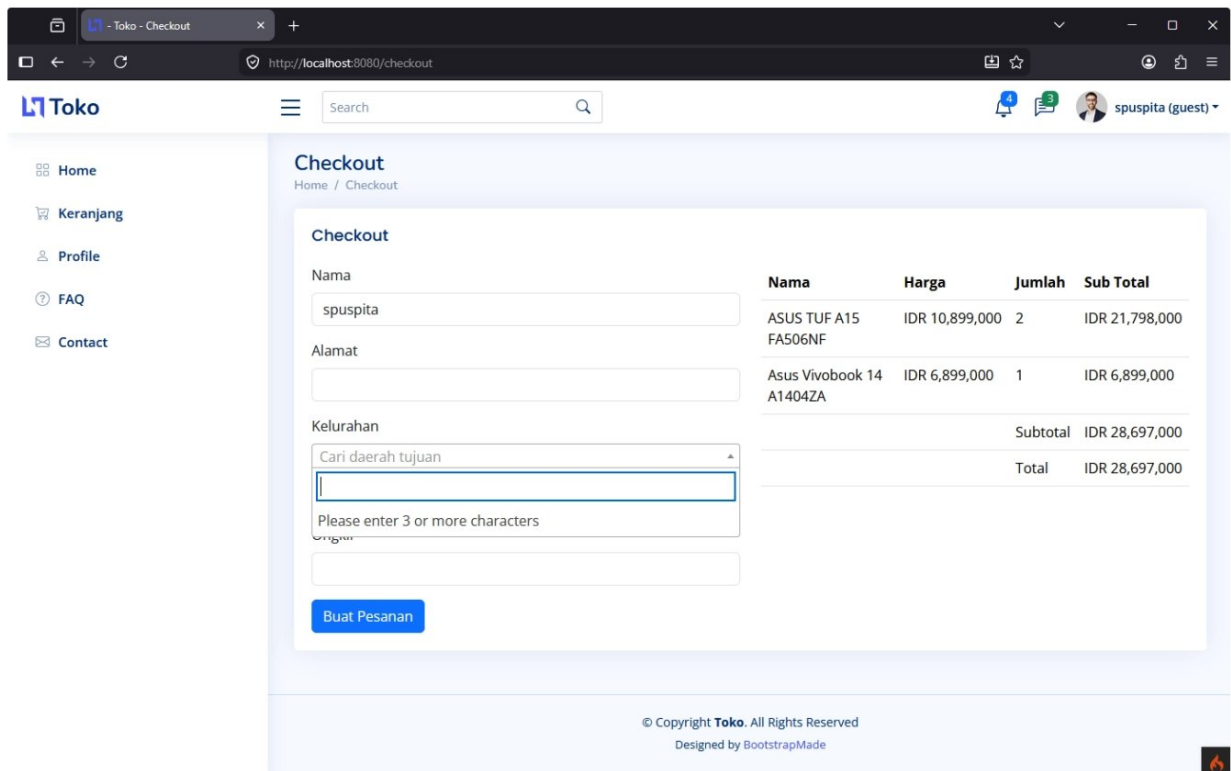
```
3 <div class="row">
4   <div class="col-lg-6">
25     <div class="col-12">
26       <?= form_label('Alamat', 'alamat', ['class' => 'form-label']) ?>
27       <?= form_input([
28         'name' => 'alamat',
29         'id' => 'alamat',
30         'class' => 'form-control']) ?>
31     </div>
32   </div>
```

```
33 <div class="col-12">
34 <?> form_label('Kelurahan', 'kelurahan', ['class' => 'form-label']) ?>
35 <?> form_dropdown('kelurahan', [], '', ['id' => 'kelurahan', 'class' => 'form-control']) ?>
36 </div>
37
38 <div class="col-12">
39 <?> form_label('Layanan', 'layanan', ['class' => 'form-label']) ?>
40 <strong>select layanan</strong>
41 </div>
42
43 <div class="col-12">
44 <?> form_label('Ongkir', 'ongkir', ['class' => 'form-label']) ?>
45 <?> form_input([
46     'name' => 'ongkir',
47     'id' => 'ongkir',
48     'class' => 'form-control']) ?>
49 </div>
```

Buat tampilan inputan tadi menggunakan plugin select2 yang sudah disiapkan sebelumnya.

```
<?> $this->section('script') ?>
<script>
$(document).ready(function() {
    $('#kelurahan').select2({
        placeholder: 'Cari daerah tujuan',
        minimumInputLength: 3,
    });
});
</script>
<?> $this->endSection() ?>
```

```
File Edit Selection View Go ... Update
v_checkout.php U
app > Views > v_checkout.php
3 <div class="row">
62 <div class="col-lg-6">
63 <table class="table">
72 <tbody>
92 <tr>
93 <td colspan="2"></td>
94 <td>Total</td>
95 <td><span id="total"><?> number_to_currency($total, 'IDR') ?></span></td>
96 </tr>
97 </tbody>
98 </table>
99 </div>
100 </div>
101 <?> $this->endSection() ?>
102 <?> $this->section('script') ?>
103 <script>
104 $(document).ready(function() {
105     $('#kelurahan').select2({
106         placeholder: 'Cari daerah tujuan',
107         minimumInputLength: 3,
108     });
109 });
110 </script>
111 <?> $this->endSection() ?>
```



Ini elemen yang akan digunakan untuk menangkap keyword dari user untuk membuat request GET daerah tujuan ke webservice RajaOngkir. Proses tersebut akan dibuat menggunakan kombinasi dari JQuery dan AJAX supaya dapat mengambil data destinasi secara dinamis saat user mengetikkan keywordnya, sehingga halaman lebih cepat, lebih ringan, dan tidak perlu memuat seluruh data destinasi sekaligus saat halaman dibuka.

JQuery adalah library untuk menggunakan code Javascript supaya lebih ringkas. Sedangkan **AJAX (Asynchronous Javascript And XML)** adalah sebuah teknik interaktif yang digunakan untuk membuat aplikasi web. Dengan AJAX, aplikasi web dapat mengambil data dari server secara **asynchronous** di belakang tanpa perlu refresh halaman. Awalnya AJAX hanya menggunakan XML untuk pertukaran datanya, namun saat ini lebih banyak developer yang menggunakan JSON dibandingkan XML, karena lebih mudah dan ringkas penulisannya. Response yang diberikan dari endpoint Raja Ongkir juga dalam format JSON.

Perbandingan

XML

vs.

JSON

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <endereco>
3   <cep>31270901</cep>
4   <city>Belo Horizonte</city>
5   <neighborhood>Pampulha</neighborhood>
6   <service>correios</service>
7   <state>MG</state>
8   <street>Av. Presidente Antônio Carlos, 6627</street>
9 </endereco>
```

```
1 {
2   "endereco": {
3     "cep": "31270901",
4     "city": "Belo Horizonte",
5     "neighborhood": "Pampulha",
6     "service": "correios",
7     "state": "MG",
8     "street": "Av. Presidente Antônio Carlos, 6627"
9   }
10 }
```

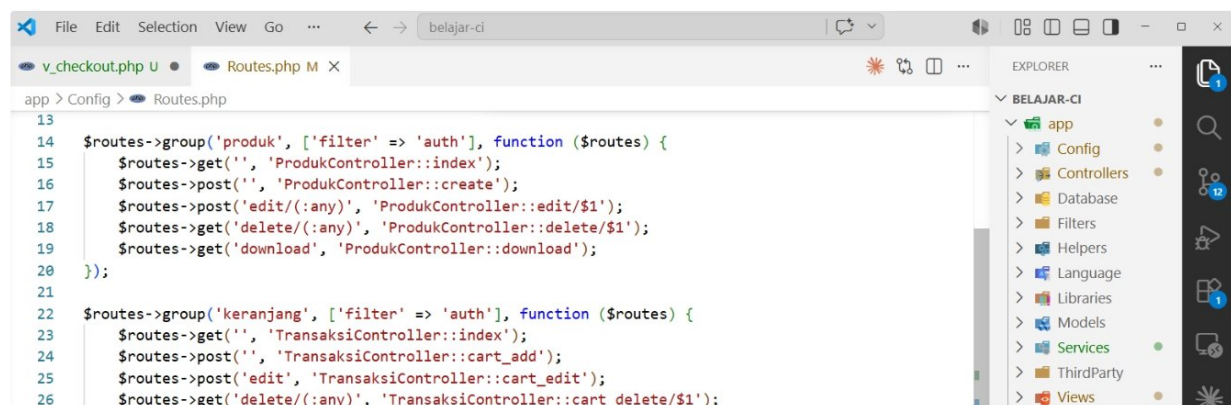
Jadi misalnya pada database tersimpan 50.000 data daerah tujuan, kalau semua data dimuat saat halaman dibuka maka halaman menjadi berat dan boros memori. Dengan AJAX, bisa melakukan request untuk mengambil data destinasi sesuai keyword yang diketikkan user saja. Misalnya jika user mengetikkan “semarang” maka akan melakukan request GET untuk data destination yang mengandung kata “semarang” saja, jadi hasilnya lebih sedikit dibandingkan memanggil 50.000 data. Dan hal itu dapat dilakukan tanpa refresh/reload halaman sehingga dapat memberikan user experience yang lebih nyaman.

Alur yang akan dibuat adalah :

1. User mencari kelurahan dengan mengetikkan keyword di element select Kelurahan (select2).
2. Pilihan kelurahan sesuai dengan keyword yang diketikkan user akan muncul pada select Kelurahan.
3. User memilih kelurahan yang diinginkan, otomatis pilihan layanan yang tersedia pada kelurahan tersebut akan muncul pada select Layanan.
4. User memilih layanan yang diinginkan, otomatis kolom ongkir akan terisi nominal sesuai dengan layanan yang dipilih oleh user.
5. Total harga pada tabel sebelah kanan akan bertambah dengan nominal ongkir yang muncul.

21. Sebelum membuat request menggunakan AJAX, siapkan dulu rutennya. Tambahkan route untuk request GET daerah tujuan di file app\Config\Routes.php.

```
$routes->get('ajax/destinations','TransaksiController::destinations', ['filter'
```



```
27 | $routes->get('clear', 'TransaksiController::cart_clear');
28 | $routes->get('checkout', 'TransaksiController::checkout');
29 | });
30 |
31 | $routes->get('checkout', 'TransaksiController::checkout', ['filter' => 'auth']);
32 |
33 | $routes->get('ajax/destinations', 'TransaksiController::destinations', ['filter' => 'auth']);
```



22. Gunakan rute yang baru saja dibuat untuk membuat code AJAX di bagian konfigurasi elemen select2 di file app\Views\lv_checkout.php.

```
ajax: {
    url: '<?= site_url('ajax/destinations') ?>',
    dataType: 'json',
    delay: 300,
    data: function(params) {
        return {
            q: params.term
        };
    },
    processResults: function(data) {
        return data;
    },
    cache: true
}
```

- Disini diatur bahwa keyword yang diketikkan user akan dikirim ke URL `http://localhost:8080/ajax/destinations` sesuai dengan route yang sudah disiapkan.
- Tipe data yang akan diterima (response dari server) berupa JSON.
- Berikan delay 300ms (3 detik) artinya tunggu 300 ms setelah user berhenti mengetik sebelum mengirim request, tujuannya agar tidak mengirim request setiap huruf.

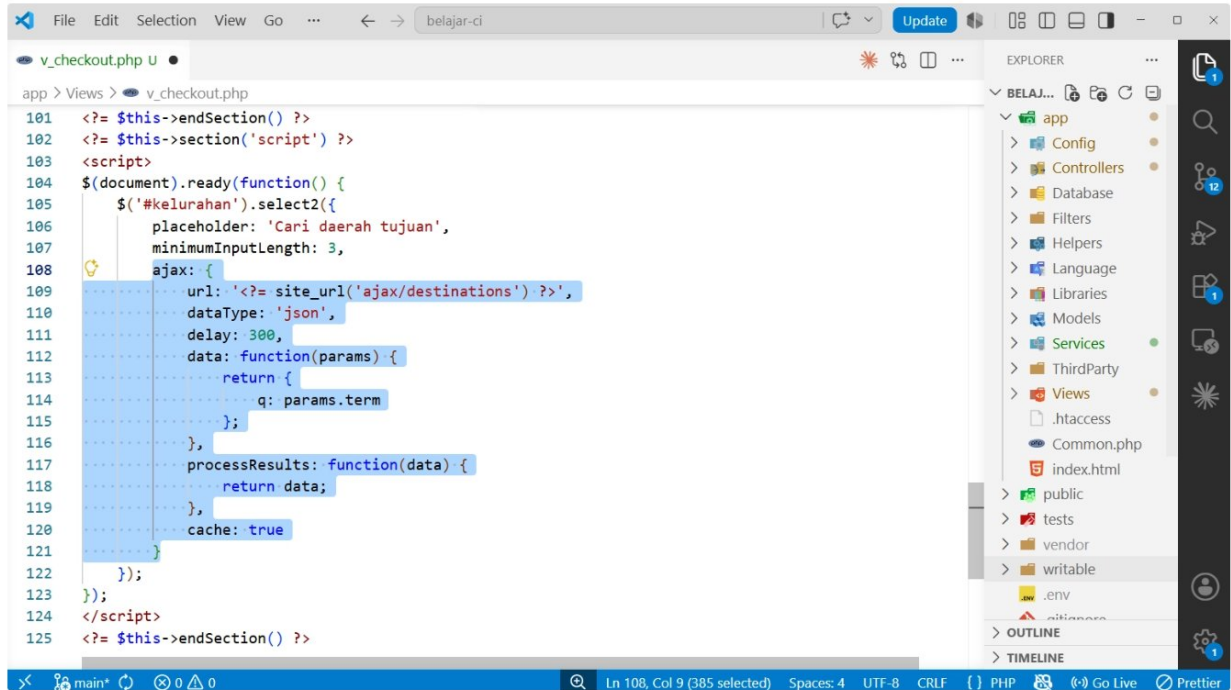
//Misalnya user mengetik keyword:

```
s
se
sem
sema
```

//jika tanpa delay maka akan ada 4 request GET ke server

```
//jika dengan delay maka hanya akan ada 1 request GET ke server
```

- Data yang diketik user dikirim ke server menggunakan parameter “q”. Jadi saat user mengetik keyword “semarang” maka requestnya menjadi `http://localhost:8080/ajax/destinations?q=semarang`
- Cache dibuat true artinya hasil request sebelumnya yang pernah dilakukan akan disimpan di browser. Jadi jika user mencari lagi keyword yang sama, maka browser bisa memakai data yang sudah ada tanpa request baru ke server.



```
101 <?= $this->endSection() ?>
102 <?= $this->section('script') ?>
103 <script>
104 $(document).ready(function() {
105     $('#kelurahan').select2({
106         placeholder: 'Cari daerah tujuan',
107         minimumInputLength: 3,
108         ajax: {
109             url: '<?= site_url('ajax/destinations') ?>',
110             dataType: 'json',
111             delay: 300,
112             data: function(params) {
113                 return {
114                     q: params.term
115                 };
116             },
117             processResults: function(data) {
118                 return data;
119             },
120             cache: true
121         }
122     });
123 });
124 </script>
125 <?= $this->endSection() ?>
```

23. Sesuai dengan route yang sudah dibuat maka tambahkan fungsi destinations() pada file `app\Controllers\TransaksiController.php`. Sementara isi dengan code sederhana hanya untuk cek apakah keyword (q) berhasil dikirimkan ke controller. Buat response dengan jenis JSON yang memiliki key “id” dan “text” karena dropdown dengan select2 itu memiliki format data standar seperti itu.

```
public function destinations()
{
    $search = $this->request->getGet('q');

    $results = [
        'id' => $search,
```

```

        'text' => $search
    ];

    return $this->response->setJSON([
        'results' => $results
    ]);
}

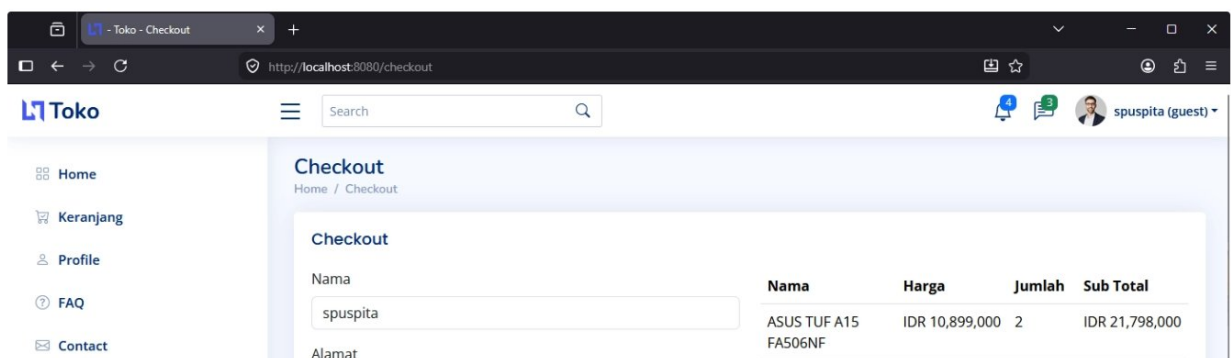
```

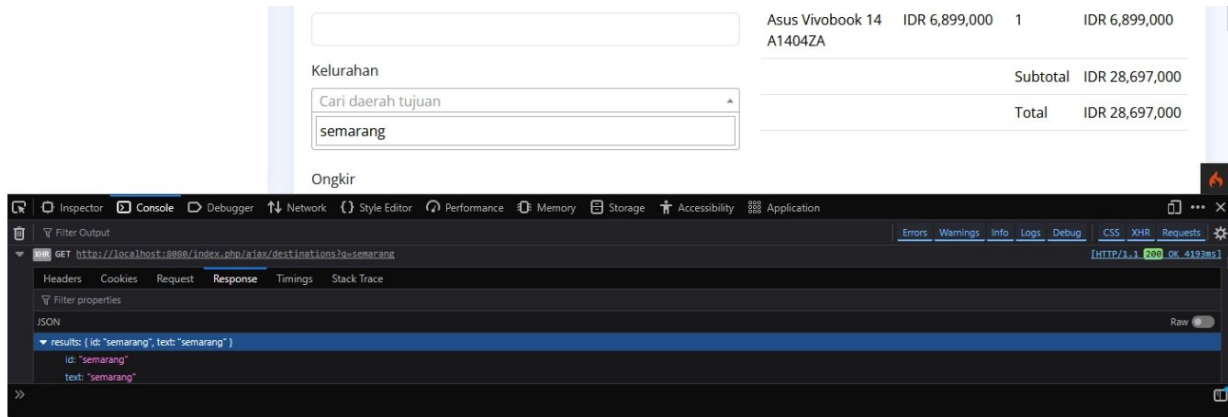
```

106 public function destinations()
107 {
108     $search = $this->request->getGet('q');
109
110     $results = [
111         'id' => $search,
112         'text' => $search
113     ];
114
115     return $this->response->setJSON([
116         'results' => $results
117     ]);
118 }

```

Pada browser, buka developer tool (klik kanan - pilih Inspect) lalu pilih tab Console. Coba ketikkan “semarang” pada tampilan Kelurahan. Pada tab console akan muncul request GET ke URL <http://localhost:8080/ajax/destinations?q=semarang>. Kemudian lihat pada detailnya, di tab Response akan muncul data JSON sesuai yang diatur di controller yaitu `{id:"semarang";text:"semarang"}`





24. Data keyword sudah berhasil dikirim ke controller dan controller juga sudah berhasil mengirimkan response ke view. Lanjutkan dengan menambahkan code untuk membuat request yang sebenarnya menggunakan webservice RajaOngkir. Tambahkan pada fungsi `destination()` di file `app\Controllers\TransaksiController.php`, mirip dengan code uji coba yang sudah pernah dilakukan pada langkah sebelumnya.

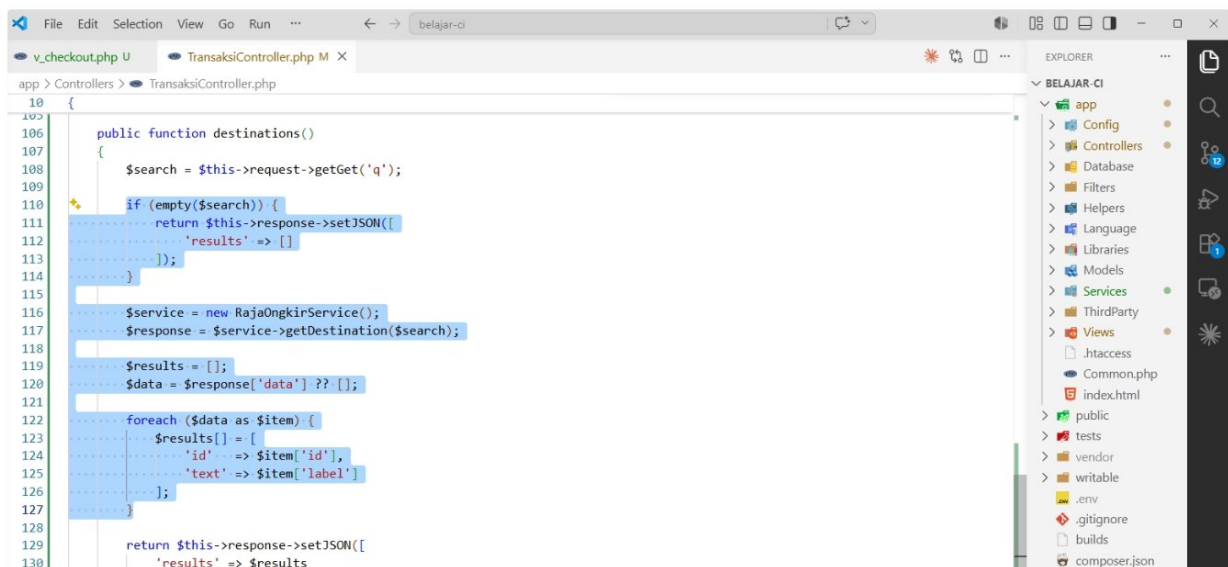
```

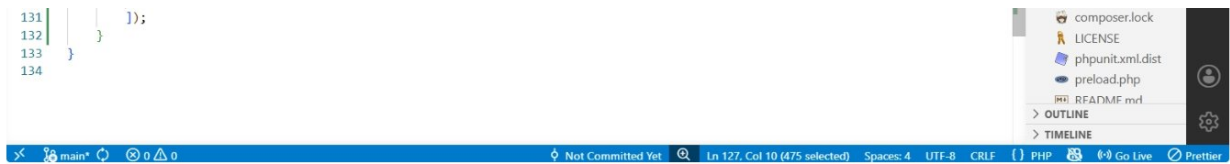
$service = new RajaOngkirService();
$response = $service->getDestination($search);

$results = [];
$data = $response['data'] ?? [];

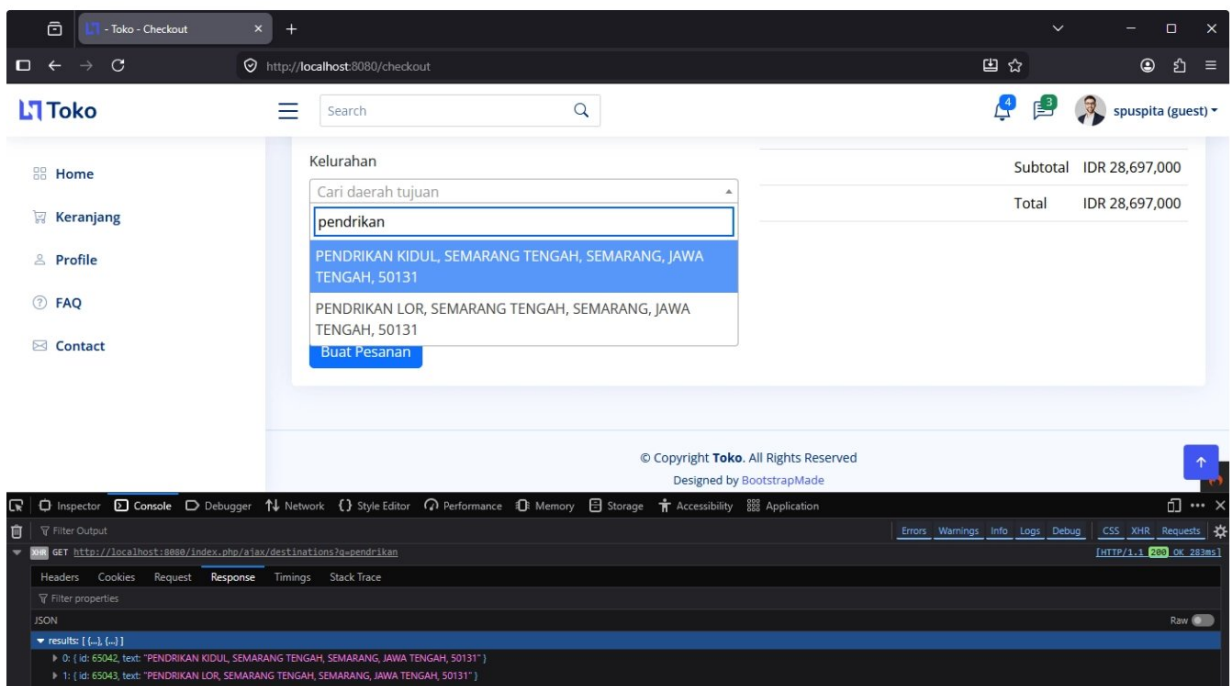
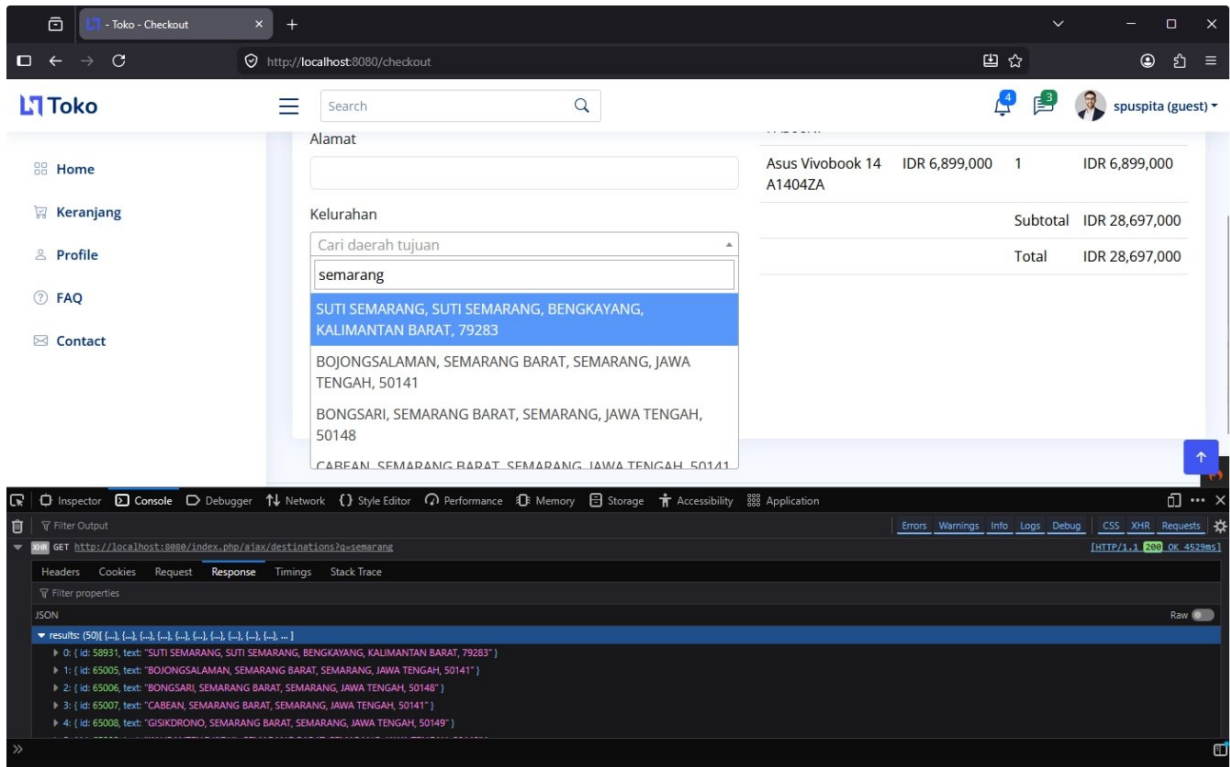
foreach ($data as $item) {
    $results[] = [
        'id' => $item['id'],
        'text' => $item['label']
    ];
}

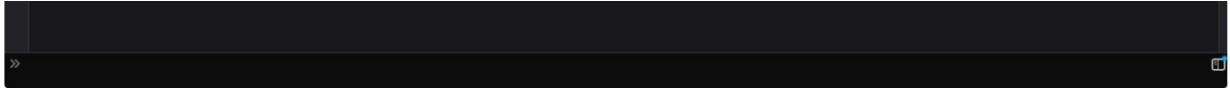
```





Coba refresh halaman lalu ketikkan keyword “semarang”. Perhatikan request yang muncul pada console tab pada developer tool browser, lalu lihat response yang diberikan.





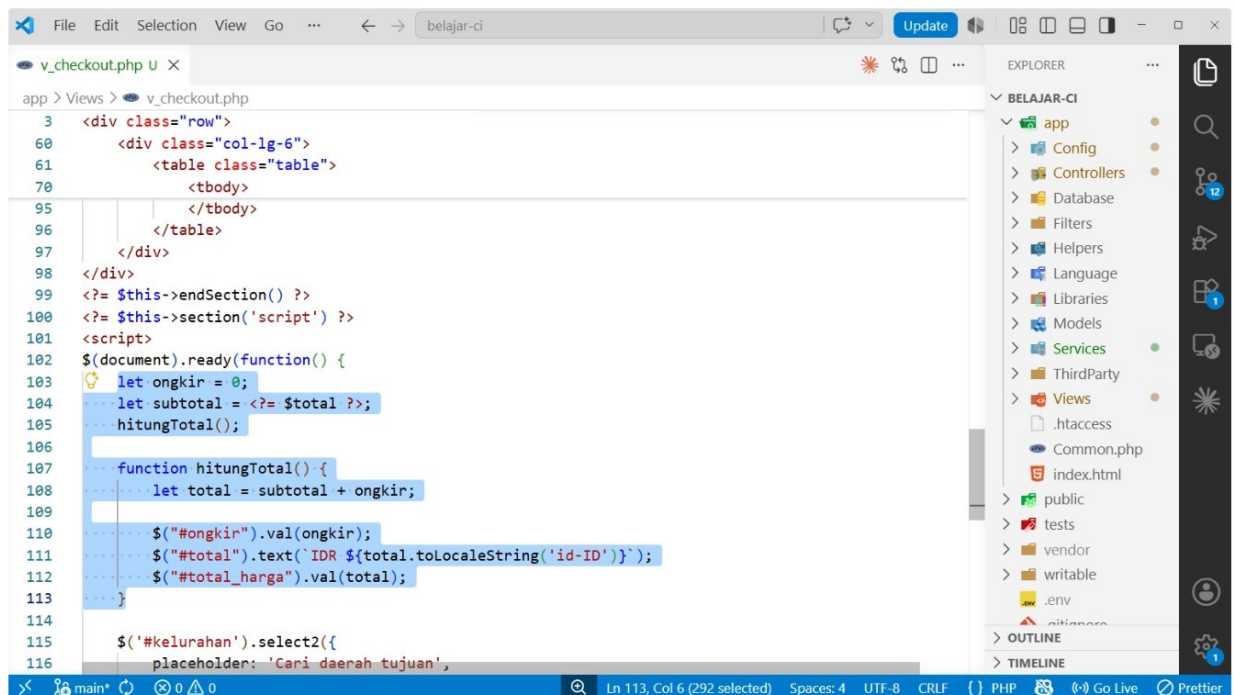
25. Data daerah tujuan (destination) sudah berhasil diambil dari webservice RajaOngkir. Lanjutkan dengan menambahkan code javascript yang nanti akan digunakan untuk menyimpan nilai ongkir dan subtotal, serta fungsi yang digunakan untuk menghitung nilai total.

Tambahkan pada file `app\Views\v_checkout.php`.

```
let ongkir = 0;
let subtotal = <?= $total ?>;
hitungTotal();

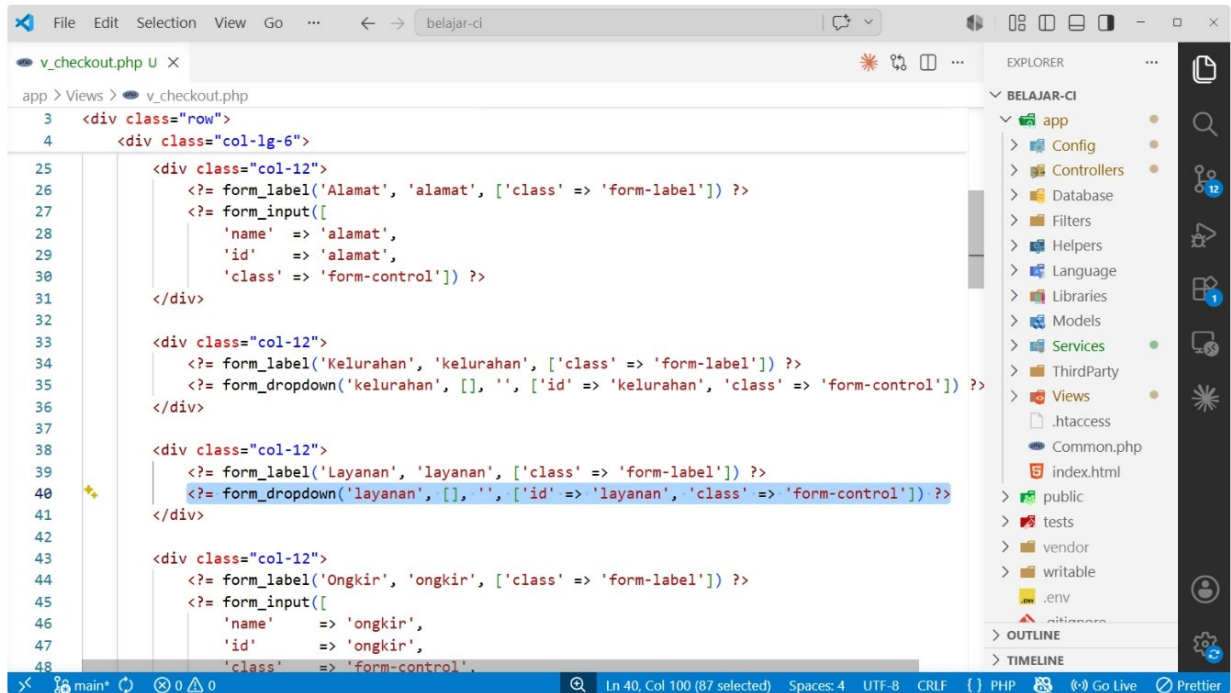
function hitungTotal() {
    let total = subtotal + ongkir;

    $("#ongkir").val(ongkir);
    $("#total").text(`IDR ${total.toLocaleString('id-ID')}`);
    $("#total_harga").val(total);
}
```

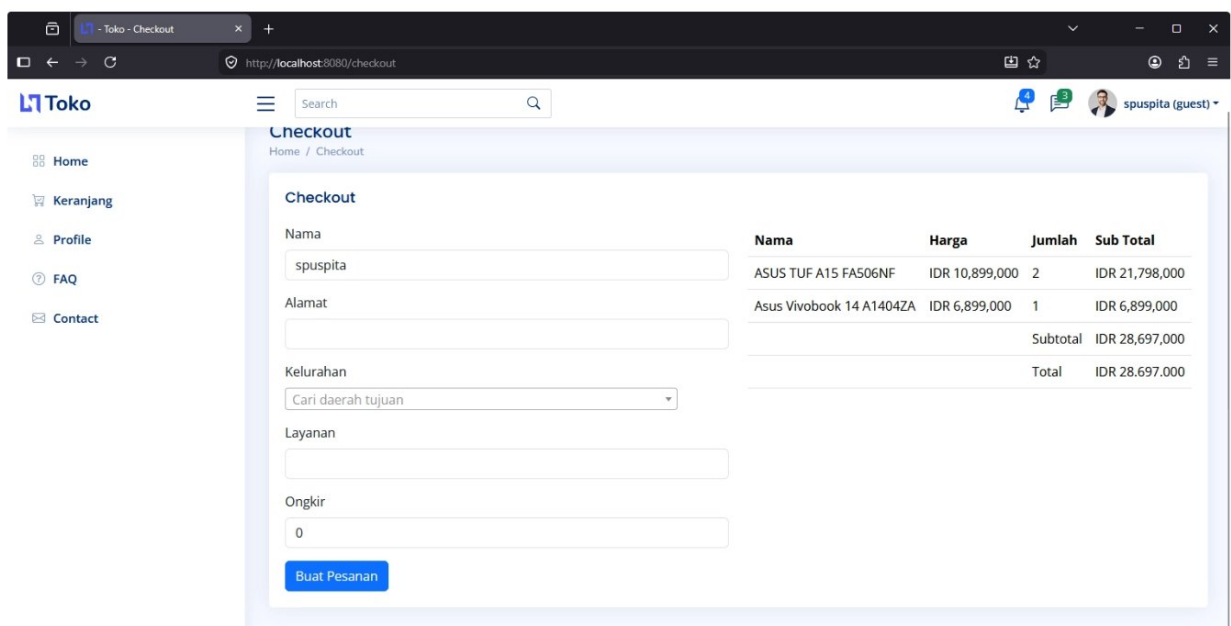


26. Masih pada file `app\Views\v_checkout.php`, tambahkan `form_dropdown` untuk menampilkan layanan pada tempat yang sudah disediakan sebelumnya. Dropdown ini akan terisi dengan data layanan yang tersedia sesuai dengan daerah yang dipilih oleh user pada pilihan kelurahan (destination).

```
<?= form_dropdown('layanan', [], '', ['id' => 'layanan', 'class' => 'form-control'])
```



```
3 <div class="row">
4   <div class="col-lg-6">
25     <div class="col-12">
26       <?= form_label('Alamat', 'alamat', ['class' => 'form-label']) ?>
27       <?= form_input([
28         'name' => 'alamat',
29         'id' => 'alamat',
30         'class' => 'form-control']) ?>
31     </div>
32
33     <div class="col-12">
34       <?= form_label('Kelurahan', 'kelurahan', ['class' => 'form-label']) ?>
35       <?= form_dropdown('kelurahan', [], '', ['id' => 'kelurahan', 'class' => 'form-control']) ?>
36     </div>
37
38     <div class="col-12">
39       <?= form_label('Layanan', 'layanan', ['class' => 'form-label']) ?>
40       <?= form_dropdown('layanan', [], '', ['id' => 'layanan', 'class' => 'form-control']) ?>
41     </div>
42
43     <div class="col-12">
44       <?= form_label('Ongkir', 'ongkir', ['class' => 'form-label']) ?>
45       <?= form_input([
46         'name' => 'ongkir',
47         'id' => 'ongkir',
48         'class' => 'form-control',
```



Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	2	IDR 21,798,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
Subtotal			IDR 28,697,000
Total			IDR 28.697.000



27. Nantinya pilihan layanan akan tampil seperti ini, misalnya jika user memilih tujuannya adalah "Pendrikan Kidul, Semarang Tengah, Jawa Tengah". Lalu jika user mengganti daerah tujuannya maka pilihan layanannya pun akan berubah. Maka request yang dilakukan itu akan dilakukan dengan trigger (pemicu) dari event "On Change" pada elemen kelurahan.

The screenshot shows the 'Checkout' page of a web application. On the left is a sidebar with links: Home, Keranjang, Profile, FAQ, and Contact. The main content area is titled 'Checkout' and contains a form with fields for Nama (filled with 'spuspita'), Alamat, and Kelurahan (filled with 'PENDRIKAN KIDUL, SEMARANG TENGAH, SEMARANG, JAWA TENGAH...'). Below the Kelurahan field is a 'Layanan' dropdown menu that is open, showing a list of shipping services with their estimated delivery times. To the right of the form is a table summarizing the items in the cart.

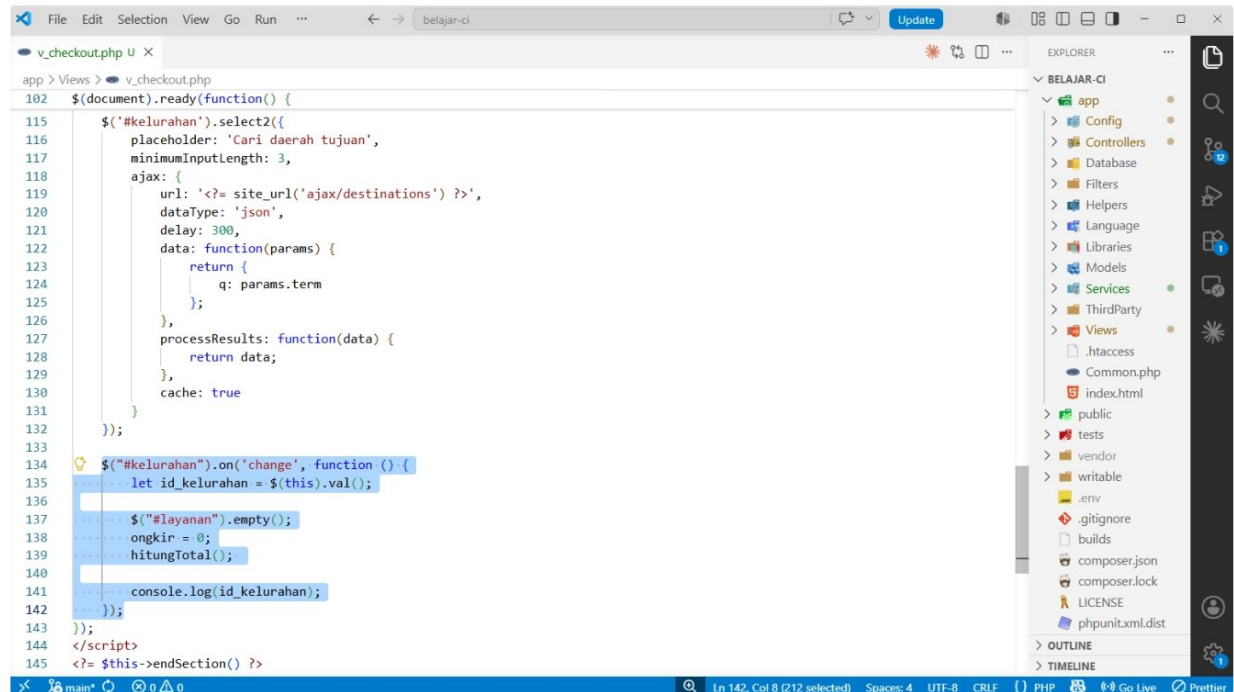
Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	2	IDR 21,798,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
Subtotal			IDR 28,697,000
Total			IDR 28,697,000

Tambahkan code javascript untuk event on change pada elemen kelurahan pada file app\Views\lv_checkout.php. Letakkan di bawah fungsi select2 yang dibuat sebelumnya.

```
$("#kelurahan").on('change', function () {  
    let id_kelurahan = $(this).val();  
  
    $("#layanan").empty();  
    ongkir = 0;  
    hitungTotal();  
  
    console.log(id_kelurahan);  
});
```

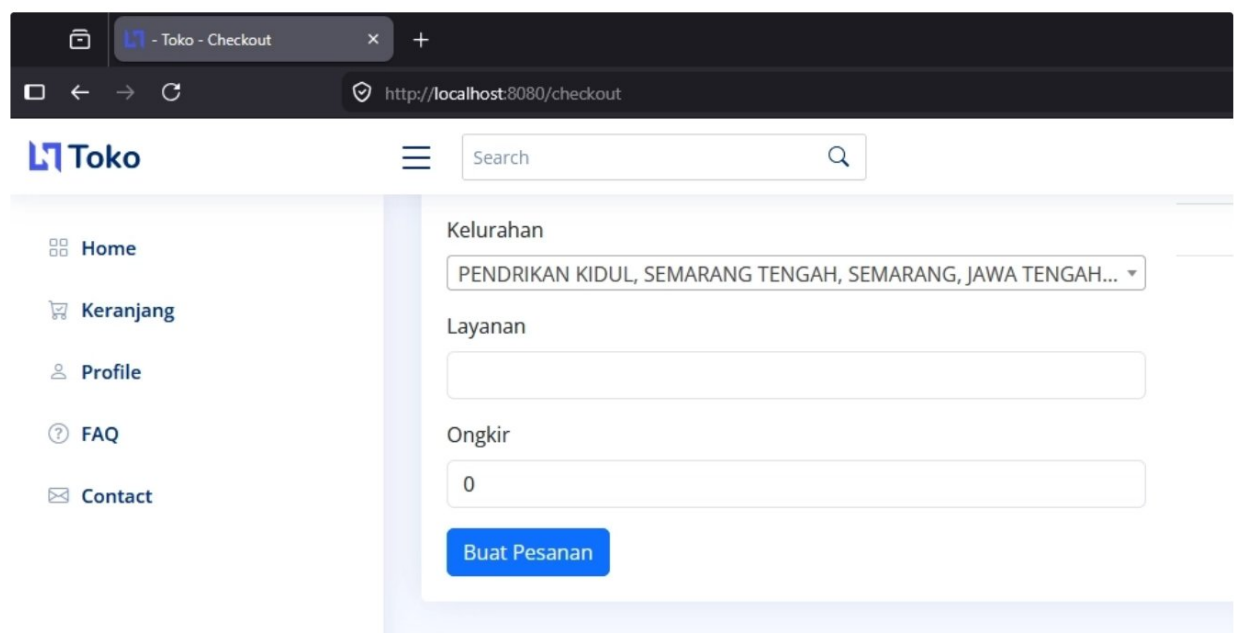
Jadi saat user memilih salah satu nilai dari elemen kelurahan, maka elemen isinya akan direset (empty) lalu nilai ongkir direset juga jadi 0 kemudian dihitung ulang nilai totalnya.

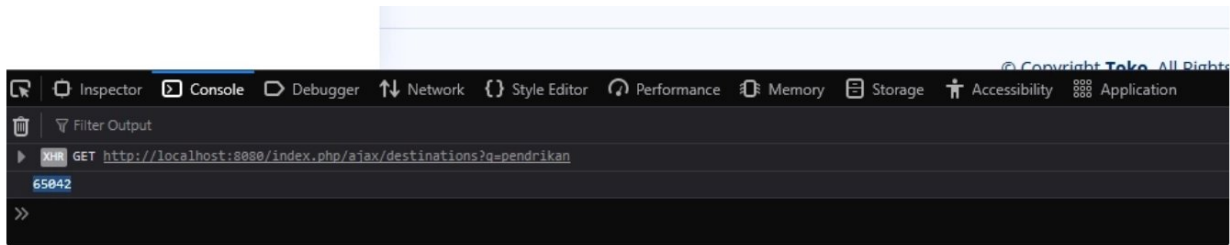
Cek dulu apakah id_kelurahan (id destination) berhasil diambil ketika event on change pada elemen kelurahan, tampilkan menggunakan console.log().



```
102 $(document).ready(function() {
115     $('#kelurahan').select2({
116         placeholder: 'Cari daerah tujuan',
117         minimumInputLength: 3,
118         ajax: {
119             url: '<?>= site_url('ajax/destinations') ?>',
120             dataType: 'json',
121             delay: 300,
122             data: function(params) {
123                 return {
124                     q: params.term
125                 };
126             },
127             processResults: function(data) {
128                 return data;
129             },
130             cache: true
131         }
132     });
133     $('#kelurahan').on('change', function () {
134         let id_kelurahan = $(this).val();
135         $('#layanan').empty();
136         ongkir = 0;
137         hitungTotal();
138         console.log(id_kelurahan);
139     });
140 }
141 </script>
142 <?>= $this->endSection() ?>
```

Refresh halaman, coba pilih sebuah daerah lalu lihat id yang sudah berhasil didapatkan akan muncul pada console tab.





28. Lanjutkan dengan mencari daftar layanan yang tersedia berdasarkan id destination yang sudah berhasil didapatkan tadi. Siapkan dulu sebuah fungsi baru bernama `cost()` pada file `app\Controllers\TransaksiController.php`.

Lakukan request POST ke webservice RajaOngkir dengan mengirimkan data :

- `origin` : id destinasi asal (statis = 64999 - PEDURUNGAN TENGAH)
- `destination` : id destinasi tujuan (didapatkan/GET dari halaman checkout)
- `weight` : berat barang (statis = 1000 gram)
- `courier` : jenis ekspedisi (statis = jne)

Lalu tampung response dari webservice RajaOngkir dalam sebuah response dengan jenis JSON yang memiliki key "service", "description", "cost" dan "etd".

```
public function costs()
{
    $origin = '64999';
    $destination = $this->request->getGet('destination');
    $weight = '1000';
    $courier = 'jne';

    $service = new RajaOngkirService();
    $response = $service->getCost($origin, $destination, $weight, $courier);

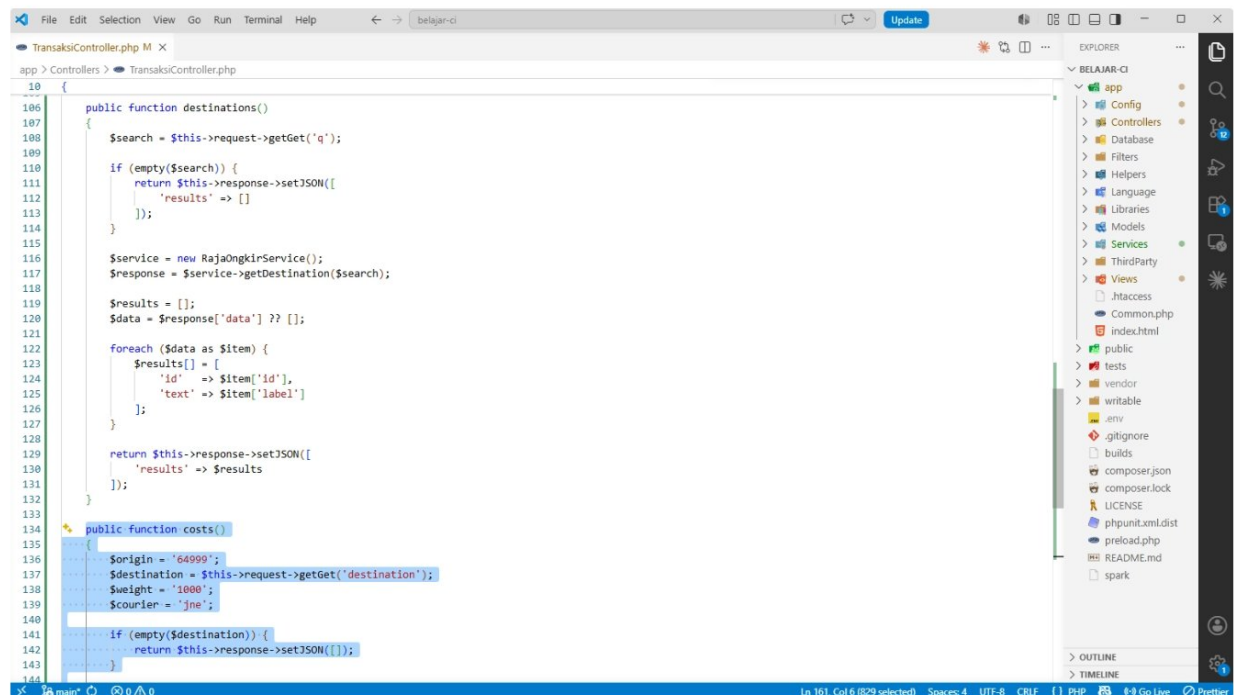
    $results = [];
    $data = $response['data'] ?? [];

    foreach ($data as $item) {
        $results[] = [
            'service'    => $item['service'],
            'description' => $item['description'],
            'cost'        => $item['cost'],
            'etd'         => $item['etd']
        ];
    }
}
```

```

    return $this->response->setJSON($results);
}

```



```

10 {
11     public function destinations()
12     {
13         $search = $this->request->getGet('q');
14
15         if (empty($search)) {
16             return $this->response->setJSON([
17                 'results' => []
18             ]);
19         }
20
21         $service = new RajaongkirService();
22         $response = $service->getDestination($search);
23
24         $results = [];
25         $data = $response['data'] ?? [];
26
27         foreach ($data as $item) {
28             $results[] = [
29                 'id' => $item['id'],
30                 'text' => $item['label']
31             ];
32         }
33
34         return $this->response->setJSON([
35             'results' => $results
36         ]);
37     }
38
39     public function costs()
40     {
41         $origin = '64999';
42         $destination = $this->request->getGet('destination');
43         $weight = '1000';
44         $courier = 'jne';
45
46         if (empty($destination)) {
47             return $this->response->setJSON([]);
48         }
49     }
50 }

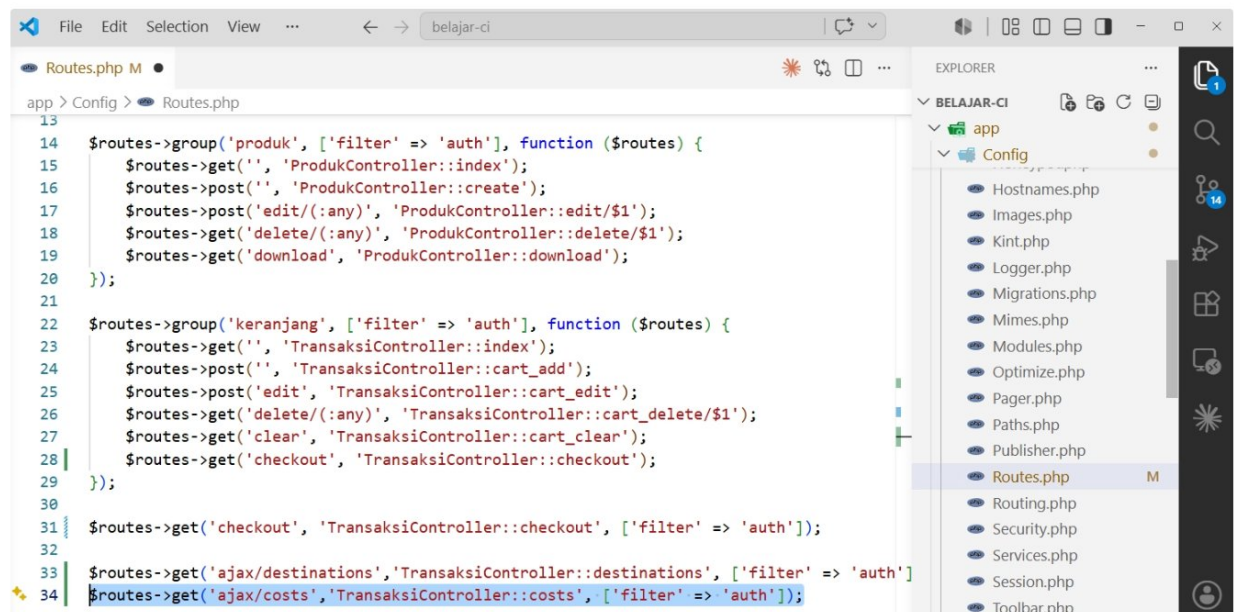
```

29. Tambahkan route untuk request GET daerah tujuan di file app\Config\Routes.php. Rute ini menggunakan fungsi costs() yang baru saja dibuat.

```

$routes->get('ajax/costs','TransaksiController::costs', ['filter' => 'auth'])

```



```

13
14 $routes->group('produk', ['filter' => 'auth'], function ($routes) {
15     $routes->get('', 'ProdukController::index');
16     $routes->post('', 'ProdukController::create');
17     $routes->post('edit/(:any)', 'ProdukController::edit/$1');
18     $routes->get('delete/(:any)', 'ProdukController::delete/$1');
19     $routes->get('download', 'ProdukController::download');
20 });
21
22 $routes->group('keranjang', ['filter' => 'auth'], function ($routes) {
23     $routes->get('', 'TransaksiController::index');
24     $routes->post('', 'TransaksiController::cart_add');
25     $routes->post('edit', 'TransaksiController::cart_edit');
26     $routes->get('delete/(:any)', 'TransaksiController::cart_delete/$1');
27     $routes->get('clear', 'TransaksiController::cart_clear');
28     $routes->get('checkout', 'TransaksiController::checkout');
29 });
30
31 $routes->get('checkout', 'TransaksiController::checkout', ['filter' => 'auth']);
32
33 $routes->get('ajax/destinations','TransaksiController::destinations', ['filter' => 'auth']);
34 $routes->get('ajax/costs','TransaksiController::costs', ['filter' => 'auth']);

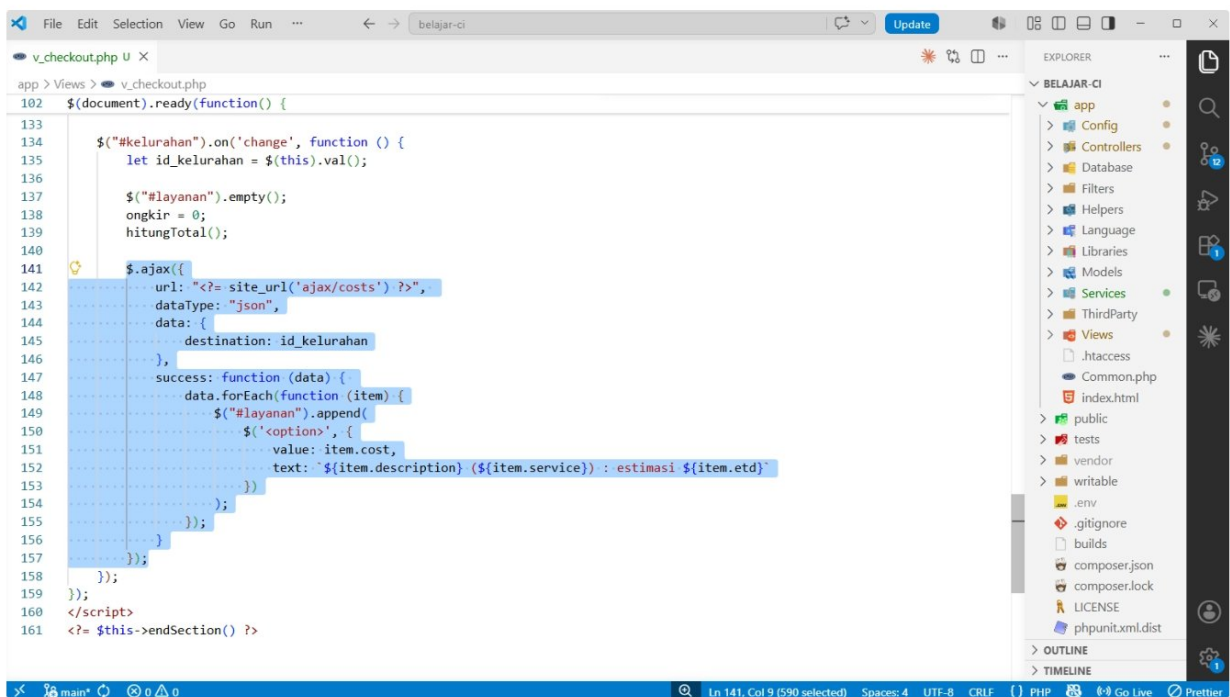
```



30. Tambahkan code AJAX pada event On Change elemen kelurahan untuk mengirimkan data destination (id_kelurahan) dan mendapatkan responsnya untuk ditampilkan pada elemen layanan.

Gunakan rute yang baru saja dibuat (<http://localhost:8080/ajax/costs>) pada URL tujuan AJAX.

```
$.ajax({
    url: "<?= site_url('ajax/costs') ?>",
    dataType: "json",
    data: {
        destination: id_kelurahan
    },
    success: function (data) {
        data.forEach(function (item) {
            $("#layanan").append(
                $('<option>', {
                    value: item.cost,
                    text: `${item.description} (${item.service}) : estimasi :
                })
            );
        });
    }
});
```



Refresh halaman lalu coba pilih sebuah daerah tujuan, maka layanan yang tersedia pada daerah tersebut akan masuk pada tampilan elemen layanan.

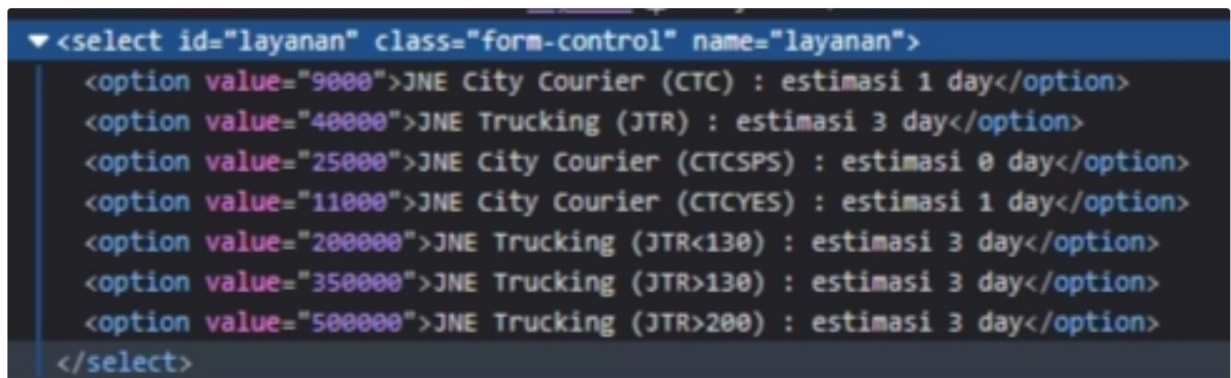
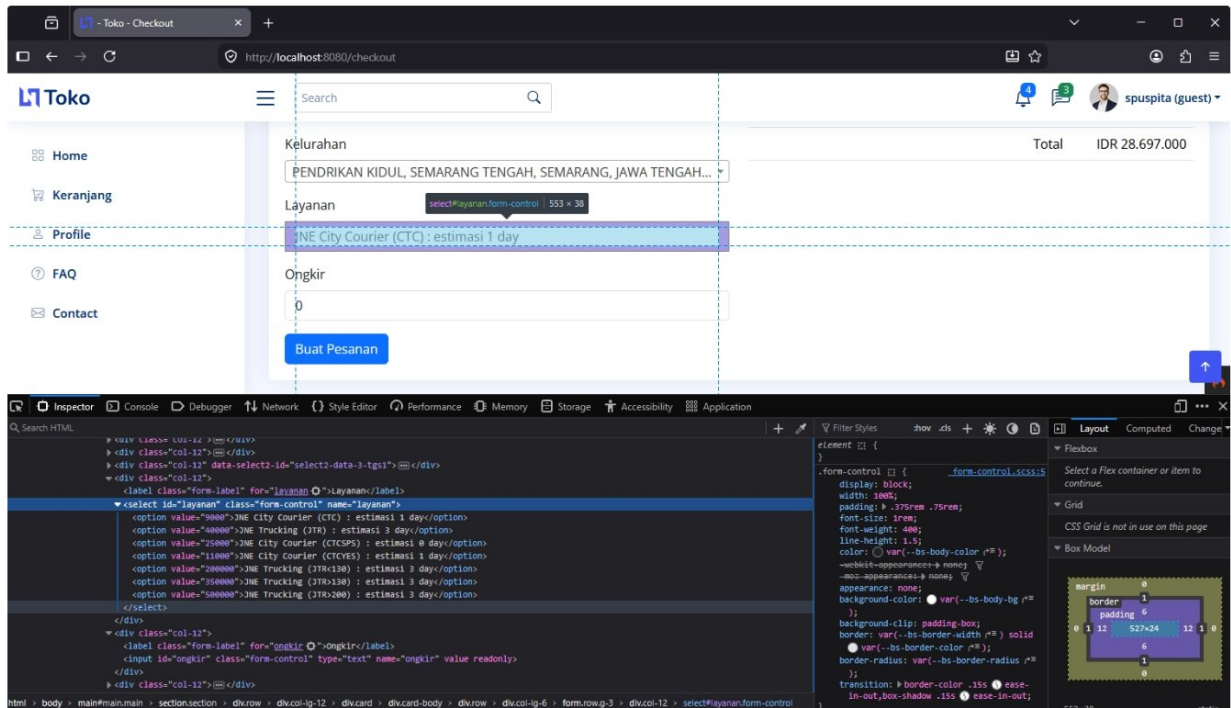
The screenshot shows the 'Toko - Checkout' page in a web browser. The page has a sidebar with links: Home, Keranjang, Profile, FAQ, and Contact. The main content area is titled 'Kelurahan' and shows a dropdown menu with the selected value 'PENDRIKAN KIDUL, SEMARANG TENGAH, SEMARANG, JAWA TENGAH...'. Below this, there are input fields for 'Layanan' (JNE City Courier (CTC) : estimasi 1 day) and 'Ongkir' (0). A 'Buat Pesanan' button is at the bottom. The total price is displayed as 'Total IDR 28.697.000'. The browser's developer console is open, showing a list of services and their costs:

Service	Description	Cost
0	(service: "CTC", description: "JNE City Courier", cost: 9000, ...)	9000
1	(service: "JTR", description: "JNE Trucking", cost: 40000, ...)	40000
2	(service: "CTCSPS", description: "JNE City Courier", cost: 25000, ...)	25000
3	(service: "CTCYES", description: "JNE City Courier", cost: 11000, ...)	11000
4	(service: "JTR<130", description: "JNE Trucking", cost: 200000, ...)	200000
5	(service: "JTR<130", description: "JNE Trucking", cost: 350000, ...)	350000
6	(service: "JTR>200", description: "JNE Trucking", cost: 500000, ...)	500000

The screenshot shows the 'Toko - Checkout' page in a web browser. The page has a sidebar with links: Home, Keranjang, Profile, FAQ, and Contact. The main content area is titled 'Checkout' and shows a dropdown menu with the selected value 'PENDRIKAN KIDUL, SEMARANG TENGAH, SEMARANG, JAWA TENGAH...'. Below this, there are input fields for 'Layanan' (JNE City Courier (CTC) : estimasi 1 day) and 'Ongkir' (0). A 'Buat Pesanan' button is at the bottom. The total price is displayed as 'Total IDR 28.697.000'. The browser's developer console is open, showing a list of services and their costs:

Service	Description	Cost
0	(service: "CTC", description: "JNE City Courier", cost: 9000, ...)	9000
1	(service: "JTR", description: "JNE Trucking", cost: 40000, ...)	40000
2	(service: "CTCSPS", description: "JNE City Courier", cost: 25000, ...)	25000
3	(service: "CTCYES", description: "JNE City Courier", cost: 11000, ...)	11000
4	(service: "JTR<130", description: "JNE Trucking", cost: 200000, ...)	200000
5	(service: "JTR<130", description: "JNE Trucking", cost: 350000, ...)	350000
6	(service: "JTR>200", description: "JNE Trucking", cost: 500000, ...)	500000

31. Tambahkan code javascript pada event On Change elemen layanan untuk mengambil nilai nominal ongkir sesuai dengan layanan yang dipilih user kemudian menampilkannya dalam kolom ongkir di bawahnya. Nominal ongkir ini tersimpan pada value di setiap option elemen dropdown layanan. Kemudian jalankan kembali fungsi hitungTotal() untuk menghitung ulang nominal total transaksi dengan menambahkan nominal ongkir terbaru.

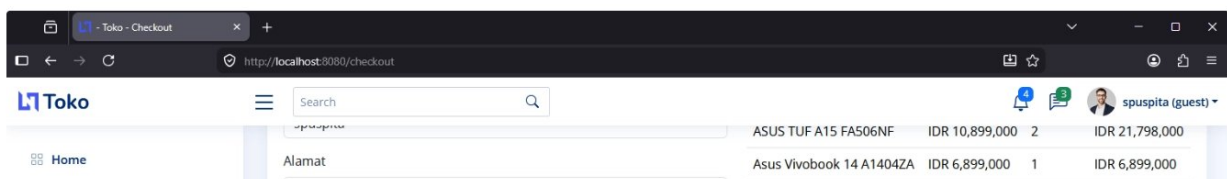
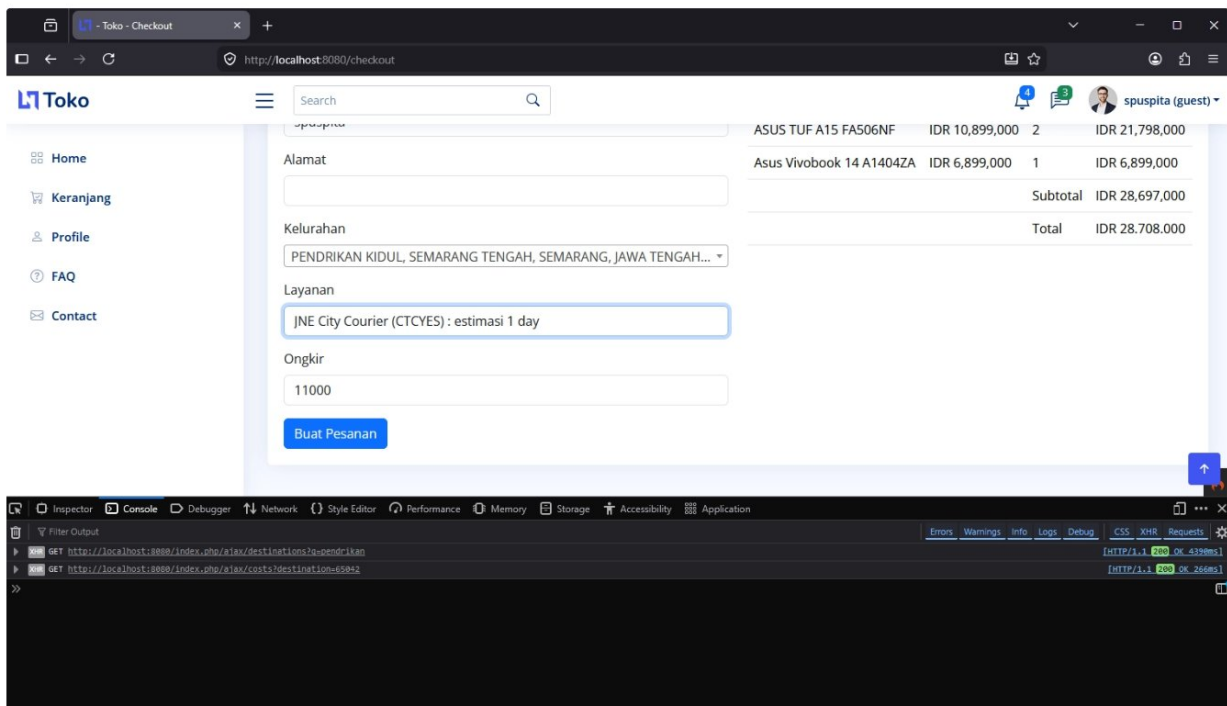


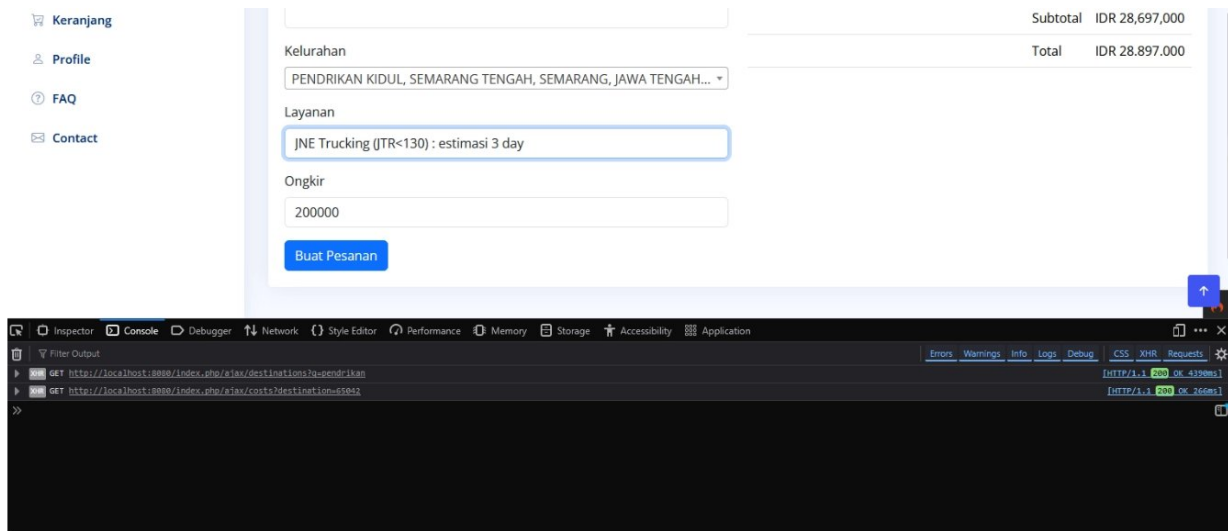
```
$("#layanan").on('change', function() {  
    ongkir = parseInt($(this).val());  
    hitungTotal();  
});
```



```
app > Views > v_checkout.php
102 $(document).ready(function() {
134   $("#kelurahan").on('change', function () {
135     let id_kelurahan = $(this).val();
136
137     $("#layanan").empty();
138     ongkir = 0;
139     hitungTotal();
140
141     $.ajax({
142       url: "<? site_url('ajax/costs') ?>",
143       dataType: "json",
144       data: {
145         destination: id_kelurahan
146       },
147       success: function (data) {
148         data.forEach(function (item) {
149           $("#layanan").append(
150             $('<option>', {
151               value: item.cost,
152               text: `${item.description} (${item.service}) : estimasi ${item.eta}`
153             });
154         });
155       }
156     });
157   });
158 });
159
160 $("#layanan").on('change', function() {
161   ongkir = parseInt($(this).val());
162   hitungTotal();
163 });
164 });
```

Refresh halaman kemudian pilih daerah tujuan dan layanan yang tersedia. Nilai Total di sebelah kanan halaman akan ikut berubah sesuai dengan layanan yang dipilih user.

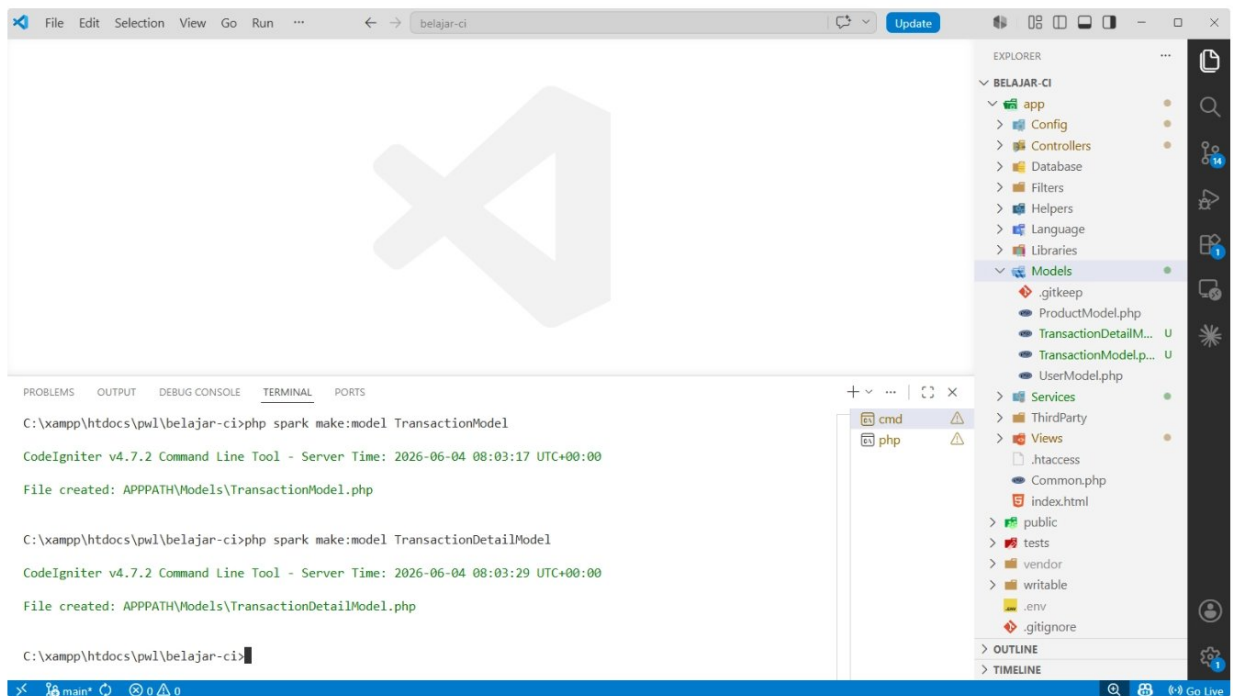




32. Data transaksi sudah siap untuk disimpan pada database. Buat dulu modelnya untuk table transaction dan transaction_detail.

```
php spark make:model TransactionModel
```

```
php spark make:model TransactionDetailModel
```



33. Sesuaikan beberapa variable pada TransactionModel dan TransactionDetailModel.

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class TransactionModel extends Model
{
    protected $table            = 'transaction'; //disesuaikan
    protected $primaryKey       = 'id';
    protected $useAutoIncrement = true;
    protected $returnType       = 'array';
    protected $useSoftDeletes   = true; //disesuaikan
    protected $protectFields    = true;
    protected $allowedFields    = ['username', 'total_harga', 'alamat', 'ongkir'];

    protected bool $allowEmptyInserts = false;
    protected bool $updateOnlyChanged = true;

    protected array $casts = [];
    protected array $castHandlers = [];

    // Dates
    protected $useTimestamps = true; //disesuaikan
    protected $dateFormat    = 'datetime';
    protected $createdField   = 'created_at';
    protected $updatedField   = 'updated_at';
    protected $deletedField   = 'deleted_at';

    // Validation
    protected $validationRules      = [];
    protected $validationMessages  = [];
    protected $skipValidation       = true; //disesuaikan
    protected $cleanValidationRules = true;

    // Callbacks
    protected $allowCallbacks = true;
    protected $beforeInsert    = [];
    protected $afterInsert     = [];
    protected $beforeUpdate    = [];
    protected $afterUpdate     = [];
```

```
protected $beforeFind      = [];  
protected $afterFind       = [];  
protected $beforeDelete   = [];  
protected $afterDelete     = [];  
}
```

```
<?php  
  
namespace App\Models;  
  
use CodeIgniter\Model;  
  
class TransactionDetailModel extends Model  
{  
    protected $table          = 'transaction_detail'; //disesuaikan  
    protected $primaryKey     = 'id';  
    protected $useAutoIncrement = true;  
    protected $returnType     = 'array';  
    protected $useSoftDeletes  = true; //disesuaikan  
    protected $protectFields   = true;  
    protected $allowedFields   = ['transaction_id', 'product_id', 'jumlah',  
  
    protected bool $allowEmptyInserts = false;  
    protected bool $updateOnlyChanged = true;  
  
    protected array $casts = [];  
    protected array $castHandlers = [];  
  
    // Dates  
    protected $useTimestamps = true; //disesuaikan  
    protected $dateFormat    = 'datetime';  
    protected $createdField   = 'created_at';  
    protected $updatedField   = 'updated_at';  
    protected $deletedField   = 'deleted_at';  
  
    // Validation  
    protected $validationRules      = [];  
    protected $validationMessages  = [];  
    protected $skipValidation       = true; //disesuaikan  
    protected $cleanValidationRules = true;
```

```

// Callbacks
protected $allowCallbacks = true;
protected $beforeInsert = [];
protected $afterInsert = [];
protected $beforeUpdate = [];
protected $afterUpdate = [];
protected $beforeFind = [];
protected $afterFind = [];
protected $beforeDelete = [];
protected $afterDelete = [];
}

```

34. Gunakan kedua model tersebut pada file `app\Controllers\TransaksiController.php`.

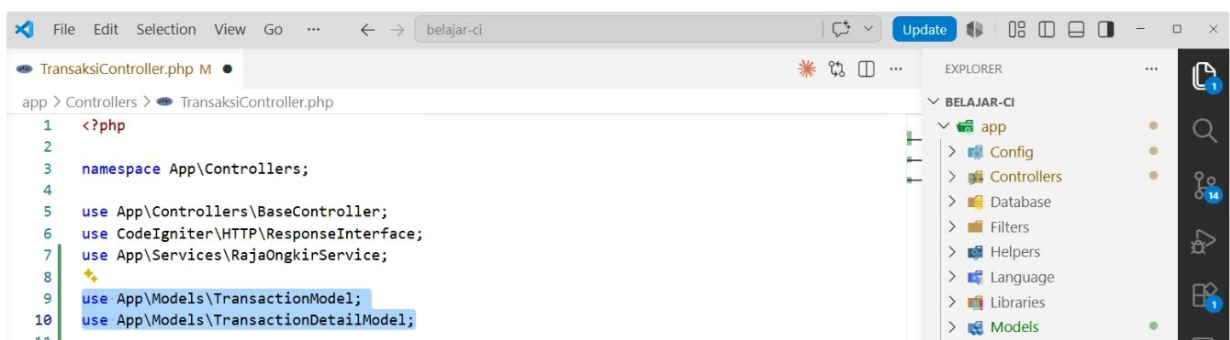
```

use App\Models\TransactionModel;
use App\Models\TransactionDetailModel;

protected $transactionModel;
protected $transactionDetailModel;

$this->transactionModel = new TransactionModel();
$this->transactionDetailModel = new TransactionDetailModel();

```



```
12
13 class TransaksiController extends BaseController
14 {
15     protected $cart;
16     protected $transactionModel;
17     protected $transactionDetailModel;
18
19     public function __construct()
20     {
21         helper(['number', 'form']);
22         $this->cart = service('cart');
23         $this->transactionModel = new TransactionModel();
24         $this->transactionDetailModel = new TransactionDetailModel();
25     }
26 }
```

35. Model sudah siap digunakan, kembali ke file `app\Views\v_checkout.php`, sesuai dengan action form pada halaman ini maka buat sebuah rute baru pada file `app\Config\Routes.php`.

Rute ini menggunakan fungsi `buy()` pada `TransaksiController` untuk menyimpan data transaksi.

```
1 <?= $this->extend('layout') ?>
2 <?= $this->section('content') ?>
3 <div class="row">
4     <div class="col-lg-6">
5         <?= form_open('buy', ['class="row g-3"]' ?>
6
7         <?= form_hidden('username', session()->get('username')) ?>
8
9         <?= form_input([
10             'type' => 'hidden',
11             'name' => 'total_harga',
12             'id' => 'total_harga']) ?>
13
14     <div class="col-12">
15         <?= form_label('Nama', 'nama', ['class' => 'form-label']) ?>
16         <?= form_input([
17             'name' => 'nama',
18             'id' => 'nama',
19             'class' => 'form-control',
20             'value' => session()->get('username'),
21             'readonly' => true]) ?>
```

```
$routes->post('buy', 'TransaksiController::buy', ['filter' => 'auth']);
```

```
11 $routes->post('login', 'AuthController::login');
12 $routes->get('logout', 'AuthController::logout');
13
14 $routes->group('produk', ['filter' => 'auth', function ($routes) {
15     $routes->get('', 'ProdukController::index');
16     $routes->post('', 'ProdukController::create');
```

```
17 $routes->post('edit/(:any)', 'ProdukController::edit/$1');
18 $routes->get('delete/(:any)', 'ProdukController::delete/$1');
19 $routes->get('download', 'ProdukController::download');
20 });
21
22 $routes->group('keranjang', ['filter' => 'auth'], function ($routes) {
23     $routes->get('', 'TransaksiController::index');
24     $routes->post('', 'TransaksiController::cart_add');
25     $routes->post('edit', 'TransaksiController::cart_edit');
26     $routes->get('delete/(:any)', 'TransaksiController::cart_delete/$1');
27     $routes->get('clear', 'TransaksiController::cart_clear');
28     $routes->get('checkout', 'TransaksiController::checkout');
29 });
30
31 $routes->get('checkout', 'TransaksiController::checkout', ['filter' => 'auth']);
32 $routes->post('buy', 'TransaksiController::buy', ['filter' => 'auth']);
33
34 $routes->get('ajax/destinations', 'TransaksiController::destinations', ['filter' => 'auth']);
35 $routes->get('ajax/costs', 'TransaksiController::costs', ['filter' => 'auth']);
```

36. Berdasarkan rute tadi, tambahkan fungsi buy() pada file app\Controllers\TransaksiController.php.

```
public function buy()
{
    $cartItems = $this->cart->contents();

    if (empty($cartItems)) {
        return redirect()->back();
    }

    $db = \Config\Database::connect();
    $db->transStart();

    $subtotal = 0;
    foreach ($cartItems as $item) {
        $subtotal += $item['qty'] * $item['price'];
    }

    $ongkir = (int) $this->request->getPost('ongkir');

    $transaction = [
        'username' => $this->request->getPost('username'),
        'alamat' => $this->request->getPost('alamat'),
        'ongkir' => $ongkir,
        'total_harga' => $subtotal + $ongkir,
        'status' => 0,
    ];

    // insert transaction
    if (!$this->transactionModel->insert($transaction)) {
```

```

        $db->transRollback();
        return redirect()->back()->with('error', 'Gagal membuat transaksi');
    }

    $transactionId = $this->transactionModel->getInsertID();

    // insert transaction detail
    foreach ($cartItems as $item) {
        $this->transactionDetailModel->insert([
            'transaction_id' => $transactionId,
            'product_id'     => $item['id'],
            'jumlah'         => $item['qty'],
            'diskon'         => 0,
            'subtotal_harga' => $item['qty'] * $item['price']
        ]);
    }

    $db->transComplete();

    if (!$db->transStatus()) {
        return redirect()->back()->with('error', 'Gagal membuat transaksi');
    }

    //hapus session keranjang belanja
    $this->cart->destroy();
    return redirect()->to(base_url());
}

```

Karena menyimpan ke 2 tabel (transaction dan transaction_detail), sebaiknya gunakan transaction database. Sehingga kalau insert detail gagal, data transaksi utama ikut dibatalkan.

Coba isi dengan lengkap data yang ada pada halaman checkout lalu klik tombol “Buat Pesanan”. Jika berhasil, di table transaction dan transaction_detail akan terisi data transaksi baru. Lalu tampilan menu keranjang akan kosong kembali isinya.

Checkout

Home / Checkout

Checkout

Nama:

Alamat:

Kelurahan:

Nama	Harga	Jumlah	Sub Total
ASUS TUF A15 FA506NF	IDR 10,899,000	2	IDR 21,798,000
Asus Vivobook 14 A1404ZA	IDR 6,899,000	1	IDR 6,899,000
Subtotal			IDR 28,697,000
Total			IDR 28,708,000

PENDRIKAN KIDUL, SEMARANG TENGAH, SEMARANG, JAWA TENGAH...

Layanan

JNE City Courier (CTCYES) : estimasi 1 day

Ongkir

11000

Buat Pesanan

© Copyright Toko. All Rights Reserved

Server: 127.0.0.1 » Database: db_c4 » Table: transaction

Showing rows 0 - 0 (1 total, Query took 0.0003 seconds)

SELECT * FROM `transaction`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	id	username	total_harga	alamat	ongkir	status	created_at	updated_at	deleted_at
☐ Edit ☐ Copy ☐ Delete	1	spuspta	28708000	Jl. Imam Bonjol 207	11000	0	2026-06-04 08:23:42	2026-06-04 08:23:42	NULL

Check all | With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

Print ☐ Copy to clipboard ☐ Export ☐ Display chart ☐ Create view

Bookmark this SQL query

Label: ☐ Let every user access this bookmark

Server: 127.0.0.1 » Database: db_c4 » Table: transaction_detail

Showing rows 0 - 1 (2 total, Query took 0.0006 seconds)

SELECT * FROM `transaction\_detail`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	transaction_id	product_id	jumlah	diskon	subtotal_harga	created_at	updated_at	deleted_at
☐ Edit ☐ Copy ☐ Delete	1	1	1	2	0	21798000	2026-06-04 08:23:42	2026-06-04 08:23:42	NULL
☐ Edit ☐ Copy ☐ Delete	2	1	2	1	0	6899000	2026-06-04 08:23:42	2026-06-04 08:23:42	NULL

Check all | With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

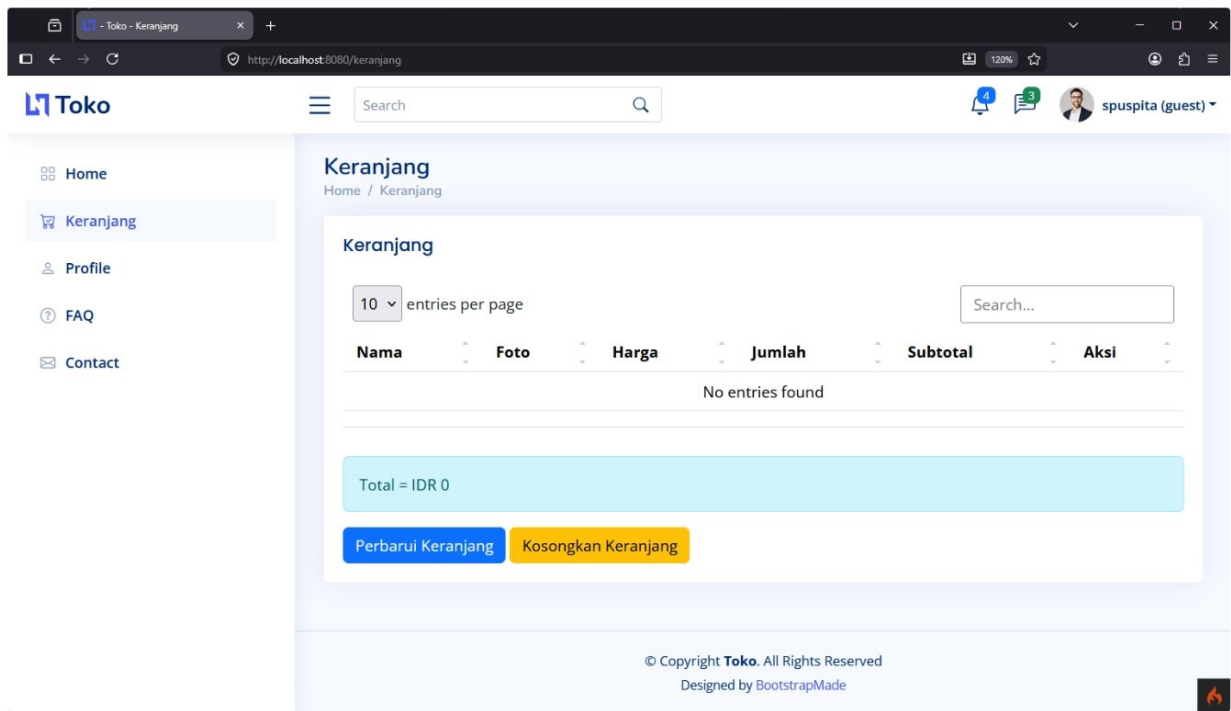
Query results operations

Print ☐ Copy to clipboard ☐ Export ☐ Display chart ☐ Create view

Bookmark this SQL query

Label: ☐ Let every user access this bookmark





37. Karena pada tugas sebelumnya sudah melakukan konfigurasi sehingga project Toko sudah terhubung dengan repository pada GitHub. Maka setiap selesai coding dan berhasil dijalankan